# A Cross-Scenario Survey and Analysis of Self-Evolving Multimodal Agents

**Jingjing Jiang**

Recent work shows that multimodal agents increasingly self-improve without recurring human annotation. However, each self-improvement mechanism is designed and evaluated within a single scenario, leaving its preconditions unverified under cross-scenario transfer. This survey investigates the structural source of this transfer asymmetry across four scenarios: multimodal reasoning, multimodal generation, GUI and Web navigation, and embodied manipulation. We first formalize the self-evolving multimodal agent by four criteria and a differentia of self-reference, and adopt the three-stage self-evolving loop of experience generation, feedback acquisition, and update consolidation as our analytical framework to position the scenarios along ceilings on rollout count and feedback fidelity. Comparing self-evolving with fixed loops within each scenario separates the limitations that the scenario imposes from those inherent in the self-evolving design. We then recast each mechanism as a trade-off that raises a scarce factor by presupposing a task-structural resource, such as output equivalence or checkable outcomes, and judge it portable, conditional, blocked, or redundant across scenarios. We attribute the asymmetry to divergent combinations of scarce factors and resources across opposite transfer directions. These verdicts remain untested, and we identify matched-budget measurements that could overturn them.

**Homepage:** https://mm-vl.github.io/SE-MMAgents
**Email:** jingjingjiang2017@gmail.com

## 1 Introduction

Recent advances in multimodal foundation models have fueled an explosive proliferation of multimodal agents [1, 2, 3, 4], spawning a flourishing spectrum of self-evolution techniques across multimodal reasoning [5, 6, 7, 8], multimodal generation [9, 10, 11, 12, 13], GUI and Web navigation [14, 15, 16, 17], and embodied manipulation [18, 19, 20, 21]. Despite strong scenario-specific gains, existing self-evolving methods are designed and evaluated within single scenarios, and their mechanisms do not transfer equally well in both directions of a scenario pair. For example, majority voting derives label-free rewards for reasoning tasks from consistency across sampled answers [5, 22, 23], yet finds no counterpart in embodied manipulation, where action trajectories admit no equivalence check and physical rollouts incur steep execution and reset costs. Conversely, world models circumvent expensive environmental interactions via imagined rollouts [24, 25], but raise a factor that is not scarce in reasoning, where sampling directly from the policy costs little. Beneath this directional breakdown in mechanism transferability, which suggests a *cross-scenario asymmetry*, also lies a shared self-evolving loop across diverse tasks. Specifically, across all four scenarios, self-evolving agents share the same cycle, *i.e.*, generating exploratory experience, acquiring evaluative feedback, and consolidating persistent updates. This survey investigates the structural source of cross-scenario asymmetry by characterizing this shared loop and explaining why the two directions of a scenario pair meet different obstacles to transfer.

However, as summarized in Table 1, existing work leaves this cross-scenario asymmetry unaddressed. Primary studies benchmark exclusively within specific scenarios [20, 40], while prior surveys either overlook the self-evolving loop as an analytical object [1, 2, 3, 41, 42] or formalize it as scenario-agnostic [28, 31, 33]. Even the closest related work [29] attributes verifiable feedback and retraining costs to domain-level paradigms, not to specific mechanisms. To fill this gap, we first formalize the surveyed object through *a genus and a differentia*. The genus comprises autonomous learning loops satisfying four criteria: (*i*) *autonomy*, where the agent operates without human intervention across goals, actions, feedback, and updates post-initialization; (*ii*) *freshness*, where experience is generated under the current policy; (*iii*) *durability*, where updates persist beyond the episode boundary; and (*iv*) *recurrence*, where the loop iterates across cycles. The differentia, *self-reference*, requires the agent to realize at least one component of its own loop: the goal sampler, the feedback function, or the evolution operator. Loops in the genus lacking self-reference, termed *fixed loops*, serve as controls across all scenarios to disentangle scenario-imposed constraints from the intrinsic effects of self-evolution.

To locate where a mechanism can transfer, we adopt **the three-stage self-evolving loop** as our analytical framework, decomposing each cycle into experience generation, feedback

TABLE 1: **Comparison with related surveys.** *Scenarios* indicates domains covered in dedicated sections: MR (multimodal reasoning), MG (multimodal generation), GW (GUI and Web navigation), and EM (embodied manipulation), or General if scenario-agnostic. *Loop decomposition* denotes the stages or components formalized by each survey, where EG, FA, and UC represent experience generation, feedback acquisition, and update consolidation, respectively (– indicates none).

| Survey | Scenarios | Loop decomposition | Focus |
|---|---|---|---|
| ✣ *Multimodal agents* | | | |
| Durante *et al.* [1] arXiv'24 | EM, games, healthcare, video, NLP | – | Agent AI paradigm and applications |
| Xie *et al.* [2] arXiv'24 | MG, GW, EM, games, driving, video | – | Components and planner types |
| Hu *et al.* [26] ACL'25 | GW | – | Construction and evaluation of OS agents |
| Yao *et al.* [3] arXiv'25 | GW, EM, deep research, healthcare, driving, recommendation | – | Agentic capabilities of MLLMs |
| Huang *et al.* [27] arXiv'26 | MG | – | Agentic control of visual generation |
| ✣ *Self-evolving and adapting agents* | | | |
| Fang *et al.* [28] arXiv'25 | Biomedicine, software, finance, law | Inputs / agent system / environment / optimizer | Prompt, memory, tool, and multi-agent optimization |
| Jiang *et al.* [29] arXiv'25 | GW, deep research, software, drug discovery | Agent vs. tool × signal source | Adaptation by post-training, memory, and skills |
| Chen *et al.* [30] arXiv'26 | General | Improvement target × loop closure | Recursive self-improvement |
| Gao *et al.* [31] TMLR'26 | GW, software, finance, healthcare, education | What / when / how / where | Self-evolution strategies |
| Pu *et al.* [32] arXiv'26 | General | Diagnosis / proposal / evaluation / update | Trustworthiness of recursive self-improvement |
| Ren *et al.* [33] arXiv'26 | GW, EM, software, games, science | Model vs. scaffolding × signal source | Model and scaffolding self-improvement |
| Xiang *et al.* [34] TechRxiv'26 | Science, software, open-world simulation | Model-centric / environment-centric / co-evolution | Environment-driven co-evolution |
| Xu *et al.* [35] arXiv'26 | General | Typed graph state / schema-constrained rewrites | Self-evolution as graph transformation |
| Zhang *et al.* [36] TMLR'26 | GW, EM, deep research, software, math, vision | State / action / transition / reward | Agentic reinforcement learning |
| ✣ *Self-evolving multimodal models and agents* | | | |
| Deng *et al.* [37] EMNLP Findings'25 | General | Collection / organization / optimization | Self-improvement of MLLMs |
| Feng *et al.* [38] arXiv'26 | EM, driving, aerial | Memory / task / environment / embodiment / model | Self-evolving embodied AI |
| Guo *et al.* [39] arXiv'26 | General | Formation × management tier | Multimodal agent memory |
| **Ours** | **MR, MG, GW, EM** | **EG / FA / UC** | **Cross-scenario portability** |

acquisition, and update consolidation. We formalize each stage's bottleneck through two governing factors: (*i*) rollout count and information gain, (*ii*) feedback fidelity and credit localization, and (*iii*) retention and generalization. Each scenario imposes ceilings on the rollouts affordable per cycle ($\bar{K}$) and the feedback fidelity attainable by admissible evaluators ($\bar{F}$), which span a two-dimensional *positioning matrix*. In this matrix, multimodal reasoning is situated where both ceilings are high, multimodal generation where $\bar{F}$ is low, GUI and Web navigation where $\bar{K}$ is low, and embodied manipulation where both are low, with simulation raising both ceilings. Comparing self-evolving and fixed loops within each cell further separates the limits imposed by scenario ceilings from those incurred by self-evolving design, *e.g.*, replacing answer keys with sampled consensus in reasoning [5, 22]. The matrix thus identifies which ceilings bind in each scenario, but alone cannot determine whether a given mechanism is portable.

Building on this matrix, we formulate each self-evolving mechanism as a *trade-off*, in which the agent raises a scarce factor by replacing a fixed component with its own records, proposals, judgments, or memory writes, at the price of presupposing a task-structural resource, such as output equivalence or checkable outcomes. In a target scenario, a mechanism is thus *portable* if the target holds this resource, *conditional* if it holds the resource only for a sub-family or after adaptation, *blocked* if the outcome to be judged is unobservable, and *redundant* if the raised factor is not scarce. We attribute the cross-scenario asymmetry structurally to the different combinations of scarce factors and resources that the two directions of a scenario pair encounter: *e.g.*, voting finds no output equivalence in manipulation, whereas world models raise a factor that is not scarce in reasoning. These verdicts, inferred from single-scenario evidence and not yet tested, define testable preconditions, not empirical guarantees. A gain over a budget-matched ablation without self-evolution would overturn any verdict that predicts no benefit from an unadapted transfer, which turns these predictions into a concrete measurement agenda.

This paper makes four contributions, the first two to the survey and the last two to the analysis.

- ✣ **A condition-based definition and paradigm demarcation.** We operationalize autonomy and the differentia through a provenance matrix that records which party realizes each loop component, and apply the definition to sixteen neighboring paradigms, of which five fall outside the genus, two never satisfy the differentia, five are divided by it, and four are self-evolving by definition.
- ✣ **A stage-resolved review and positioning matrix.** We review each scenario through the three-stage self-evolving loop, place the four scenarios in the positioning matrix, contrast self-evolving with fixed loops in each cell, and trace five degradation modes along the cross-cycle paths through which they propagate.
- ✣ **Directed portability across scenario pairs.** We recast the mechanisms of all four scenarios as trade-offs and assign each a portability verdict for every target scenario, yielding a structural account of the cross-scenario asymmetry.
- ✣ **A falsifiable measurement agenda.** We review the evaluation of task-solving capability, self-evolving capability, and judges, and identify the measurements whose absence

leaves untestable three claims, *i.e.*, that a gain stems from self-evolution, that a scenario occupies its assigned position, and that the verdicts hold.

The remainder of this paper is organized as follows. Sec. 2 defines the surveyed object, and Sec. 3 develops the three-stage loop and the positioning matrix. Secs. 4 to 7 review the four scenarios, and Sec. 8 derives the portability verdicts. Finally, Sec. 9 treats evaluation, and Secs. 10 and 11 discuss open challenges and conclude.

## 2 Preliminaries

Conceptually, a *multimodal agent* is a goal-directed decision-making system that interacts with an external environment through a closed perception-cognition-action loop over heterogeneous observation and scenario-specific action spaces [43]. We restrict the scope to agents whose cognitive core is a large (multimodal) foundation model [44, 45, 46, 47, 48, 49, 50, 51] extended via interface [52, 53, 54], memory [55], and tools [56, 57, 58, 59], excluding architectures lacking such a core, such as world-model planners and task-specific visuomotor policies. Since the environment supplies perceptions, accepts actions, and specifies goals, we adopt *multimodal agent-environment interaction* as our formal scaffold. To systematically diagnose the cross-scenario asymmetry, we unfold this loop across two temporal scales: a *micro-scale* capturing step-by-step behaviors within a single episode, and a *macro-scale* tracking the cycle-by-cycle evolution of the agent's own components over long-term experiences. The resulting dual-scale dynamics are formalized under goal-conditioned partially observable Markov decision processes (POMDPs) [60, 61]. On this scaffold, we define the surveyed object across four levels, from the static agent and environment through the learning loop and the autonomous learning loop to the self-evolving loop.

### 2.1 Multimodal Agent

We formulate the static architecture of a multimodal agent as a four-component tuple $\mathcal{A}$ (Definition 1) and its environment as a goal-conditioned, modality-decomposed POMDP $\mathcal{P}_g$ (Definition 2). The multimodality condition of Definition 1 requires $\Phi_\theta$ to directly encode or decode a non-textual channel without intermediate text conversion, such as screen pixels, camera frames, generated images or videos, or low-level motor commands. Under this criterion, an agent perceiving a web page solely through its HTML or accessibility tree is text-only despite rendered visual content.

**Micro-Scale Interaction Loop.** Within one episode, shown in the micro-scale panel of Figure 1, the agent acts under a configuration $\mathcal{A}$ with its persistent components $(\Phi_\theta, \mathcal{M}_{\text{long}}, \mathcal{T}, \mathcal{I})$ held fixed, leaving only the short-term buffer $\mathcal{M}_{\text{short}}$ to evolve from initial state $m_0$. The episode starts from $s_0 \sim \mu_0(\cdot \mid g)$, runs over $t \in \{0, \dots, T-1\}$, and terminates upon a STOP action or horizon limit $T_{\max}$, yielding a random stopping time $T \le T_{\max}$. Each step passes through the environment kernels and agent components:

$$s_t \xrightarrow{Z} o_t \xrightarrow{\phi(\cdot, m_t, g)} x_t \xrightarrow{\Phi_\theta^\star} y_t \xrightarrow{\rho} u_t \xrightarrow{P} s_{t+1}, \tag{1}$$

where $\phi$ and $\rho$ are deterministic, $Z$, $\Phi_\theta^\star$, and $P$ are stochastic, and the buffer updates as $m_{t+1} = \Psi(m_t, o_t, u_t)$. The fixed argument $\mathcal{M}_{\text{long}}$ in $\phi$ is suppressed per episode. This synchronous discrete-time chain abstracts away the gap between decision and control frequencies, which is immaterial for text actions but adds per-rollout cost under real-time embodied control. Over the micro-history $h_t = (o_{0:t}, u_{0:t-1})$, the agent induces the policy $\pi_{\mathcal{A}} = \rho_\sharp \Phi_\theta^\star$,

$$\pi_{\mathcal{A}}(u_t | h_t, g) = \Phi_\theta^\star\big(\rho^{-1}(u_t) \,\big|\, \phi(o_t, m_t, g)\big), \tag{2}$$

where conditioning on $h_t$ suffices, fixed $(\mathcal{A}, m_0)$ and update $\Psi$ uniquely determining $m_t$ from $h_t$. Compounded with environment transitions, this policy induces a trajectory distribution $p_g^{\pi_{\mathcal{A}}}(\tau)$ over $\tau = (s_{0:T}, o_{0:T-1}, u_{0:T-1})$:

$$p_g^{\pi_{\mathcal{A}}}(\tau) = \mu_0(s_0 \mid g) \prod_{t=0}^{T-1} Z(o_t | s_t) \pi_{\mathcal{A}}(u_t | h_t, g) P(s_{t+1} | s_t, u_t). \tag{3}$$

**Goal-Conditioned Expected Return.** The loop induces the goal-conditioned expected return

$$J(\mathcal{A};\, g) = \mathbb{E}_{\tau \sim p_g^{\pi_{\mathcal{A}}}} \left[ \sum_{t=0}^{T-1} \gamma^t\, R(s_t,\, u_t,\, g) \right], \tag{4}$$

a performance functional of $\mathcal{A}$, not a quantity optimized within an episode. Self-evolution modifies $\mathcal{A}$ between episodes, as the macro-scale loop below formalizes.

### 2.2 Self-Evolving Multimodal Agent

Informally, a self-evolving multimodal agent improves its persistent components across cycles without human input post-initialization, learns from experience generated under its own policy, and internally realizes at least one improvement driver: selecting its goals, deriving its feedback, or authoring its updates. We formalize the agent and its environment as the triples $\mathcal{E}_{\mathcal{A}}$ (Definition 3) and $\mathcal{E}_{\mathcal{E}}$ (Definition 4).

**Macro-Scale Learning Loop.** Across episodes, the macro-scale frame (Figure 1) wraps interactions into agent-updating cycles. Starting from $\mathcal{A}_0$, each cycle $k$ passes through goal sampling, rollout, evaluation, and update:

$$\mathcal{H}_k \xrightarrow{\sigma_\omega} \boldsymbol{g}_k \xrightarrow{p_{\boldsymbol{g}_k}^{\pi_{\mathcal{A}_k}}} \mathcal{D}_k \xrightarrow{V_\nu} V_\nu(\mathcal{D}_k) \xrightarrow{f_\zeta} \mathcal{A}_{k+1}, \tag{5}$$

where $\boldsymbol{g}_k$ denotes the goal batch drawn from $\sigma_\omega(k, \mathcal{H}_k)$ with support $\mathcal{G}_k \subseteq \mathcal{G}$, rollout law $p_{\boldsymbol{g}_k}^{\pi_{\mathcal{A}_k}}$ applies $p_g^{\pi_{\mathcal{A}_k}}$ across each $g \in \boldsymbol{g}_k$, and $\mathcal{D}_k$ aggregates the resulting goal-trajectory pairs (with single goals corresponding to unit batch size). The history then extends to $\mathcal{H}_{k+1} = (\mathcal{A}_{0:k+1}, \mathcal{D}_{0:k}, V_\nu(\mathcal{D}_{0:k}))$, recording realized scores, stochastic evaluator outputs being non-recoverable from trajectories alone and failure-conditioned samplers conditioning directly on realized values. We designate this recursive mapping a *learning loop*. The term denotes the cross-cycle structure without presupposing monotonic gains in $J(\mathcal{A}_k; \mu_{\mathcal{G}})$, and excludes within-episode adaptation (*e.g.*, in-context learning), which remains confined to $\mathcal{M}_{\text{short}}$. A *cycle* is one iteration of this mapping. When training proceeds in explicit rounds (each regenerating tasks, evaluator states, or rollouts via the antecedent agent before optimization), a cycle corresponds to one round, whose internal optimizer steps form *within-cycle optimization* under $f_\zeta$. In round-free regimes, such as online reinforcement learning drawing fresh rollouts at every optimizer step, each step executes the full mapping as a *degenerate cycle*.

**Definition 1 (Multimodal Agent)**

A *multimodal agent* over observation, action, and goal spaces $(\mathcal{O}, \mathcal{U}, \mathcal{G})$ is specified by the four-component tuple

$$\mathcal{A} \;=\; (\,\Phi_\theta,\; \mathcal{M},\; \mathcal{T},\; \mathcal{I}\,),$$

whose components are the following.

- $\Phi_\theta : (\mathbb{R}^d)^* \to \Delta(\widetilde{\mathcal{V}} \sqcup \mathcal{Z}_{\text{latent}})$ is the **core foundation model**, a next-step distribution over a discrete multimodal vocabulary $\widetilde{\mathcal{V}}$ and a latent space $\mathcal{Z}_{\text{latent}} \subseteq \mathbb{R}^{d'}$ for diffusion or flow-matching heads, whose iterated sampling defines the sequence-level distribution $\Phi_\theta^\star$.
- $\mathcal{M} = \mathcal{M}_{\text{short}} \times \mathcal{M}_{\text{long}}$ is the **memory**, an episode-scoped buffer $\mathcal{M}_{\text{short}} \subseteq (\mathbb{R}^d)^*$ and a persistent key-indexed store $\mathcal{M}_{\text{long}} \subseteq \big(\mathbb{R}^d \times (\mathbb{R}^d)^*\big)^*$. A write operator $\Psi : \mathcal{M} \times \mathcal{O} \times \mathcal{U} \to \mathcal{M}_{\text{short}}$ updates the buffer within an episode, and only the evolution operator of Definition 3 writes $\mathcal{M}_{\text{long}}$.
- $\mathcal{T} = \{\eta_j : \mathcal{X}_j \to \Delta(\mathcal{Y}_j)\}_{j=1}^{|\mathcal{T}|}$ is the **tool set**, a registry of typed callable interfaces. Tool actions $\mathcal{U}_{\text{tool}}$ are calls of a fixed form, a tool name with its arguments, $\mathcal{T}$ fixes which calls execute, and their execution effects enter the transition $P$ of Definition 2.
- $\mathcal{I} = (\phi, \rho)$ is the **interaction interface**, a prompt assembler $\phi : \mathcal{O} \times \mathcal{M} \times \mathcal{G} \to (\mathbb{R}^d)^*$ that embeds heterogeneous context into one sequence and an action parser $\rho : (\widetilde{\mathcal{V}} \sqcup \mathcal{Z}_{\text{latent}})^* \to \mathcal{U}$ that decodes a generated sequence into an action.

The agent is multimodal when $\mathcal{O}$ or $\mathcal{U}$ contains a non-textual channel that $\Phi_\theta$ encodes or decodes natively.

**Definition 2 (Multimodal Environment)**

Under a goal $g \in \mathcal{G}$, the environment of a multimodal agent is the goal-conditioned POMDP

$$\mathcal{P}_g \;=\; (\,\mathcal{S},\; \mathcal{O},\; \mathcal{U},\; \mathcal{G},\; \mu_0,\; P,\; Z,\; R,\; \gamma\,),$$

with latent state space $\mathcal{S}$, initial-state distribution $\mu_0(\cdot \mid g) \in \Delta(\mathcal{S})$, transition kernel $P : \mathcal{S} \times \mathcal{U} \to \Delta(\mathcal{S})$ that includes tool-execution effects, observation function $Z : \mathcal{S} \to \Delta(\mathcal{O})$, goal-conditioned reward $R : \mathcal{S} \times \mathcal{U} \times \mathcal{G} \to \mathbb{R}$, which we treat as the idealized task objective, and discount $\gamma \in [0, 1]$, which the horizon cap $T_{\max}$ keeps well defined at $\gamma = 1$.

The interface spaces are multimodal.

- $\mathcal{O} = \mathcal{O}_{\text{vis}}^{\perp} \times \mathcal{O}_{\text{txt}}^{\perp} \times \mathcal{O}_{\text{aud}}^{\perp} \times \mathcal{O}_{\text{prop}}^{\perp}$ ranges over visual, textual, auditory, and proprioceptive or tactile channels, where $\mathcal{X}^{\perp} := \mathcal{X} \cup \{\perp\}$ and $\perp$ marks a channel absent at a step.
- $\mathcal{U} = \mathcal{U}_{\text{text}} \sqcup \mathcal{U}_{\text{gen}} \sqcup \mathcal{U}_{\text{act}} \sqcup \{\textsc{stop}\}$ admits one category per step, text generation, artifact synthesis, or digital and physical intervention $\mathcal{U}_{\text{act}} = \mathcal{U}_{\text{tool}} \sqcup \mathcal{U}_{\text{gui}} \sqcup \mathcal{U}_{\text{motor}}$.
- $\mathcal{G}$ contains multi-channel goals, such as a language constraint paired with a target image or pose, together with any task parameter, such as an initial scene, that a goal fixes.

**Meta-Operators and Substrate.** We designate $\sigma_\omega$, $V_\nu$, and $f_\zeta$ the three meta-operators of the loop, each acting on whole episodes at the macro scale. Across multimodal scenarios, the goal sampler draws from a fixed task pool [62], poses questions about unlabeled images [5], reselects GUI tasks by historical success [17], or synthesizes simulated scenes [63]. The feedback function verifies discrete answers [64], inspects generated images via the internal understanding pathway [10], queries a vision-language judge on the final screen [14], or evaluates simulator success predicates [62]. The evolution operator updates $\Phi_\theta$ via policy gradient [65] or writes lessons, skills, and memory entries [66, 67, 68]. Auxiliary agents enter either as self-play opponents sampled via the task argument (Definition 4) [7] or as learned proposer-solver roles instantiated within $\Phi_\theta$ [22]. The *substrate* denotes the persistent components that $f_\zeta$ modifies across cycles [28, 31]. Specifically, a *parametric* loop updates $\Phi_\theta$ alone, a *non-parametric* loop modifies only $\mathcal{M}_{\text{long}}$, $\mathcal{T}$, or $\mathcal{I}$ under fixed $\theta$ (training-free), and a *mixed* loop alters both [69, 70].

**Expected Evaluation Return.** We evaluate a learning loop at each cycle via its expected return under the evaluation distribution $\mu_{\mathcal{G}}$, which lies outside the control of $\sigma_\omega$ to prevent samplers from inflating performance through easier goals. With cycles counted in rounds by some methods and in optimizer steps by others, $n$ cycles span horizons of different lengths, and the return is therefore reported per cycle. With $\mathcal{A}_k$ itself random, depending on the goals, rollouts, and feedback recorded in $\mathcal{H}_k$, the expectation at cycle $k$ also ranges over this history,

$$\mathcal{J}_k = \mathbb{E}_{\mathcal{H}_k,\, g \sim \mu_{\mathcal{G}}}\big[\, J(\mathcal{A}_k;\, g)\,\big], \quad k \in \mathbb{N}. \tag{6}$$

This sequence evaluates against the idealized reward $R$ (Eq. (4)), whereas the loop ascends the proxy $V_\nu$.

## 2.3 Defining Conditions of Self-Evolving Loops

To distinguish it from general learning loops, we formalize the self-evolving loop through a genus of four operational criteria and a differentia of self-reference.

**The Genus: Autonomous Learning Loops.** We adopt four criteria to bound the genus of autonomous learning loops. Autonomy bars human input across all three meta-operators and rollout actions, whereas freshness, durability, and recurrence respectively exclude static datasets, episodic buffer writes, and uniterated single-shot updates:

$\mathcal{C}_1$ **Autonomy.** Autonomy requires the loop to operate without human intervention across its meta-operators and action stream post-initialization. Formally, at every

**Definition 3 (Self-Evolving Multimodal Agent)**

Let $\mathcal{A}_k = (\Phi_{\theta_k}, \mathcal{M}_k, \mathcal{T}_k, \mathcal{I}_k)$ denote the agent at cycle $k \in \mathbb{N}$. A *self-evolving multimodal agent* is specified by the triple

$$\mathcal{E}_{\mathcal{A}} \;=\; (\,\mathcal{A}_0,\; V_\nu,\; f_\zeta\,),$$

whose components are the following.

- $\mathcal{A}_0 \in \mathbb{A}^{\mathrm{mm}}$ is the **initial agent**, with $\mathbb{A}^{\mathrm{mm}}$ the set of configurations of Definition 1.
- $V_\nu : (\mathbb{D}^{\mathrm{mm}})^* \times (\mathbb{A}^{\mathrm{mm}})^* \to (\mathbb{Y}^{\mathrm{mm}})^*$ is the **feedback function** with configuration $\nu$. It maps a batch of goal-trajectory pairs $(g, \tau) \in \mathbb{D}^{\mathrm{mm}}$, with $\tau$ distributed as in Eq. (3), together with the agent history $\mathcal{A}_{0:k}$, to feedback in $\mathbb{Y}^{\mathrm{mm}}$, which subsumes scalar rewards, preferences, and step-level, token-level, or structured critiques, and we write $V_\nu(\mathcal{D}_k)$ for $V_\nu(\mathcal{D}_k, \mathcal{A}_{0:k})$. It is *agent-independent* when the preference it induces over trajectories that share a goal depends only on those trajectories and on artifacts fixed before the loop other than copies of the agent's model, and it *reads the current agent* when that preference depends on $\mathcal{A}_k$, through the agent's model or through other samples of its current policy.
- $f_\zeta : \mathbb{A}^{\mathrm{mm}} \times \mathbb{H} \times (\mathbb{D}^{\mathrm{mm}})^* \times (\mathbb{Y}^{\mathrm{mm}})^* \to \mathbb{A}^{\mathrm{mm}}$ is the **evolution operator** with configuration $\zeta$, which writes the update of a cycle into the agent tuple and reads the history $\mathcal{H}_k$ of Definition 4 when it replays earlier experience.

A proposer or evaluator that the agent system trains belongs to $\Phi_\theta$, which places it under the updates of $f_\zeta$ and makes $\sigma_\omega$ and $V_\nu$ depend on it through $\mathcal{A}_k$. The criteria $\mathcal{C}_1$ to $\mathcal{C}_4$ and the differentia below decide whether the loop of the triple with the environment of Definition 4 is an autonomous learning loop and whether that loop is self-evolving, and only a triple whose loop passes all five tests is a self-evolving multimodal agent.

**Definition 4 (Multimodal Macro-Environment)**

Let $\mathcal{H}_k = (\mathcal{A}_{0:k}, \mathcal{D}_{0:k-1}, V_\nu(\mathcal{D}_{0:k-1})) \in \mathbb{H}$ denote the history of agents, trajectories, and realized scores at the start of cycle $k$. The **multimodal macro-environment** is the triple

$$\mathcal{E}_{\mathcal{E}} \;=\; (\,\{\mathcal{P}_g\}_{g\in\mathcal{G}},\; \sigma_\omega,\; \mu_{\mathcal{G}}\,),$$

whose components are the following.

- $\{\mathcal{P}_g\}$ is a task-indexed family of POMDPs of Definition 2 that share $\mathcal{S}$, $\mathcal{O}$, $\mathcal{U}$, $P$, $Z$, and $R$, in which the task argument $g$ fixes the goal and may fix the initial-state distribution or a generated scene. Environment synthesis therefore acts through $g$, whereas drift of $P$ acts on the loop from outside.
- $\sigma_\omega : \mathbb{N} \times \mathbb{H} \to \Delta(\mathcal{G})$ is the **goal sampler** with configuration $\omega$, which draws the goals of cycle $k$ from $\sigma_\omega(k, \mathcal{H}_k)$. It is *stationary* when it ignores $(k, \mathcal{H}_k)$, *scheduled* when it depends on $k$ alone, as a difficulty order fixed before the loop does, and *adaptive* when it reads $\mathcal{H}_k$, as a success-conditioned curriculum does. Placing $\sigma_\omega$ in the macro-environment locates where goals enter the loop, whereas the provenance matrix of Sec. 2.3 records who realizes it.
- $\mu_{\mathcal{G}} \in \Delta(\mathcal{G})$ is a fixed **evaluation distribution** outside the control of $\sigma_\omega$, which measures performance through $J(\mathcal{A}; \mu_{\mathcal{G}}) := \mathbb{E}_{g\sim\mu_{\mathcal{G}}} J(\mathcal{A}; g)$.

cycle $k \geq 0$, the goals $\boldsymbol{g}_k$, the signal $V_\nu(\mathcal{D}_k)$, and the update of $f_\zeta$ consume no annotation, verdict, or approval added by a human while the loop runs, and no human-issued action, such as a teleoperated correction, enters $\mathcal{D}_k$. Artifacts fixed prior to cycle 0 (task sets, verification checkers, answer keys, external evaluators and judges, or optimizers) constitute the bootstrap. Human resets modifying $\mathcal{P}_g$ and not the learning loop itself, autonomy does not imply reset-free execution [71], and reset overhead is documented separately.

$\mathcal{C}_2$ **Freshness.** Freshness requires per-cycle experience to be drawn from the agent's current policy, not from a fixed dataset or an external stream. Formally, the cycle-$k$ distribution depends on $\mathcal{A}_k$ through $\pi_{\mathcal{A}_k}$, admitting on-policy sampling $\mathcal{D}_k \sim p_{\boldsymbol{g}_k}^{\pi_{\mathcal{A}_k}}$ and asynchronous variants whose behavior policy lags $\mathcal{A}_k$ by a bounded number of updates, but excluding any agent-independent $p_{\mathrm{static}}$ [31]. Experience collected once by $\mathcal{A}_0$ and reused in later cycles fails the criterion. Generating on-policy rollouts, however, satisfies freshness without constituting self-reference.

$\mathcal{C}_3$ **Durability.** Durability requires the update to alter the agent beyond the episode boundary. Formally, $\mathcal{A}_{k+1} = f_\zeta(\mathcal{A}_k, \mathcal{H}_k, \mathcal{D}_k, V_\nu(\mathcal{D}_k))$ must modify persistent components ($\Phi_{\theta_k}$, long-term memory $\mathcal{M}_{\mathrm{long}} \subseteq \mathcal{M}_k$, toolset $\mathcal{T}_k$, or interface $\mathcal{I}_k$), thereby altering induced policy $\pi_{\mathcal{A}_{k+1}}$ on at least one goal. This excludes adaptation confined to the within-episode buffer $\mathcal{M}_{\mathrm{short}}$ and updates restricted to auxiliary proposers or evaluators, which belong to $\Phi_\theta$ yet occupy parameters that $\pi_{\mathcal{A}}$ does not read, while admitting memory writes, tool additions, and prompt mutations even without any gradient descent [31, 34].

$\mathcal{C}_4$ **Recurrence.** Recurrence requires the loop to close across cycles instead of terminating after a single update. Formally, the map $\mathcal{A}_k \mapsto \mathcal{A}_{k+1}$ of $\mathcal{C}_3$ is iterated over more than one cycle, whether rounds or degenerate cycles, and each output becomes the next cycle's agent, not a branch discarded after evaluation. This excludes single-pass optimization pipelines ($\mathcal{A}_0 \mapsto \mathcal{A}_1$), even if compliant with $\mathcal{C}_1$ through $\mathcal{C}_3$. The criterion presupposes $\mathcal{C}_3$ and, being satisfied by every multi-step online method, discriminates loops only in round-based regimes.

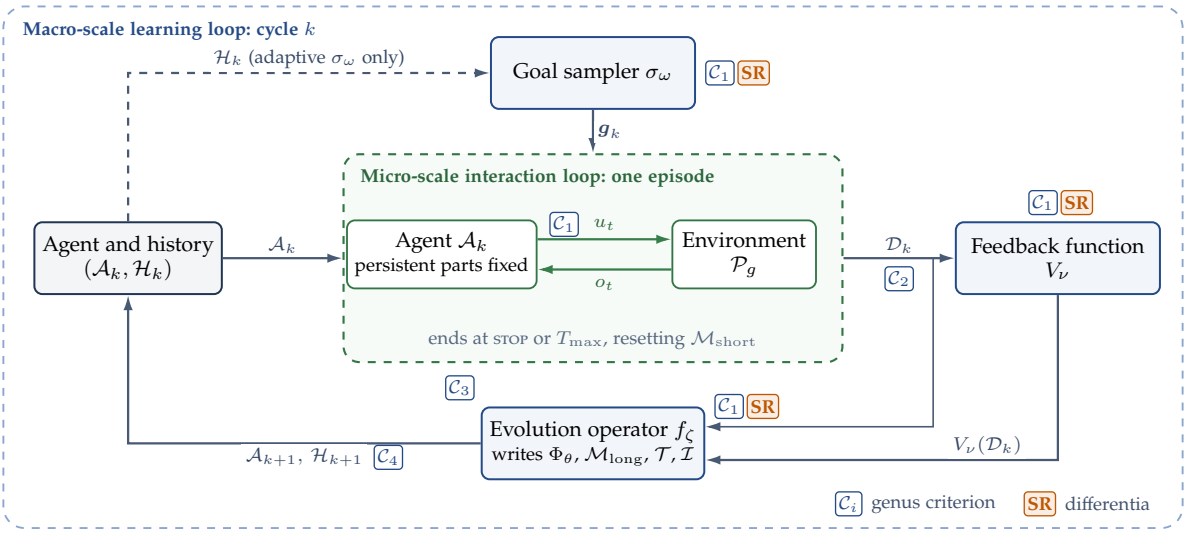



Fig. 1: **Dual-scale learning loop and its defining conditions.** Within an episode (green), $\mathcal{A}_k$ acts in $\mathcal{P}_g$ with its persistent components fixed. Across episodes (blue), each cycle samples goals with $\sigma_\omega$, scores the resulting batch $\mathcal{D}_k$ with $V_\nu$, and updates the agent with $f_\zeta$ (Eq. (5)). Blue tags mark where each genus criterion applies, and orange SR tags mark the meta-operators, at least one of which a self-evolving agent must realize.

**The Differentia: Self-Reference.** The four criteria constrain how the loop runs, not who realizes it, a loop being able to satisfy $\mathcal{C}_1$–$\mathcal{C}_4$ while every meta-operator is realized outside the agent. The differentia, *self-reference* (SR), separates self-evolving loops by requiring the agent to internally realize at least one evolutionary meta-operator, marked by SR tags in Figure 1.[1] Self-reference and the meta-operator clause of autonomy both ask who realizes each meta-operator, and we therefore record their evaluation for a method $m$ in the *provenance matrix* $\mathbf{\Gamma}(m) \in \{0,1\}^{3\times 4}$. Its rows index $(\sigma_\omega, V_\nu, f_\zeta)$ and columns span four provenance classes: runtime human input (H), bootstrap artifacts fixed prior to initialization (B), environment/task generation (E), and the agent (A), with $\Gamma_{ij}(m) = 1$ when class $j$ participates in realizing meta-operator $i$ (rows admit multiple ones). Both conditions evaluate over single columns:

$$\mathcal{C}_1 : \ \mathbf{\Gamma}(m)\,\mathbf{e}_{\text{H}} = \mathbf{0}, \qquad \text{SR} : \ \mathbf{\Gamma}(m)\,\mathbf{e}_{\text{A}} \neq \mathbf{0}, \tag{7}$$

where $\mathbf{e}_{\text{H}}$ and $\mathbf{e}_{\text{A}}$ select the H and A columns, rendering the split between B and E descriptive. For example, SimpleVLA-RL [62] realizes all three meta-operators externally, whereas VisPlay [5] samples goals via an internal Questioner within $\Phi_\theta$ and derives feedback from votes over its own answers, the two matrices differing in the $\sigma_\omega$ and $V_\nu$ rows:

$$\mathbf{\Gamma}_{\text{SimpleVLA-RL}} = \begin{array}{c} \begin{matrix} \text{H} & \text{B} & \text{E} & \text{A} \end{matrix} \\ \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{array} \quad \mathbf{\Gamma}_{\text{VisPlay}} = \begin{array}{c} \begin{matrix} \text{H} & \text{B} & \text{E} & \text{A} \end{matrix} \\ \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix} \end{array}. \tag{8}$$

1. Our criteria adapt those of [28, 31, 34] and treat updated components as a descriptive substrate, but enforce a narrower scope. While prior work categorizes learning from environmental or rule-based rewards as reward-based self-evolution [31] or subsumes reinforcement learning with verifiable rewards on fixed datasets [28], self-reference excludes these regimes whenever the agent realizes zero meta-operators. It does not require co-evolving environment dynamics [34], evolving inter-agent topologies [28], or lifelong operation ($k \to \infty$).

The A entry is tested under row-specific criteria. The goal-sampler row is marked when $\sigma_\omega(k, \mathcal{H}_k)$ conditions on the agent or its history, as with learned proposers, success-dependent curriculum rules, or self-play opponents drawn via the task argument (Definition 4); post-hoc filtering of tied goals leaves subsequent draws unconditioned and is excluded. The feedback row is marked when $V_\nu$ depends on the agent, through the active policy (self-judges, consensus voting over rollouts, or reward models refitted on trajectories) or frozen snapshots (including the initial agent $\mathcal{A}_0$); group-relative normalization and per-goal monotone transforms add no dependence. Snapshots count because shared model lineage preserves priors, and model-based evaluators reportedly favor their own generations [112], a bias plausibly transmitted to descendant policies. The evolution-operator row is marked when the agent model, its replica, or an internally trained component writes update content beyond raw rollouts, such as a lesson, skill, memory entry, or relabeled goal. This row tests writer provenance, not functional dependence, every $f_\zeta$ depending trivially on the updated $\mathcal{A}_k$. Promptbreeder [109], which rewrites its own improvement mechanism, is a special case of an A entry in this row. Beyond membership, an A entry tracking the current agent couples successive cycles through shared model parameters or history-conditioned goal rules, whereas a frozen snapshot couples none.

**Fixed Loops and Scope.** An autonomous learning loop with self-reference is *self-evolving*, whereas one lacking it is *fixed-loop*, completing the membership test in Definition 3. Because the subsequent three-stage decomposition, its six factors, the two ceilings, and cross-cycle coupling are defined over autonomous learning loops, fixed-loop methods serve there as evidence characterizing a scenario's operating point, whereas scenario chapters review self-evolving loops, reporting their provenance rows and design variables. A method can have A entries in several rows (*e.g.*, proposer-solver methods realize both the goal sampler and the feedback function [5, 22]).

TABLE 2: **Demarcation of self-evolution from established paradigms.** C1 to C4 test autonomy, freshness, durability, and recurrence, with parentheses locating the runtime human input under C1. SR marks the meta-operators the agent realizes, *i.e.*, the goal sampler ($\sigma$), the feedback function ($V$), or the evolution operator ($f$). In the instance-dependent group, $^\circ$ and $^\bullet$ mark fixed-loop and self-evolving instances, respectively. $^\dagger$ Acts on classifiers, not on the foundation-model agents that the genus presupposes, and its foundation-model counterpart, label-free reinforcement learning on a fixed question pool, falls under Self-Rewarding and Label-Free RL. (✓ satisfied, ✗ unsatisfied, ◐ satisfied by some variants, – inapplicable.)

| Paradigm | Genus criteria | | | | Differentia | Defining Mechanism | Representative Works |
|---|---|---|---|---|---|---|---|
| | C1 | C2 | C3 | C4 | SR | | |
| ✣ *Outside the genus* | | | | | | | |
| Self-Training$^\dagger$ | ✓ | ✓ | ✓ | ✓ | $V$ | Iterative training of a classifier on its own pseudo-labels over a static unlabeled pool. | Noisy Student [72]; FixMatch [73]; Meta Pseudo Labels [74] |
| Test-Time Adaptation$^\dagger$ | ✓ | ✓ | ◐ | ◐ | ◐($V$) | Inference-time adaptation of a classifier to an external test stream under a self-supervised or entropy objective. | TTT [75]; TENT [76]; MEMO [77]; CoTTA [78]; EATA [79] |
| Test-Time Scaling | ✓ | ✓ | ✗ | – | ◐($V$) | Additional inference compute per query by repeated sampling, voting, or search, with nothing persisting past the query. | Self-Consistency [80]; Snell *et al.* [81]; Corvid [82] |
| Self-Correction | ✓ | ✓ | ✗ | – | ◐($V$) | Refinement of an output through critique within one episode. | Self-Refine [83]; CRITIC [84]; Self-Debug [85]; CoVe [86] |
| Interactive Imitation Learning | ✗(act) | ✓ | ✓ | ✓ | ✗ | Training on the learner's own state distribution with expert actions or interventions supplied while learning runs. | DAgger [87]; $\pi^*_{0.6}$ [88] |
| ✣ *Instance-dependent, divided by a genus criterion or the differentia* | | | | | | | |
| Self-Improvement | ✓ | ✓ | ✓ | ◐ | ◐($\sigma, V, f$) | Training on self-generated outputs retained by a filter, once or over rounds. | Self-Instruct [89]; ReST$^\circ$ [90]; ReST$^{EM\circ}$ [91]; STaR$^\bullet$ [92] |
| Continual Learning | ✓ | ◐ | ✓ | ✓ | ✗ | Sequential learning over a task or data stream fixed in advance, with mechanisms against catastrophic forgetting. | EWC [93]; GEM [94]; CRL-VLA$^\circ$ [95] |
| RLHF | ◐($V$) | ✓ | ✓ | ✓ | ✗ | Policy optimization against a reward model fit to human preferences. | InstructGPT [96]; Bai *et al.* [97]; Llama 2 [98] |
| RLVR | ✓ | ✓ | ✓ | ✓ | ◐($\sigma$) | Policy optimization against rewards computed by a programmatic check of the output. | Tülu 3$^\circ$ [99]; Kimi k1.5$^\bullet$ [100] |
| Agentic RL | ✓ | ✓ | ✓ | ✓ | ◐($\sigma, V$) | Policy optimization of a multi-turn agent from rewards earned in an interactive environment. | SimpleVLA-RL$^\circ$ [62]; ZeroGUI$^\circ$ [101]; DigiRL$^\bullet$ [14] |
| Prompt and Workflow Optimization | ✓ | ✓ | ✓ | ✓ | ◐($f$) | Iterative rewriting of an agent's instructions or workflow from the scores of its executions. | TextGrad$^\circ$ [102]; SymbOmni$^\bullet$ [103] |
| Experiential Learning | ✓ | ✓ | ✓ | ✓ | ◐($f$) | Distillation of an agent's trajectories into lessons, memory entries, or skills that later tasks retrieve. | UI-Mem$^\circ$ [104]; ExpeL$^\bullet$ [66]; Voyager$^\bullet$ [67] |
| ✣ *Self-evolving by definition* | | | | | | | |
| Self-Play | ✓ | ✓ | ✓ | ✓ | $\sigma$ | Training against tasks or opponents drawn from the agent itself or its history. | Absolute Zero [105]; Vision-Zero [7] |
| Automatic Curriculum Learning | ✓ | ✓ | ✓ | ✓ | $\sigma$ | Adaptation of the goal distribution to the agent's measured competence. | Portelas *et al.* [106]; SEAgent [16] |
| Self-Rewarding and Label-Free RL | ✓ | ✓ | ✓ | ✓ | $V$ | Training on rewards or preferences that the trained model produces, by judgment or by consensus over its samples. | Self-Rewarding [107]; TTRL [108]; EvoLMM [22] |
| Recursive Self-Improvement | ✓ | ✓ | ✓ | ✓ | $f$ | Rewriting of the system's own improvement procedure by the system. | Promptbreeder [109]; STOP [110]; Gödel Agent [111] |

### 2.4 Demarcation from Established Paradigms

The term *self-evolving* shares vocabulary with several established paradigms, and recent surveys draw its boundary at different points [28, 31, 34, 113, 114]. Table 2 compares adjacent paradigms; contested boundaries hinge on the A entry. In the goal-sampler row, discarding tied groups within a batch adds no A entry [62], whereas allocating subsequent rollouts by task success qualifies [115]. In the evolution-operator row, the test is who writes the update: a separate fixed model (*e.g.*, GPT-4o guiding GPT-3.5-Turbo in TextGrad) keeps the loop fixed [102, 104], whereas the acting model itself, as with ExpeL's insights [66] or STaR's rationales for failed instances [92], establishes self-evolution. Reflexion [116], where the acting model preserves self-reflections across episodic trials, qualifies as self-evolving, although its reflections, written for one task, improve no other task. Anti-forgetting mechanisms of continual learning map onto the evolution operator $f_\zeta$, whereas classical curriculum samplers such as POET [117], PLR and other regret-based environment designers [118, 119, 120], goal-proposing teachers [121, 122], and expert-iteration and AlphaZero self-play [123, 124] remain outside our foundation-model scope.

## 3 The Three-Stage Self-Evolving Loop

We adopt *the three-stage self-evolving loop* as our analytical framework, decomposing each cycle of an autonomous learning loop into experience generation, feedback acquisition, and update consolidation. Fixed and self-evolving

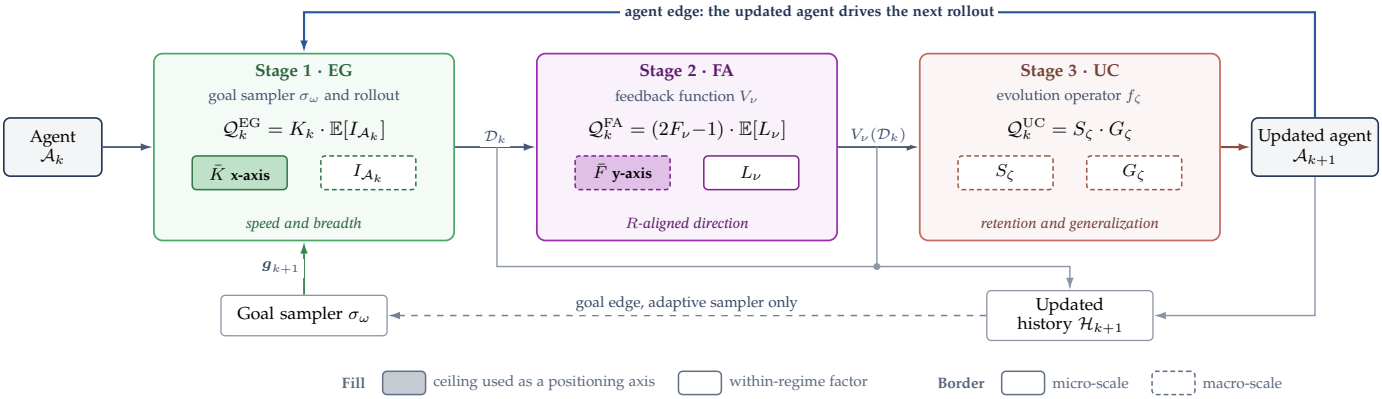



Fig. 2: **The three-stage self-evolving loop and its six factors.** Each stage centers on one meta-operator, and its effectiveness $\mathcal{Q}_k$ is the product of two factors. The filled ceilings $\bar{K}$ and $\bar{F}$ serve as the axes of the positioning matrix (Figure 3). The goal edge, fed by the trajectories and scores of cycle $k$, reaches $\sigma_\omega$ only when it is adaptive, and extensions of the agent edge into $\sigma_\omega$, FA, or UC are not drawn.

loops across all four scenarios sharing this structure, we differentiate them by where the bottleneck binds instead of by topology, treating rollout cost and feedback fidelity as bounds on viable mechanisms. We first formalize the three stages and their governing factors, then position the four scenarios within a positioning matrix spanned by these bounds, and trace how the stages couple across cycles.

### 3.1 Loop Decomposition

As shown in Figure 2, we factor each cycle into three scenario-invariant stages, each organized around one meta-operator ($\sigma_\omega$, $V_\nu$, $f_\zeta$). This decomposition applies across the entire genus of autonomous learning loops and treats auxiliary operations, such as credit assignment or data filtering, as refinements within a stage, not as additional stages.

**Stage 1** **Experience Generation (EG):** the goal sampler $\sigma_\omega$ draws the goal batch $\boldsymbol{g}_k$, and the current agent $\mathcal{A}_k$ rolls out in $\mathcal{P}_g$ for each goal $g \in \boldsymbol{g}_k$, producing the trajectory batch $\mathcal{D}_k$. Through the quantity and informativeness of $\mathcal{D}_k$, the only trajectories FA and UC act on, EG bounds how fast and how broadly the loop can evolve (*speed and breadth*).

**Stage 2** **Feedback Acquisition (FA):** the feedback function $V_\nu$ maps $\mathcal{D}_k$, together with the agent history $\mathcal{A}_{0:k}$ when $V_\nu$ reads the agent, to scores, preferences, or critiques sourced from the environment, the bootstrap, or the agent itself. FA is the loop's only source of information about the idealized task reward $R$ (*the $R$-aligned direction*).

**Stage 3** **Update Consolidation (UC):** the evolution operator $f_\zeta$ computes an update from $(\mathcal{A}_k, \mathcal{H}_k, \mathcal{D}_k, V_\nu(\mathcal{D}_k))$ and writes it into at least one persistent component of $\mathcal{A}_k$, producing the next-cycle agent $\mathcal{A}_{k+1}$. UC determines whether an update persists across later cycles and generalizes beyond the goals that produced it (*retention and generalization*).

The six factors originate at the two scales of Figure 1. The per-episode cost $c(\tau)$ governing $\bar{K}$ and action localization $L_\nu$ arise within the micro-scale interaction loop, whereas information gain, evaluator fidelity $F_\nu$, and consolidation factors ($S_\zeta$, $G_\zeta$) emerge from the macro-scale learning loop, the positioning matrix thereby pairing a micro-scale axis with a macro-scale one.

### 3.2 Stage 1: Experience Generation (EG)

EG produces the trajectory batch $\mathcal{D}_k$ of each cycle by rolling out the current agent $\mathcal{A}_k$ in the POMDPs $\mathcal{P}_g$ of Definition 4. For each goal $g$ in the batch $\boldsymbol{g}_k$ drawn from $\sigma_\omega(k, \mathcal{H}_k)$, $\mathcal{A}_k$ draws $K_k$ rollouts from $p_g^{\pi_{\mathcal{A}_k}}$ and pools them into $\mathcal{D}_k$. EG must supply experience of sufficient *scale* and *informativeness* under finite compute, wall-clock, and safety budgets, a constraint on quantity and one on quality. We summarize this bottleneck through an effective-experience decomposition,

$$\mathcal{Q}_k^{\mathrm{EG}} \;:=\; K_k \cdot \mathbb{E}_{g \sim \sigma_\omega(k, \mathcal{H}_k)}[I_{\mathcal{A}_k}(g)]\,, \tag{9}$$

where $K_k$ is the number of per-goal rollouts in cycle $k$, and $I_{\mathcal{A}_k}(g) := \mathrm{Var}_{\tau \sim p_g^\pi}[\mathrm{ret}_g(\tau)]$ is the outcome variance of $g$. Here, $\mathrm{ret}_g(\tau) = \sum_t \gamma^t R(s_t, u_t, g)$ denotes the return of $\tau$, and $p_g^\pi$ abbreviates $p_g^{\pi_{\mathcal{A}_k}}$. This variance, a goal-level proxy for the learnable information gain [131], vanishes on goals that $\mathcal{A}_k$ has mastered or never solves, where GRPO-style group-relative updates receive zero advantage under an $R$-aligned evaluator. Under a binary terminal reward with $\gamma = 1$, it reduces to $p_g(1 - p_g)$, which difficulty-targeting rewards and filters track via an estimated success probability $p_g$. Defined on the idealized reward $R$ of Eq. (4), the proxy renders $\mathcal{Q}_k^{\mathrm{EG}}$ diagnostic, and it exposes two failure modes. As $K_k$ shrinks, too few per-goal rollouts remain to contrast success with failure. As $\mathbb{E}[I_{\mathcal{A}_k}]$ approaches zero, the sampled goals are mastered or unreachable. Concretely, $K_k$ is bounded by per-rollout *interaction cost* and *action recoverability*, while $\mathbb{E}[I_{\mathcal{A}_k}]$ is governed by the *diversity and information gain* targeted by $\sigma_\omega$ and by the *interaction mode* realizing $p_g^{\pi_{\mathcal{A}_k}}$ (Table 3).

**Quantity Constraint.** Interaction cost and action recoverability govern $K_k$ via a single per-rollout cost $c(\tau)$, eliminating the need for a separate term for actions that resist reverting, resetting, or safe repetition. Specifically, $c(\tau)$ absorbs the time, compute, synchronization, and reset cost of one rollout, with the reset term penalizing both environment restoration and expected damage from failed trajectories. Under a fixed per-goal budget $B$, the rollout count satisfies the affordability

TABLE 3: **Asymmetric substitutability of $\mathcal{Q}_k^{\mathrm{EG}}$ across four scenarios.** Profiling experience generation across the quantity and quality factors from Eq. (9). The bottom row highlights the binding bottleneck for each scenario, showing where cheap rollouts saturate and where costly environments constrain scale.

| | MR | MG | GW | EM |
|---|---|---|---|---|
| ① **Quantity factor $K_k$:** *how many rollouts can a cycle afford?* | | | | |
| ✣ Cost of one rollout $c(\tau)$ | one decoding pass | one sampling pass or tool chain | per-step environment latency | per-step latency plus episode reset |
| ✣ Action recoverability | none needed (no side effects) | none needed (no side effects) | partial (DOM and account-level writes) | largely irreversible (contact, wear) |
| ✣ Reported wall-clock cost | not reported per rollout (one pass) | not reported per rollout (one pass) | 2.5–4.0 s per step [125], episodes of tens of minutes [115], 5–10 min environment re-initialization per training step [15] | about 20 h per real-world experiment with human resets [19], 2.6–16.9 s per judge query [21] |
| ✣ Reported group size $K_k$ | 4−10 [7, 8, 22, 126, 127] | 3−16 [9, 128, 129] | 8−64 [17, 101, 115, 130] | 8 [62] |
| ② **Quality factor $\mathbb{E}[I_{\mathcal{A}_k}]$:** *how informative is each rollout?* | | | | |
| ✣ Goal-sampling strategy | task synthesis by a co-trained proposer (adaptive) | random sampling from a fixed prompt pool (stationary) | fixed task sets reweighted by success records (adaptive) | fixed task suites, success-conditioned or self-proposed practice (adaptive) |
| ✣ Interaction mode | single action, sometimes tool-integrated | single output or a short tool chain | multi-step screen states, partially observable | multi-step physical states, partially observable |
| **Primary bottleneck of $\mathcal{Q}_k^{\mathrm{EG}}$** | **Quality, at saturation** | **Quality** | **Quantity** | **Both, on hardware** |

ceiling $K_k \le \bar{K} := B/\mathbb{E}_\tau[c(\tau)]$, where the expectation runs over episodes of the target task family at the horizon cap $T_{\max}$, not over the rollouts of a particular policy, which lets $\bar{K}$ reflect task structure and not policy choice. When $c(\tau)$ is small, group-sampling operators such as self-consistency [80, 132] or GRPO [65, 133, 134] sample a group of rollouts per goal at negligible marginal cost. As $c(\tau)$ grows, the bottleneck shifts from theoretical execution feasibility to the budget of exploratory failures the system can absorb.

**Quality Constraint.** Even with a large $K_k$, a trajectory set is unproductive if its goals are already mastered or still unreachable for $\mathcal{A}_k$, leaving EG viable only while $\mathbb{E}[I_{\mathcal{A}_k}] > 0$. For instance, self-play [131] stops improving once its synthetic pipeline no longer supplies learnable information gain. *Diversity and information gain* determines whether $\sigma_\omega$ draws goals with nonzero $I_{\mathcal{A}_k}$. Sampling schemes escalate in intervention from random sampling [65] through curriculum mining over past failures [17, 135] and active task selection [136, 137] to environment synthesis [63, 138, 139, 140, 141]. Random sampling keeps $\sigma_\omega$ stationary, whereas curriculum mining and active selection make it adaptive. Environment synthesis acts through the task argument $g$ of Definition 4 and is adaptive only when the generator reads the agent's record. Only an adaptive sampler conditions the next goal draw on the agent's record and places an A entry in the goal-sampler row. *Interaction mode* fixes what evidence each trajectory can expose. Single-pass generation and tool-augmented reasoning yield single-action episodes, whereas digital and embodied environments expose multi-step state transitions whose informativeness depends on platform dynamics, reset conditions, and observability.

**Asymmetric Substitutability.** The product in Eq. (9) enables mutual compensation between its two factors, locating the cross-scenario asymmetry in the cost of raising each factor. Raising $K_k$ offsets low information gain only when rollouts are cheap, parallelizable, and recoverable and feedback retains contrast as group size grows. Consensus rewards losing contrast beyond small groups [22], the loop elsewhere must instead raise $\mathbb{E}[I_{\mathcal{A}_k}]$ via curriculum mining [17, 115], active exploration [5, 18], demonstrations at initialization [19], or synthetic environments [6, 63, 138]. Equal values of $K_k$ therefore do not imply identical bottlenecks. Multimodal generation and GUI/Web navigation report overlapping group sizes (Table 3), yet generation samples cheap, reversible passes below $\bar{K}$ and is bounded by $\mathbb{E}[I_{\mathcal{A}_k}]$ over a fixed prompt pool, whereas navigation incurs latency-bound episodes and is bounded by $\bar{K}$ itself. Cheap but uninformative rollouts saturate the loop, whereas informative but costly rollouts throttle its scaling.

### 3.3 Stage 2: Feedback Acquisition (FA)

FA maps the trajectory batch $\mathcal{D}_k$ of each cycle to the feedback $V_\nu(\mathcal{D}_k) \in (\mathbb{Y}^{\mathrm{mm}})^*$, the scores, preferences, critiques, or structured judgments that direct $f_\zeta$. Any scoring, ranking, filtering, or best-of-$N$ selection by $V_\nu$ belongs to FA even when interleaved with generation, whereas $V_\nu$-independent filtering such as deduplication remains in EG. FA must supply feedback that is *aligned* with the intended objective $R$ and *localized* at an actionable grain without recurring human supervision ($\mathcal{C}_1$), a constraint on fidelity and one on localization. We summarize this bottleneck through an effective-feedback decomposition,

$$\mathcal{Q}_k^{\mathrm{FA}} := \left(2F_\nu(\mathcal{D}_k; R, \boldsymbol{g}_k) - 1\right) \cdot \mathbb{E}_{\tau\sim\widehat{p}_{\mathcal{D}_k}}[L_\nu(\tau)]\,, \qquad (10)$$

where $\widehat{p}_{\mathcal{D}_k}$ is the empirical measure on $\mathcal{D}_k$, $F_\nu \in [0,1]$ is the rate at which the ordering $V_\nu(\mathcal{D}_k)$ induces on same-goal trajectory pairs agrees with their ordering by $\mathrm{ret}_g$, and $L_\nu(\tau) \in (0,1]$ is the fraction of environment actions along $\tau$ to which $V_\nu$ can assign credit. Here, pairs tied under either ordering are excluded, and $F_\nu := 1/2$ if none remains. This pairwise form covers scalar, pairwise, and ranked feedback alike, and $2F_\nu - 1$ is the Goodman-Kruskal rank correlation of the two orderings, zero for a random evaluator and negative for a reversing one. Like $I_{\mathcal{A}_k}$, $F_\nu$ is defined on $R$, which

TABLE 4: **Asymmetric vulnerability of $\mathcal{Q}_k^{\mathrm{FA}}$ across four scenarios.** Profiling feedback acquisition across the fidelity and localization factors from Eq. (10), with signal sources tagged by provenance. The bottom row identifies the binding bottleneck for each scenario, showing where gameable signals misdirect the loop and where coarse signals under-constrain it.

| | MR | MG | GW | EM |
|---|---|---|---|---|
| ① **Fidelity factor** $F_\nu$: *how faithfully does* $V_\nu(\mathcal{D}_k)$ *order trajectories by* $\mathrm{ret}_g$*?* | | | | |
| ✣ Signal source (provenance) | majority vote A, self-judgment A, computed answers E | self-rewards A, same-model judges A, external judges B | task checkers B, VLM judges B, co-evolving RMs A, same-model judges A | episode outcomes E, progress heads and same-model judges A, VLM judges B |
| ✣ Channel coverage | full (discrete answer) | often perceptual only | full on screen, partial for backend state | full in simulation, visual only on hardware |
| ✣ Gaming resistance under $f_\zeta$ | high for computed answers, untested for votes | low [9, 129] | untested beyond downstream success [142] | low without judge oversight [143] |
| ② **Localization factor** $L_\nu$: *how finely can credit be apportioned along* $\tau$*?* | | | | |
| ✣ Temporal granularity | episode = step ($T=1$ without tool calls) | episode ($T=1$ if single-pass) | episode (judges), step (co-evolving RMs) | episode (outcomes), step (progress heads) |
| ✣ Credit-assignment cost | none ($T=1$) | none if single-pass, low otherwise | co-trained step RMs [135, 144] | demonstration-trained progress heads [19, 145] |
| **Primary bottleneck of** $\mathcal{Q}_k^{\mathrm{FA}}$ | **Realized** $F_\nu^{(k)}$ | **Fidelity ceiling** $\bar{F}$ | **Localization** | **Both, on hardware** |

makes $\mathcal{Q}_k^{\mathrm{FA}}$ diagnostic, and it exposes two failure modes. As $F_\nu$ falls toward $1/2$, $f_\zeta$ follows an uninformative proxy, and below $1/2$ it optimizes against $R$. As $L_\nu$ shrinks, a correct signal becomes too coarse to guide improvement. Concretely, $F_\nu$ is bounded by the *signal source* and *gaming resistance* of $V_\nu$, while $L_\nu$ is governed by the *temporal granularity* and *credit-assignment cost* of its scores (Table 4).

**Fidelity Constraint.** Signal source and gaming resistance act on $F_\nu$ at different levels: the signal source fixes the evaluator configurations $\Xi_V$ a scenario admits, hence the attainable-fidelity ceiling $\bar{F} := \sup_{\nu\in\Xi_V} F_\nu$, whereas gaming resistance governs how much of it survives repeated $f_\zeta$ updates. Where a closed-form success condition or a discrete answer space exists, deterministic instruments such as answers computed by task generation [64, 146], unit tests [147, 148], and physical-constraint rewards [18, 145, 149] sustain high $F_\nu$ at modest cost. Lacking both, evaluators rely on self-rewards [9, 10, 150, 151], reward models [135, 144, 152, 153, 154, 155, 156], or VLM judges [14, 18, 157, 158], and gaming resistance binds [9, 159]. *Channel coverage* further caps $F_\nu$, an instrument leaving unobserved channels of $\mathcal{U}$ unconstrained [9, 18, 145]. Self-rewards and co-trained reward models read $\mathcal{A}_k$ and place an A entry in the feedback row, and their realized fidelity $F_\nu^{(k)}$ therefore drifts with the policy they score; all others enter as B or E. Reported audits measure label or verdict accuracy $a$, not the pairwise agreement that defines $F_\nu$. Under a binary terminal reward with verdict errors independent across trajectories and symmetric across outcomes, a success-failure pair is ordered correctly with probability $a^2$ and reversed with probability $(1-a)^2$, giving $F_\nu = a^2/(a^2+(1-a)^2)$, which exceeds $a$ whenever $a > 1/2$. A reported accuracy thus bounds $F_\nu$ from below only under these assumptions, and errors correlated within a goal void the bound. In multimodal reasoning, program-computed answers stay correct on 94.4% of retained questions [64], yet self-generated pseudo-labels degrade across rounds [5, 160], consistent with self-referential evaluators drifting below a high $\bar{F}$, although rising question difficulty confounds these declines.

**Localization Constraint.** Even with high $F_\nu$, a trajectory set is directionally unproductive if $L_\nu$ covers a vanishing fraction of environment actions, and FA accordingly remains viable only while $\mathbb{E}_\tau[L_\nu(\tau)]$ suffices for the grain $f_\zeta$ requires. *Temporal granularity* fixes the scoring unit along $\tau$: the terminal outcome for episode-level scores, the action $u_t$ for step-level scores, and the output $y_t$ decoded into $u_t$ for token-level scores. Because $L_\nu$ counts environment actions, a single-action episode has $L_\nu = 1$, and credit within a long reasoning chain $y_t$ lies outside $L_\nu$ even where process reward models assign it [153]. Episode-level feedback suffices when the task reduces to one discrete outcome, step-level process supervision becomes necessary as horizons lengthen and terminal success grows too sparse [18, 153, 167, 168, 169], and token-level credit remains rare outside independently checkable reasoning chains [170]. *Credit-assignment cost* determines how much of that grain a cycle can afford, step-level scores requiring a process model or per-step estimates whose cost grows with horizon length [153]. Granularity is thus an FA choice that reaches $f_\zeta$ only through its signal.

**Asymmetric Vulnerability.** The product in Eq. (10) admits only conditional, one-directional compensation between its two factors. Higher $F_\nu$ from cross-model verification or understanding-generation duality offsets moderate localization loss only when an unbiased anchor exists [10, 128], whereas finer $L_\nu$ yields no directional gain once $F_\nu$ is gameable [9, 159]. Equal ceilings $\bar{F}$ therefore do not imply identical bottlenecks. Multimodal reasoning and GUI/Web navigation both admit a high $\bar{F}$ (Table 4), through program-computed answers [64] and per-task state scripts in sandboxed environments [17, 115], yet reasoning fixes $L_\nu = 1$ and is bounded by the realized $F_\nu^{(k)}$ of its consensus and self-judged signals [5, 160, 171], while navigation scores whole long-horizon episodes and is bounded by $L_\nu$ [16, 135, 144]. Multimodal generation instead faces a low $\bar{F}$ on open-ended prompts, where self-rewards and perception-only instruments are readily gamed [9, 10], and embodied manipulation loses both factors on hardware [19, 145, 172, 173]. Fidelity-bounded signals thus misdirect the loop and localization-bounded ones under-constrain it, making $\bar{F}$ a positioning axis and $L_\nu$ a within-regime variable.

TABLE 5: **Within-regime specialization of $\mathcal{Q}_k^{\mathrm{UC}}$ across four scenarios.** Profiling update consolidation across the retention and generalization factors from Eq. (11), both lacking a scenario-fixed ceiling. The bottom row identifies the binding bottleneck for each scenario, showing where later cycles erode gains and where updates stay within sampled goals.

| | MR | MG | GW | EM |
|---|---|---|---|---|
| ① **Retention factor** $S_\zeta$**:** *does the cycle-k gain survive subsequent updates?* | | | | |
| ✣ Dominant substrate | $\Phi_\theta$, except training-free methods | $\Phi_\theta$, or $\mathcal{M}, \mathcal{T}, \mathcal{I}$ if training-free | $\Phi_\theta$, or $\mathcal{M}, \mathcal{T}$ if training-free | $\Phi_\theta$, or $\mathcal{M}, \mathcal{T}, \mathcal{I}$ |
| ✣ Iteration robustness under $f_\zeta$ | untested beyond a few cycles | compounding hacking within single runs [9, 129] | sustained by trajectory replay [14, 17], no forgetting on held-out environments [15] | fixed-loop continual RL only [95, 161] |
| ✣ Dominant failure mode | long-cycle drift | compounding reward hacking | distribution shift | forgetting and distribution shift |
| ② **Generalization factor** $G_\zeta$**:** *does the gain transfer beyond* $\mathcal{G}_k$*?* | | | | |
| ✣ Transfer locus | parameters (any goal) | parameters, or memory and skills (on use) | parameters, or memory and skills (on use) | parameters, or skills and harnesses (on use) |
| ✣ Experience coverage in $\mathcal{D}_k$ | tasks synthesized over unlabeled media | a fixed prompt pool | the evaluation task set or stream [104, 162, 163, 164] | one simulator suite |
| ✣ Achieved generalization scope | cross-task [5, 7, 22] | in-distribution, partly cross-benchmark [129, 165] | in-distribution, one platform | in-distribution, rare sim-to-real [18, 166] |
| **Primary bottleneck of** $\mathcal{Q}_k^{\mathrm{UC}}$ | **Retention** | **Retention** | **Generalization** | **Generalization** |

### 3.4 Stage 3: Update Consolidation (UC)

UC writes the update that $f_\zeta$ computes from $\mathcal{D}_k$ and $V_\nu(\mathcal{D}_k)$ into one or more persistent components of $\mathcal{A}_k = (\Phi_{\theta_k}, \mathcal{M}_k, \mathcal{T}_k, \mathcal{I}_k)$, as $\mathcal{C}_3$ requires. Writes to the episode-scoped buffer $\mathcal{M}_{\mathrm{short}}$ through $\Psi$ fall outside UC, and $V_\nu$-based selection of what to write remains in FA. UC must write updates that *persist* undegraded across subsequent cycles and *generalize* beyond the cycle-$k$ goals, a constraint on retention and one on generalization. We summarize this bottleneck through an effective-consolidation decomposition,

$$\mathcal{Q}_k^{\mathrm{UC}} \;:=\; S_\zeta(\mathcal{A}_{k:k+j+1}) \cdot G_\zeta(\mathcal{A}_{k+1};\, \mu_{\mathcal{G}}), \qquad (11)$$

where $\Delta_k(g) = J(\mathcal{A}_{k+1}; g) - J(\mathcal{A}_k; g)$ is the cycle-$k$ gain on goal $g$, $\mathcal{G}_k$ is the support of $\boldsymbol{g}_k$, and $\mu_{\mathcal{G}}$ is the fixed evaluation distribution of Eq. (6). Here, $S_\zeta \in [0, 1]$ is the fraction of the cycle-$k$ gain on $\mathcal{G}_k$ that survives $j$ further cycles, $\min\{1, \max\{0, \mathbb{E}_{g\sim\mu_{\mathcal{G}}|\mathcal{G}_k}[J(\mathcal{A}_{k+1+j}; g) - J(\mathcal{A}_k; g)] \,/\, \mathbb{E}_{g\sim\mu_{\mathcal{G}}|\mathcal{G}_k}[\Delta_k(g)]\}\}$ for a positive denominator, and $G_\zeta \in [0, 1]$ is the $\mu_{\mathcal{G}}$-measure of goals outside $\mathcal{G}_k$ with $\Delta_k(g) > 0$, renormalized to the complement of $\mathcal{G}_k$. The gain and the two factors correspond to the adaptivity, retention, and generalization goals proposed for self-evolving agents [31], and $S_\zeta$ normalizes by the cycle-$k$ gain the change on earlier goals that backward transfer records as a signed difference [94]. Like $F_\nu$, both factors are defined through $J$ and hence on $R$, leaving $\mathcal{Q}_k^{\mathrm{UC}}$ diagnostic as well, and it exposes two failure modes. As $S_\zeta$ falls toward zero, later cycles erase or distort the cycle-$k$ gain. As $G_\zeta$ falls toward zero, the update stays confined to $\mathcal{G}_k$. Concretely, $S_\zeta$ is bounded by the *substrate* that $f_\zeta$ writes and its *iteration robustness*, while $G_\zeta$ is governed by the *transfer locus* of that substrate and the *experience coverage* of $\mathcal{D}_k$ (Table 5).

**Retention Constraint.** Substrate and iteration robustness act on $S_\zeta$ at different levels: the substrate fixes how far a single write can interfere with earlier gains, whereas iteration robustness governs whether those gains survive repeated $f_\zeta$ applications. The substrate is either $\Phi_\theta$, written by GRPO [65], PPO [174], or DPO [175], or a non-parametric component such as memory [55, 116, 176, 177, 178], tools [67, 179], or prompts [180, 181, 182]. Three documented failure modes threaten iteration robustness: compounding reward hacking through accumulated Stage 2 bias within single runs [9, 129], distribution shift as the rollout law leaves the calibration region of $V_\nu$ [14], and catastrophic forgetting through interference in $\Phi_\theta$ [95, 161]. Hacking and shift enter across the cycle boundary, whereas forgetting arises inside $f_\zeta$ and yields to experience replay, parameter isolation, and regularized updates [95, 161, 183, 184, 185, 186]. Within one task family, the three modes can compound into *long-cycle drift*, a slow multi-source degradation that terminal-only reporting leaves undetected and per-round scores on a fixed benchmark suite reveal only as non-monotone curves.

**Generalization Constraint.** Even with high $S_\zeta$, a persisted update confers no broader benefit if it improves only the goals in $\mathcal{G}_k$, UC thus remaining useful only while $\mathcal{A}_{k+1}$ improves on a non-trivial share of the goals outside $\mathcal{G}_k$. *Transfer locus* fixes where a write takes effect: on any goal that activates the affected weights for parameters, upon retrieval for memory, upon invocation for tools, and within the templating scope for prompts. *Experience coverage* determines the subspace of $\mathcal{G}$ to which the update can generalize, and reported gains stay mostly within the support of $\mathcal{D}_k$, as the fixed loops UI-Voyager [187], UI-Mem [104], and an online skill library [164] train and evaluate on the same task sets. Achieved scope rises from in-distribution transfer to unsampled goals of the same regime, through cross-task transfer to neighboring goal subspaces, to cross-domain transfer to other GUI platforms or real hardware, the last reached by few methods on small trial counts [18, 166]. The substrate thus upper-bounds transfer, and the coverage of $\mathcal{D}_k$ sets how much of it a cycle realizes.

**Within-Regime Specialization.** Neither factor of Eq. (11) has a scenario-fixed ceiling, and UC therefore specializes within each quadrant, not along an axis. $S_\zeta$ binds when FA is gameable or EG is non-recoverable, and $G_\zeta$ binds when $\mathcal{D}_k$ covers too little of $\mathcal{G}$. Shared substrates therefore do not imply identical bottlenecks. Multimodal reasoning and GUI/Web navigation share $\Phi_\theta$ as the dominant substrate

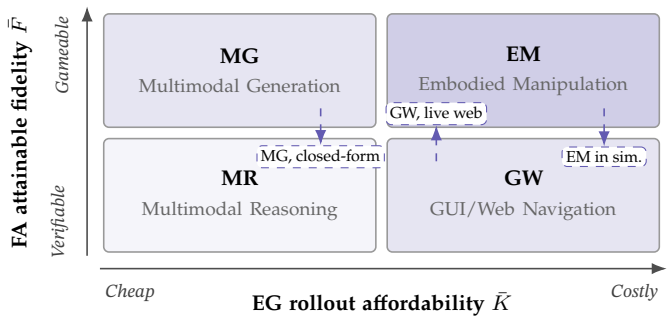



Fig. 3: **Cross-scenario positioning matrix.** The axes are the scenario-fixed ceilings $\bar{K}$ and $\bar{F}$, both decreasing away from the origin. Each scenario sits at the inferred height of its ceilings, with shading deepening from MR (neither binding) to EM (both binding). The MG, GW, and EM cells denote open-ended prompts, sandboxed environments with state scripts, and physical deployment, and dashed arrows show the displacement of closed-form generation sub-families, of navigation on live websites, and of simulated manipulation.

(Table 5), yet reasoning sustains intensive parametric writes under low-cost EG and rule-checkable FA and is bounded by long-cycle drift [5, 22, 64], while navigation stabilizes its parametric writes by trajectory replay [14, 17] and, in both parametric and training-free forms, is bounded by transfer that stays within one platform. Multimodal generation shares the retention bottleneck through compounding hacking of its parametric writes [9, 129], and embodied manipulation shares the generalization bottleneck, writing to $\Phi_\theta$ and to skills, memory, and harnesses [166, 188, 189, 190] within single simulator suites. Few methods measure either factor across cycles, and retention on earlier-round goals is measured in a single self-evolving loop, RSI in UMMs [191], besides fixed-loop continual reinforcement learning [95, 161]. UC thus inherits its bottleneck from the first two stages and refines a method within its quadrant without moving it.

## 3.5 Cross-Scenario Positioning

The stage analyses locate where each scenario's loop is bound, and this subsection condenses them into a positioning matrix that accounts for the cross-scenario asymmetry. To serve as an axis, a stage factor must be *scenario-exogenous*, fixed by task structure instead of algorithmic choice, and *substitution-resistant*, bounded in a way that the paired factor of its stage cannot relax. Realized values, which each method sets through its budget and evaluator, fail the first condition, leaving the ceilings as the only admissible axes.

**Axis Selection.** We select the ceilings $\bar{K}$ and $\bar{F}$ as the two axes, the only stage factors satisfying both conditions. The horizontal axis $\bar{K}$ is fixed by interaction cost and recoverability, and no information-gain intervention lifts it. With $B$ shared as a comparison convention, not a measured quantity, $\bar{K}$ orders scenarios by $\mathbb{E}_\tau[c(\tau)]$. The vertical axis $\bar{F}$ is fixed by verifier availability, and no localization refinement raises it. We assess $\bar{F}$ on the task families that define a scenario's target, such as open-ended prompts in generation and open task sets on hardware in manipulation, excluding sub-families whose closed-form success predicates push the ceiling near one. Its value is the highest fidelity a reported evaluator attains on these families, an estimate that may run high, agreement measured on fixed outputs tending to fall under optimization. The other four factors each fail a condition: the information gain fails scenario-exogeneity, its ceiling $\sup_\omega \mathbb{E}[I_{\mathcal{A}_k}]$ being set by backbone coverage; the localization $L_\nu$ fails substitution-resistance in that a higher $F_\nu$ offsets its deficit when an unbiased anchor exists; and $S_\zeta$ and $G_\zeta$ have no scenario-fixed ceiling.

**Quadrant Assignment.** We place each scenario in a quadrant of Figure 3 by the height of its two ceilings, calling a ceiling binding when it is low. Multimodal reasoning faces neither low ceiling, multimodal generation a low $\bar{F}$ on open-ended prompts, GUI/Web navigation a low $\bar{K}$ under latency-bound episodes in sandboxed environments whose end states per-task scripts inspect, and embodied manipulation both in physical deployment, its target. Three displacements, drawn dashed in the matrix, qualify this placement: (*i*) most embodied loops learn in simulation, where success predicates raise $\bar{F}$ and group sampling eases $\bar{K}$, shifting the operating point toward the navigation cell; (*ii*) generation contains verifiable sub-families such as OCR-scored prompts [195, 196, 197, 198, 199] and program-rendered charts [191], raising $\bar{F}$ toward the reasoning cell; and (*iii*) navigation on live websites or free-form goals lacks state scripts and falls to screenshot judges [101, 200, 201], lowering $\bar{F}$ toward the manipulation cell. The four scenarios thus occupy four quadrants, consistent with the bottom rows of Tables 3 and 4, and the two off-diagonal cells, cheap but gameable generation and costly but verifiable navigation, are combinations that a single difficulty ordering would merge. In the absence of any study that estimates either ceiling under one definition in more than one scenario, the placement remains inferred from reported operating points.

**Backbone Prior as a Confound.** The backbone $\mathcal{A}_0$ confounds this placement by bounding what a loop can reach independently of task structure. In text-only models, self-verification sharpens a model toward outputs it already ranks highly [202], reinforcement learning with verifiable rewards narrows the problems solved at large sampling budgets [203], and gains from spurious rewards vary with the model family [204]. Backbones also differ across scenarios, from vision-language models pretrained on large question-answering corpora in reasoning to action policies with far less motor pretraining in manipulation. Part of the contrast that the matrix attributes to $\bar{K}$ and $\bar{F}$ may thus reflect backbone coverage, a factor that stays entangled with task structure.

**Feasibility Filter.** The matrix supplies the first of two checks on whether a mechanism transfers across scenarios. Scenario-fixed by construction, the two ceilings bound the rollouts and the fidelity open to any learning loop within a quadrant. A transferred mechanism further presupposes resources of its source scenario that neither ceiling records, such as output equivalence or checkable outcomes, and where the target lacks them, the mechanism is conditional on an adaptation or a sub-family that supplies them. The matrix thus bounds within-cycle feasibility without deciding portability, its verdicts remaining untested while each reported comparison stays confined to one scenario.

TABLE 6: **Cross-cycle degradation modes.** Each failure mode is characterized by its mechanism, the stage where it surfaces, the path that carries it, and where it is primarily mitigated, all inferred from within-run, fixed-loop, and out-of-loop evidence.

| Failure mode | Cross-cycle mechanism | Surfaces at | Carried by | Primary mitigation locus |
|---|---|---|---|---|
| ① Compounding reward hacking | An FA fidelity deficit inside $V_\nu$'s calibration support, tolerable per cycle, re-amplifies into a directional bias | UC | Agent edge, into FA if $V_\nu$ reads $\mathcal{A}_k$ | Tighten $F_\nu$ at FA, or shrink the update step |
| ② Distribution shift | Rollouts drift off $V_\nu$'s calibration support, where its errors are extrapolations | FA | Agent edge, environment path | Recalibrate $V_\nu$ at FA, or resample in EG |
| ③ Forgetting and plasticity loss | A moving goal distribution makes writes to $\Phi_\theta$ interfere, erasing earlier skills and eroding plasticity [192] | UC | Environment path, goal edge | Replay earlier goals in EG, or isolate or reset parameters |
| ④ Curriculum collapse | $\sigma_\omega$ contracts the goal support toward mastered regions, causing $\mathbb{E}[I_{\mathcal{A}_k}]$ to decay | EG | Goal edge, or agent edge into $\sigma_\omega$ | Inject exploration into $\sigma_\omega$, or score goals by difficulty |
| ⑤ Model collapse | Self-generated training data narrows $\pi_{\mathcal{A}_k}$ and erases its tails, driving $\mathbb{E}[I_{\mathcal{A}_k}]$ down under any sampler [193] | EG | Agent edge | Accumulate real or external data instead of replacing it [194] |

### 3.6 Cross-Cycle Coupling

Beyond the matrix's within-cycle bounds, the recurrence of Eq. (5) couples successive cycles. It feeds two cycle-$k$ outputs into Stage 1 of cycle $k{+}1$, the updated agent $\mathcal{A}_{k+1}$ that induces the next rollout policy and the extended history $\mathcal{H}_{k+1}$ that conditions the next goal draw, each defining a directed edge across the cycle boundary. Environmental drift, which Definition 4 holds fixed but live websites exhibit [200, 205], enters as a third path from outside the loop. These paths carry both self-reference and long-run degradation.

**Two Coupling Edges.** The two edges differ in carrier and conditionality. The *agent edge* runs from Stage 3 of cycle $k$ to Stage 1 of cycle $k{+}1$, carrying the persisted update into $\mathcal{A}_{k+1}$ and hence into the rollout distribution of $\mathcal{D}_{k+1}$. Present in every autonomous learning loop, it supplies the closure on which $\mathcal{C}_4$ rests and extends to any further component that reads the current agent: to $\sigma_\omega$ under a trained proposer [5, 22], to Stage 2 under an evaluator sharing $\Phi_\theta$ [6, 22], rendering $F_\nu^{(k)}$ endogenous, and to Stage 3 under a generative model that records lessons or skills [20, 162]. The *goal edge* runs from Stages 1 and 2 of cycle $k$ to Stage 1 of cycle $k{+}1$ through $\sigma_\omega(k{+}1, \mathcal{H}_{k+1})$, carrying the cycle-$k$ trajectories and their scores into the next goal draw. A stationary or scheduled sampler, reading no cycle-specific signal, leaves this edge inert, whereas an adaptive sampler activates it and superimposes a second feedback loop on the agent edge. Self-reference thus acts through the extensions of the agent edge and through the goal edge: a fixed-loop method retains only the agent edge into Stage 1, and a frozen snapshot satisfies self-reference without opening any of these paths.

**Five Degradation Modes.** We interpret long-run degradation through these paths under a *mitigation-locality reading*, in which the stage where a degradation surfaces, the path that carries it, and the stage where it is best mitigated need not coincide, and Table 6 records all three for five modes. Compounding reward hacking and distribution shift travel along the agent edge as the in-support and extrapolation variants of Goodhart's law [206], which reward-model overoptimization exhibits together [207], with shift also entering along the environment path [14, 200]. Forgetting and plasticity loss follow a moving goal distribution carried by the environment path or the goal edge [192], and the two collapse modes contract the support of EG, curriculum collapse through the goal edge or the agent edge into $\sigma_\omega$ [5, 131] and model collapse through the agent edge under any sampler [193]. The three loci diverge most for hacking and forgetting, which both surface at UC yet admit remedies upstream of the symptom, a tighter $F_\nu$ at FA [9] and replay in EG. Long-cycle drift compounds the first three modes within one task family and forms no sixth mode. The agent-edge extension into UC, absent from the table, surfaces through erroneous writes, within a single pass in Mobile-Agent-E [162] and across later rollouts in SEAR [20]. With existing loops running only a few rounds, these mechanisms are inferred from within-run and fixed-loop evidence, and the reading stops at the cycle boundary.

**Scenario Exposure.** The scenarios engage these paths unequally. In reasoning, a learned proposer extends the agent edge into $\sigma_\omega$ and pass-rate curricula open the goal edge [208, 209], both inviting curriculum collapse. Reasoning and generation both extend the agent edge into FA through self-referential verifiers, yet the outcome follows $\bar{F}$: under reasoning's high ceiling, $F_\nu^{(k)}$ drifts below $\bar{F}$, whereas under generation's low ceiling, compounding hacking sets in unless a frozen judge cuts that extension. In navigation and manipulation, task or environment changes drive shift and forgetting along the environment path and agent edge, and costly rollouts limit the EG resampling that counters them. Coupling thus extends the asymmetry across cycles.

## 4 Self-Evolution in Multimodal Reasoning

Multimodal reasoning occupies the quadrant of Figure 3 in which neither ceiling is low, its rollouts being single decoding passes and its short answers admitting exact match against a pre-collected or computed reference. Training against pre-collected keys under a stationary sampler forms a fixed loop, from which self-evolving methods depart along two branches. Weight-updating loops mostly pose their own questions or train on unlabeled ones, letting the agent's outputs supply most answers and tying the evaluator to the model it trains, which makes the realized fidelity $F_\nu^{(k)}$, not $\bar{F}$, the binding factor. Training-free loops keep a frozen VLM with the key or a program and let the agent write the skills or tools it reuses, several gating each write against a validation reference, moving the cost of self-reference to the frozen writer. Reported loops run within one training run, over up to nine rounds of weight updates or sixteen of training-free writes, or over one pass of a benchmark stream.

TABLE 7: **Self-evolving methods for multimodal reasoning.** Methods are grouped by whether the agent generates questions and visual inputs, and each loop component is tagged by its realizing party: A agent, B bootstrap, and E environment or task generation. Notation: $^{f}$ frozen-snapshot, $^{ema}$ moving-average, and $^{s}$ same-model evaluator; M memory, T tool, and I instruction update; $^{\dagger}$ cross-round and $^{\ddagger}$ single-point fidelity audit; *1 run* continuous updating within one run; – not reported.

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| ✣ *Questions from datasets or fixed generators* | | | | | | | | | |
| MM-UPT [23] NeurIPS'25 | B | unlabeled and pre-synthesized questions | – | A | consensus, majority vote | B | GRPO | 1 run | Qwen2.5-VL |
| TTRV [171] CVPR'26 | B | 20 unlabeled test inputs | – | A | consensus, answer frequency, entropy control | B | GRPO | 1 run | InternVL, Qwen |
| TTRSD [210] arXiv'26 | B | 20 unlabeled inputs | – | A | consensus, agreement across three views | B | token-selective GRPO | 1 run | InternVL3 and others |
| IQA-SelfEv. [211] ICLR'26 | B | unlabeled image pairs | – | A | consensus, pairwise majority vote | B | GRPO | 2 | Qwen2.5-VL |
| SelfJudge [212] arXiv'26 | B | unlabeled questions | 8 | A | consensus with self-judgment$^{f}$ | B | GRPO | 1 run | Qwen2.5-VL |
| Sherlock [213] NeurIPS'25 | B | 5K unlabeled questions per round | – | A | self-judgment, self-correction preference | B | trajectory-level DPO | 2 | Llama-3.2-V |
| ISR-DPO [214] AAAI'25 | B | unlabeled video questions | – | A | self-judgment, self-retrospective preference | B | iterative DPO | 9 | 7B video LMM |
| BUS [215] arXiv'26 | B | unlabeled questions | 8 | A | self-judgment, backward prediction over sampled paths | B | GRPO | 1 run | Qwen2.5/3-VL |
| Agent0-VL [216] ICML'26 | B | tool-integrated questions | 4/8 | A | self-judgment, tool-grounded self-critique | B | joint GRPO | 3 | Qwen2.5-VL |
| TVP [217] ICLR'26 | B | 501 benchmark test queries | – | A | self-judgment, program quality score | B | tool and example library (T+M) | 3 | GPT-4o, frozen |
| Vision-Zero [218] ICLR'26 | B | arbitrary images | 8 | E+A | computed, game outcome | B | self-play and RLVR | 1 run | Qwen2.5-VL |
| CwI [146] arXiv'26 | B | 30 constructive task families | – | E | computed, answer match | A | validation-gated skill library (T+I) | up to 12 | 4 frozen VLMs |
| Spatial Memory Ag. [219] arXiv'26 | B | verifiable spatial environment | – | E+B | computed and key, environment reward | A | memory cards (M) | episode | 4 frozen VLMs |
| EvoSpatial [220] arXiv'26 | B | spatial benchmark streams | – | E+A | computed checks and self-judgment | A | skill and tool library (T+I) | online | Gemma4-31B, frozen |
| Vision-SR1 [221] arXiv'25 | B | 47K labeled questions | – | B+A | key and self-judgment, perception re-answering | B | multi-reward GRPO | 1 run | Qwen2.5-VL |
| M$^2$Note [222] arXiv'26 | B | benchmark batches | – | B+A | key or self-judgment, rollback-gated | A | mistake notebook (M) | online | Qwen3-VL, GPT-5.4, frozen |
| VisualEvolve [223] arXiv'26 | B | labeled development tasks | – | B | key, failure diagnosis | A | validation-gated skill library (T+I) | 16 | 4 frozen VLMs |
| CoWorld [224] arXiv'26 | B | 120 labeled development tasks | – | B | key, answer scoring | A | selected harness (T+I) | 3 | GPT-5.6, frozen |
| Dynamo [225] arXiv'26 | B | labeled training subset | – | B | key, training-sample accuracy | A | gated skill-tool library (T+I) | 3 | 5 frozen VLMs |
| PhysNote [226] ICLRW'26 | B | 500 labeled physics videos | – | B | key, answer match | A | knowledge notes (M) | 1 pass | Qwen2.5-VL-72B, frozen |
| NeSy-Spatial [227] arXiv'26 | B | spatial benchmark streams | – | B | key, revealed after answering | A | skill library (T+I) | online | GPT-5.4, Gemini, frozen |
| MetaForge [228] arXiv'26 | B | labeled multimodal tasks | – | B | key, composite tool-use reward | A | GRPO + tool library (T) | 1 run | Qwen2.5/3-VL |
| SyncLoop [208] arXiv'25 | A | generator-evolved diagram problems | – | B | key, constructed answers | B | SFT and GRPO | 3 | Qwen2-VL |
| ✣ *Agent-generated questions* | | | | | | | | | |
| EvoLMM [22] arXiv'25 | A | unlabeled images | 5 | A | consensus, soft agreement | B | REINFORCE | 1 run | Qwen2.5-VL |
| VisPlay [5] CVPR'26 | A | unlabeled images | – | A | consensus, majority vote | B | GRPO | 3$^{\dagger}$ | Qwen2.5-VL, MiMo |
| V-Zero [7] arXiv'26 | A | unlabeled images | 5 | A | consensus, majority vote | B | iterative GRPO | 2 | Qwen2.5-VL |
| iReasoner [170] ACL Findings'26 | A | unlabeled images | 5 | A | consensus, step-level agreement | B | GRPO | 1 run | Qwen2.5-VL |
| Video-Zero [160] NeurIPS'26 | A | unlabeled videos, temporal spans | – | A | consensus, vote with temporal alignment | B | alternating GRPO | 5$^{\dagger}$ | Qwen3-VL |
| EvoVid [127] arXiv'26 | A | unlabeled videos, shuffle-sensitive questions | 10 | A | consensus, vote with temporal IoU | B | alternating GRPO | 3 | Qwen2.5/3-VL |
| RISE [126] arXiv'26 | A | unlabeled images, skill-balanced | 10 | A | consensus with supervisor$^{s}$ | B | alternating GRPO | 4$^{\ddagger}$ | Qwen2.5/3-VL |
| VISE [229] arXiv'26 | A | unlabeled images, localization queries | – | A | consistency under image transformations | B | REINFORCE | 1 run | Qwen3-VL and others |
| SVQ [230] arXiv'26 | A | images | – | A | self-judgment$^{f}$, rewrite and filter | B | SFT | 2 | Qwen2.5/3-VL |
| Trinity [231] arXiv'26 | A | images | – | A | self-judgment$^{ema}$, verifier scores | B | joint GRPO | 1 run | Qwen2.5/3-VL |
| Marvo [232] arXiv'26 | A | unlabeled images, tool-use questions | 8 | A | proposer answer, tool-use margin | B | alternating GRPO | 3 | Qwen2.5/3-VL |
| VideoEvoHarness [233] arXiv'26 | A | unlabeled videos, memory-grounded questions | – | A | proposer answer from video memory | A | validated harness (T+I) | 3 | Qwen3.7 Plus, Gemini, frozen |
| VQS [64] arXiv'26 | A | parsed records, template questions | 8 | E+A | computed, program answers | B | GRPO | 3$^{\ddagger}$ | Qwen3-VL |
| EVE [234] arXiv'26 | A | executable visual transformations | 6 | E+A | computed, code execution | B | alternating GRPO | 3 | Qwen2.5-VL |
| SpatialEvo [235] arXiv'26 | A | unlabeled 3D scenes, spatial questions | – | E+A | computed, geometric environment | B | GRPO | 4 | Qwen2.5-VL |
| DUEL [236] arXiv'26 | A | unlabeled images, perturbed claims | – | E+A | computed, claim-verification outcome | B | GRPO | – | Qwen2.5-VL |
| SAGE [237] arXiv'26 | A | dual transformations of labeled items | – | B+E | key and answer mapping, duality consistency | B | GRPO | 1 run | Qwen2.5/3-VL |

TABLE 7: **Self-evolving methods for multimodal reasoning** (continued).

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| ✣ *Agent-synthesized or retrieved images and videos* | | | | | | | | | |
| MM-Zero [6] EMNLP'26 | A | code-rendered images | 4 | A | consensus, majority vote | B | GRPO | 3 | Qwen3-VL, MiMo |
| Active-Zero [8] arXiv'26 | A | retrieved open-world images | 8 | A | consensus, majority vote | B | GRPO | 3 | Qwen2.5-VL |
| ViCo [238] NeurIPS'26 | A | edited scene graphs and chart tables | – | E+B | computed or key | B | GRPO and RWR | 5 | Qwen3-VL-8B |
| Tempo-Zero [209] arXiv'26 | A | program-rendered videos | 8 | E+A | computed, execution | B | GRPO | 5 | Qwen3-VL |
| PhysEvolve [239] arXiv'26 | A | simulated physical scenes | – | E+A | computed, physics engine | B | GRPO | 1 run | Qwen3-VL-8B |

## 4.1 EG Bound by Informative Goals

Experience generation in multimodal reasoning draws goals that pair a visual input (an image, a video clip, or frames of a 3D scene) with a question whose short final answer admits an equivalence check. The task families center on visual question answering over mathematical, chart, and diagram inputs [240, 241, 242], extending to pairwise quality judgments [211], temporally grounded video questions [127, 160], metric spatial questions over 3D scenes [235], and physical dynamics in videos [209, 226, 239]. These goals come from a dataset, a fixed generator, or generative roles of the agent, and in weight-updating loops the solver typically answers each goal in $K_k$ independent decoding passes, occasionally with tool calls, yielding a $\mathcal{D}_k$ that groups answers to the same question, whereas training-free loops run the frozen VLM on each goal with its current skills, tools, or notes. Table 7 summarizes the self-evolving methods that satisfy the conditions of Sec. 2.3.

**Quantity and Quality Constraints.** Experience generation is bound by quality at saturation (Table 3), the task structure leaving quantity slack. A rollout is a single decoding pass over a static input, an episode with $T=1$ except where tool calls precede the answer [216, 220, 224, 225, 228, 232], and leaves no side effect to revert, keeping both interaction cost and recoverability from lowering $\bar{K}$. Information gain, in contrast, arises in single-answer episodes only from goals whose pass rate under the current solver lies strictly between zero and one. A dataset or a fixed generator bounds the supply of such goals by how many lie near the edge of the current agent's competence, a number that depends on the backbone's pretraining and, by the mechanism that stalls self-play [131], shrinks as the agent masters these goals.

**Self-Posed Goals.** Larger groups do not appear to substitute for informative goals under self-consistency rewards, which lose contrast as groups grow, and loops raise the information gain instead by letting the agent pose its goals or by reweighting goals by pass rate, at a cost paid in feedback. Reported group sizes stay between four and ten rollouts per goal despite cheap decoding passes [6, 7, 8, 22, 126, 127, 215, 216, 218, 232, 234], and EvoLMM finds its self-consistency reward best at four to six samples, reporting over-consensus collapse at twelve [22], consistent with the reward, not the cost, limiting $K_k$. The agent enters $\sigma_\omega$ either as a generative role within $\Phi_\theta$ that synthesizes the question beside the solver, and in some loops the visual input, or through a rule that reweights dataset or generator goals by the solver's pass rate, alone in SyncLoop [208] or on top of a learned proposer [209, 237, 239]. Proposer-solver loops after R-Zero [243] reward goals of intermediate solver agreement through terms that vanish with $p_g(1-p_g)$ [126, 234], adding diversity terms against a narrowing of goals [5, 127]. These rewards are computed in FA yet reshape the goal distribution of EG. In VisPlay, a trained Questioner lifts Qwen2.5-VL-3B from 30.61 to 44.16 after one round against 33.77 for an untrained Challenger [5], a comparison that attributes about three quarters of the gain to learning the role. The gain is paid in feedback, where a self-posed question lacks a key unless a program computes its answer, leaving the answer to the agent's own outputs.

## 4.2 FA Bound by Realized Fidelity

Each cycle hands feedback acquisition groups of short answers to the same question, or the tool-calling episodes of a frozen agent, and the loops score each answer against one or more of three references (Table 7). The agent's own outputs (A), through a majority vote, a self-judgment, the proposer's answer, or consistency under image transformations, score self-posed or unlabeled questions [5, 22, 23]. An answer computed by task generation (E) scores questions whose answers programs, game rules, a geometric environment, or a physics engine determine [64, 218, 235, 239], and a pre-collected key (B) scores labeled sets [221, 223, 225]. Weight-updating methods mostly turn scores into group-normalized rewards for GRPO-style updates or into preference pairs for DPO [5, 7, 213, 214], whereas training-free methods use them to admit skills, tools, or notes into a library [146, 222, 225].

**Fidelity and Localization Constraints.** Feedback acquisition is bound by realized fidelity (Table 4), its attainable fidelity being high and localization imposing little. A short answer admits exact match against a reference fixed outside the agent and leaves no channel unchecked, and without a key the program-computed answers of VQS stay correct on 94.4% of retained questions among 500 human-audited ones [64], a lower bound on $\bar{F}$ under the conversion of Sec. 3.3. Consensus already falls below this bound within one round, with 24% of retained majority-vote labels and 18% of retained model-judge labels wrong in the same audit [64]. Across rounds, externally estimated pseudo-label accuracy falls from 72.0 to 61.0 over three iterations in VisPlay [5] and from 64.5 to 37.5 over five in Video-Zero [160], declines that bear on $F_\nu^{(k)}$ only jointly with rising question difficulty, to which VisPlay attributes its own. Gaming resistance of the vote under long optimization remains untested here, whereas prolonged training against a majority-vote self-reward in text-only reasoning ends in reward hacking and abrupt collapse [244]. Single-answer episodes fix $L_\nu=1$, the step-level agreement of iReasoner refining credit only within the

output [170], and only the short tool-integrated episodes of Agent0-VL, scored as a whole [216], leave credit coarser.

**Vote Preconditions and Evaluator Drift.** With localization free, every deficit of feedback acquisition is a fidelity deficit, and self-derived rewards make that fidelity a property of the model being trained. A vote recovers the model's belief and tracks $R$ only where answers admit an equivalence relation and the pretrained prior concentrates on the correct answer [80, 132], discrete answers supplying the first condition and the backbone, not the task structure, deciding the second. Within these conditions the reward form matters, TTRV lowering accuracy on four of five VQA benchmarks under a hard majority-vote reward and raising it on all five under a frequency-proportional reward with entropy control [171]. The evaluators further differ in whether they move with the policy: votes over current samples, parameter-sharing supervisors [126], and self-judging models [214] keep the agent edge into FA, judges frozen at the initial model cut it [212, 230], and the moving-average verifier of Trinity lags the policy by construction [231]. A frozen evaluator still shares the prior of the policies descended from it, removing drift but not a fixed bias, and reports under differing numbers of rounds leave the designs uncompared. Answers fixed outside the agent, by a key or a program, restore $\bar{F}$ only for labeled sets and program-computable families, the same restriction that experience generation imposes on the goals.

## 4.3 UC Bound by Retention

Update consolidation in multimodal reasoning persists the scored experience in two forms, weight updates and training-free writes (Table 7). Weight-updating methods write $\Phi_\theta$ with an update rule fixed before the loop, through group-relative policy gradients, preference optimization, or fine-tuning, either once per round over two to nine reported rounds [5, 64, 160, 209, 214, 235, 238] or continuously within a single training run. Training-free methods keep a frozen VLM and let the agent write text or code into skill and tool libraries [146, 220, 223, 225, 227], memory cards or notes [219, 222, 226], or harnesses of tools, skills, and execution code [224, 233], over benchmark streams or, in VisualEvolve, up to sixteen rounds, whereas a separate o4-mini abstracts the tool library of TVP [217]. MetaForge alone combines the two, registering validated tools that its policy forges during GRPO into a persistent library [228].

**Retention and Generalization Constraints.** Update consolidation is bound by retention (Table 5), inferred from non-monotone curves, not a measured loss, whereas generalization, though reported, is confounded with the backbone prior. No method measures retention on earlier-round goals, and the closest evidence, per-round scores on a fixed benchmark suite, traces the increment $\mathcal{J}_{k+1} - \mathcal{J}_k$ of Eq. (6) over the evaluation distribution, not the trained goals. These curves rise monotonically in Agent0-VL [216] but peak or dip mid-run elsewhere [5, 8], while single benchmarks reverse more often, MM-Vet falling from 48.62 to 38.07 over three VisPlay iterations as the average rises [5]. Among weight-updating methods, only ISR-DPO and Marvo run beyond five rounds, the former gaining through its ninth iteration [214] and the latter degrading after about the fifth or sixth of eight [232]. Parametric writes generalize more freely, reaching any goal that activates the updated weights and lifting standard benchmarks after training on synthesized questions [5, 7, 22]. These gains are consistent with elicitation of the backbone prior, most weight-updating methods resting on Qwen2.5-VL or Qwen3-VL, the family whose text-only counterparts gain even under spurious rewards [204]. V-Zero offers a partial control, reaching 51.9 against 50.8 for supervised GRPO on the same Qwen2.5-VL-7B backbone [7], and only an equal-budget non-evolving baseline would separate self-evolution from elicitation. Non-parametric writes act only when retrieved, and their transfer is tested directly, EvoSpatial evaluating libraries frozen after one benchmark on 18 others [220] and VideoEvoHarness moving an evolved harness to a second backbone [233].

**Reference-Gated Writes.** Consolidation inherits its exposure from the first two stages, and the two forms of writing differ in how they guard against it. Where the evaluator moves with the policy or a proposer supplies the goals, a parametric write is scored and driven by quantities that change with the update itself, and the update rule accepts every scored step without a fixed reference that could reveal a later round erasing an earlier gain. Several training-free methods gate their writes against such a reference, admitting a skill or change only on a validation or held-out gain [146, 233], promoting a skill-tool pair only on a strict gain over its training sample [225], rolling back note edits that do not raise batch accuracy [222], or selecting snapshots on fixed validation or development sets [223, 224], whereas TVP checks a new tool only against the outputs of the programs it replaces [217]. A guard on retention thus exists only where gated training-free writes keep a fixed reference.

## 4.4 Limits and Preconditions

**Bottleneck Attribution.** The binding bottlenecks of feedback and consolidation arise from self-reference, not from task structure. Both ceilings are slack for any learning loop, and a fixed loop that trains against pre-collected keys, as M-STAR does [152], scores its rollouts at the fidelity of an exact match. The weight-updating branch gives up this reference to pose and answer its own questions, its answers coming from the agent's own outputs, whose realized fidelity falls below $\bar{F}$, and its parametric writes lacking a fixed reference against which a later round could be checked. The training-free branch keeps the key or a validation set and admits self-reference only through the update, shifting the cost to the writer, whose frozen backbone both diagnoses a bottleneck and writes its remedy, its capability bounding both, a limitation that Dynamo reports for itself [225]. Self-reference repays part of this cost in experience generation, where a learned proposer relieves the saturation of fixed goal supplies and accounts for most of the first-round gain of VisPlay [5]. The few cross-round audits report declines confounded with rising question difficulty, and a large question-answering prior in the backbones may account for part of the gains in both branches, disqualifying the least constrained cell as a clean baseline for the other scenarios. This attribution rests on the stage evidence and on the provenance of each method in Table 7, not on a controlled comparison with fixed loops.

**Cross-Cycle Degradation.** Across cycles, self-reference exposes multimodal reasoning to degradations that a fixed loop

with keys avoids. On the goal side, learned proposers extend the agent edge into $\sigma_\omega$ and pass-rate curricula open the goal edge [208, 209, 237, 239], both inviting curriculum collapse. On the feedback side, self-derived rewards extend the agent edge into FA, inviting compounding reward hacking, and training on self-labeled answers invites model collapse along the edge into the rollout policy. The extension of training-free writers into UC remains untested, and cross-round evidence reduces to the Vision-Zero plateau, where a self-derived signal stops informing updates [218], and the late-round decline of Marvo [232], with per-round reversals of single benchmarks fitting hacking and forgetting alike.

**Presupposed Resources.** The mechanisms of multimodal reasoning presuppose resources that its short, discrete answers supply. Majority votes and agreement-based proposer rewards presuppose answers that admit an equivalence relation [22, 126, 234], program-computed answers presuppose a program that determines the answer at task generation [64], and reference-gated writes presuppose a key or validation set held outside the writer [146, 225].

## 5 Self-Evolution in Multimodal Generation

Multimodal generation occupies the quadrant of Figure 3 in which only the fidelity ceiling is low, an open-ended image or video having no reference output to be checked against outside sub-families such as object counts, rendered text, or program-drawn charts. Training against learned preference scorers fixed before the loop forms a fixed loop that this ceiling already binds, such scorers being exploitable under optimization [129, 245]. Weight-updating loops mostly derive the verifier from the agent, leaving the ceiling unchanged but letting the verifier's bias move with the policy and exposing the loop to compounding reward hacking. Training-free loops mostly let a frozen backbone judge its own outputs and write its memory or skills, sharing one bias between judge and writer. Reported loops run within one training run, over at most five rounds, or over one pass of a task stream.

### 5.1 EG Bound by Checkable Prompts

In multimodal generation, experience generation draws prompts as goals, paired with a source image in editing, and the outputs are images, videos, 3D scenes, charts, diagrams, or educational animations. The task families center on compositional text-to-image prompts [11, 12, 128], extending to aesthetic prompt optimization [246, 247], text rendering [245], safety [248], charts [191], text-to-video generation [249, 250], image editing [251, 252], and diagrams, 3D scenes, method illustrations, and animations built through tool chains [253, 254, 255, 256]. These goals come from a fixed pool or from generative roles of the agent, and weight-updating loops sample $K_k$ outputs per prompt in one pass each, whereas training-free loops run a frozen agent through a workflow of tool calls, renders, and edits. Table 8 summarizes the self-evolving methods that satisfy the conditions of Sec. 2.3.

**Quantity and Quality Constraints.** Experience generation is bound by quality at the coverage of checkable prompts (Table 3), the task structure leaving quantity slack. A rollout costs one sampling pass or an offline tool chain [103] and leaves no side effect to revert, with reported group sizes of three to sixteen samples per prompt [9, 13, 128, 129, 245, 257]. Information gain, in contrast, runs out at the edge of what a check can score, not at the edge of the agent's competence as in multimodal reasoning. Fixed pools come from benchmark categories whose attributes a detector, OCR, or a VQA model scores, such as those of GenEval and T2I-CompBench [10, 11, 128, 129], and a learning loop drives pass rates on such a pool toward one, where $p_g(1-p_g)$ approaches zero, as the fixed loop Flow-GRPO raises SD3.5-M from 0.63 to 0.95 on GenEval [195]. Open-ended prompts would still separate good outputs from poor ones under $R$, yet no check fixed before the loop scores them, leaving the loop unable to draw information from them. Loops drawing every round from one prompt distribution report diminishing gains by about the third round, consistent with this saturation [12, 248].

**Checkable Goal Composition.** A larger group adds samples of a prompt without extending the checkable pool, and loops raise $\mathbb{E}[I_{\mathcal{A}_k}]$ instead by letting the agent compose or reselect goals, at a cost paid in fidelity once the goals leave what a fixed check scores. Built-in checks keep composed goals within checkable sub-families, fixing answers at prompt construction in UniRL [128], decomposing each prompt into yes/no questions in SILMM [12], and restricting the specifications that RSI in UMMs plans from its own failures to charts that program execution verifies [191]. Goals beyond these sub-families fall to frozen external judges [258, 259, 260], whereas ASG poses understanding questions over unlabeled images, scored by the agreement of its own answers across framings at a fidelity that moves with the policy [40], and reselection stays within the pool, InternalGap regenerating the prompts that yielded no accepted image [11]. Controlled returns are reported only for built-in checks and external judges, RSI in UMMs reaching 59.0% on charts after four rounds against 46.3% for longer training on its original data and leading a fixed distribution of equally many verified charts in every round [191], and RSIgen gaining 4.7, 3.1, and 2.2 points over curricula compiled from frozen sources across its three attempts [260].

### 5.2 FA Bound by Attainable Fidelity

Each cycle hands feedback acquisition a group of outputs for the same prompt or the renders of one workflow, and the loops score each output as a whole (Table 8). The scores come from the agent itself (A), through the generator's confidence, its understanding branch, a lagged, snapshot, or co-trained judge, or the frozen backbone that runs the workflow, or from outside it, through an external judge (B) or program execution (E). Weight-updating methods mostly turn scores into group-normalized rewards for GRPO-style updates or into preference pairs for DPO [9, 12, 13, 129, 246], whereas training-free methods receive verbal critiques, checklist scores, or pass/fail verdicts [103, 253, 254, 264] that trigger repairs within an episode and gate persistent writes.

**Fidelity and Localization Constraints.** Feedback acquisition is bound by the fidelity ceiling (Table 4), to which signal source, channel coverage, and gaming resistance each contribute, while localization imposes little. An image admits no equivalence check against a fixed reference, leaving learned scorers and model judges, whose agreement with humans on open-ended content can turn negative, the frozen reviewer of ManimAgent correlating with blind ratings of 37 educational

TABLE 8: **Self-evolving methods for multimodal generation.** Methods are grouped by whether and how the verifier changes as the policy is updated, and each loop component is tagged by its realizing party: A agent, B bootstrap, and E environment or task generation. Notation: $^{f}$ frozen-snapshot, $^{p}$ previous-iteration, $^{c}$ co-trained, $^{ema}$ moving-average, and $^{s}$ same-model evaluator; M memory, T tool, and I instruction update; *1 run* continuous updating within one run; *1 pass* a single pass over the goals; – not reported.

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| ✣ *Self-verifiers updated with the policy* | | | | | | | | | |
| SOLACE [9] CVPR'26 | B | fixed prompt pool | 16 | A | denoising confidence under re-noising probes | B | Flow-GRPO | 1 run | SD3.5-M |
| IRIS [129] arXiv'25 | B | 553 GenEval prompts | 8 | A | negative self-certainty over text and image tokens | B | GRPO | 1 run | Janus-Pro |
| CoRL [13] NeurIPS'25 | B | COCO captions paired with images and QA | 8–16 | A+B | own token-embedding alignment, BLIP re-caption and LPIPS cycle consistency | B | joint then task-specific GRPO | 2 runs | Janus-Pro-1B |
| UniRL [128] arXiv'25 | B | constructed GenEval-category prompts | 3 | A | own answers against answers fixed at prompt construction | B | SFT and GRPO | 1 run | Show-o, Janus |
| SUDER [10] arXiv'25 | B | T2I-CompBench prompts, unpaired images | – | A | reverse-direction likelihood in both branches | B | SimPO or GRPO | 1 run | Janus-Pro |
| SILMM [12] CVPR'25 | B | LMM-composed compositional prompts | – | A | own answers to self-decomposed questions | B | DPO or KC-DPO | 3 | DreamLLM, SEED-LLaMA |
| InternalGap [11] ICLR'26 | A | T2I-CompBench++ prompts, discarded ones regenerated | 10 | A | own yes/no probability of alignment | B | SFT and DPO | 1 run | Janus-Pro, Show-o |
| Read It Back [257] NeurIPS'26 | B | AlphaGRPO20k prompts | 16 | A | own likelihood of the prompt given the image | B | AWM online RL | 1 run | BAGEL |
| GvU [261] arXiv'26 | B | 50K compositional prompts | – | A | own token-level likelihood of the prompt | B | GRPO | 1 run | X-Omni (LoRA) |
| ASG [40] arXiv'26 | A | self-posed questions over unlabeled images | 3 | A | answer agreement across seven framings, cycle captioning | B | GRPO on three LoRA adapters | 1 run | BLIP3o, BAGEL, VARGPT |
| Safe Codebooks [248] ICML'26 | B | harmful-prompt sets, self-rewritten safe prompts | – | A | own judgment of unsafe content | B | codebook projection, null-space fine-tuning | 5 | Janus, VILA-U, Emu3 |
| JarvisEvo [252] CVPR'26 | B | photo-editing instructions over tool chains | – | A | own evaluator, aligned to human scores fixed before the loop | B | SEPO RL and reflection fine-tuning | 1 run | Qwen3-VL-8B |
| ✣ *Co-evolving or lagged verifiers* | | | | | | | | | |
| SRL-VLM [246] ACL Findings'25 | B | raw prompts, 8 rewrites each, frozen SD1.5 | 8 | A | previous-iteration LVLM$^{p}$, aesthetics and alignment | B | DPO | 2 | VILA-1.5-8B |
| Image as Its Own Reward [245] CVPR'26 | B | PickScore, OCR, and GenEval prompt sets | 16 | A | PickScore or DINO model$^{c}$, Qwen-Image positives | B | GRPO | 1 run | SD3 (LoRA) |
| Dual-IPO [249] ICLR'26 | B | LLM-expanded caption pool | – | A | critic$^{c}$ refined on its pseudo-labels and a human seed set | B | Diffusion-DPO or KTO | 3 | CogVideoX, Wan-1.3B |
| ImageRSI [262] arXiv'26 | B | 530 requests, up to two repairs each | 1–3 | A | judge retrained on loop records, blinded A/B admission | A | experience library (I), judge SFT and GRPO | 1 judge update | Qwen-Image-2.0, frozen |
| UniEvo-VL [263] arXiv'26 | B | image generation prompts | – | A+B | moving-average teacher$^{ema}$ with external critiques | B | on-policy self-distillation | 1 run | Qwen-Image-2512 (LoRA) |
| ✣ *Frozen self-verifiers* | | | | | | | | | |
| PRISM [247] arXiv'26 | B | community prompts, 8 rewrites each, frozen SD3.5-M | 8 | A | initial VLM$^{f}$, dimension scores | B | GSPO | 2 | Qwen3-VL-8B |
| MT-OPSD [251] arXiv'26 | B | OmniEdit pairs, self-generated editing states | – | A | clean-conditioned checkpoint$^{f}$, gated promotion | B | on-policy self-distillation | 1 run | three editing backbones |
| ManimAgent [256] arXiv'26 | B | stream of 39 lesson tasks | – | E+A | renderer errors, keyframe review$^{s}$ | A | gated success and failure memory (M) | 39 tasks | GPT-5.5, frozen |
| COMFYCLAW [264] arXiv'26 | B | GenEval 2 and DPG-Bench prompts | – | A | region checklist$^{s}$, held-out gain gate | A | skill library (T) | – | Claude, Qwen, Gemma, frozen |
| SceneCraft [254] ICML'24 | B | synthetic scene queries | – | A | review of renders$^{s}$ against parsed constraints | A | constraint-function library (T) | batches | GPT-4V, frozen |
| EvoDesign [253] arXiv'26 | B | CanvasBench queries, alternative requests | – | A | refiner critique$^{s}$ of each render | A | three-tier design memory (M+I) | 5 | GPT-5.2, Gemini-3-Pro, frozen |
| SymbOmni [103] arXiv'26 | B | ComfyUI generation and editing instructions | – | E+A | compiler errors, evaluation agent$^{s}$ | A | symbolic concept memory (T+I) | online | Gemini 2.5 Flash, frozen |
| SIDiffAgent [165] arXiv'26 | B | GenAI-Bench prompts with edit loops | – | A | evaluation agent$^{s}$, aesthetics and alignment | A | trajectory memory (M) | 2 | Qwen2.5-VL-72B, frozen |
| Drawing Middleware [255] arXiv'26 | B | paper and method-illustration pairs | – | A+B | evaluator agent$^{s}$, reference illustrations | A | middleware library with rollback (T) | – | GPT-5, frozen |
| MemoGen [265] arXiv'26 | B | benchmark task set rerun each round | – | A+B | controller self-review, independent memory judge | A | experience memory (M) | 3 | Qwen-Image, GPT-5.5, frozen |
| PhysRepair [250] arXiv'26 | B | physics benchmark cases processed in sequence | – | E+A | re-diagnosis$^{s}$ with perception tools | A | repair experience memory (M) | case stream | GPT-6-Astra, MiniMax H3, frozen |
| ✣ *External or programmatic verifiers* | | | | | | | | | |
| RSI in UMMs [191] arXiv'26 | A | chart specifications planned from self-diagnosed failures | 2 | E | execution of own programs against the specification | B | fine-tuning of generation, understanding, programs | 4 | BAGEL-7B-MoT |
| MATE [259] arXiv'26 | A | branches pose challenges from evolving input pools | – | B | frozen UnifiedReward scorer, SigLIP source consistency | B | GRPO on both branches | 5 epochs | Janus-Pro-1B |

TABLE 8: **Self-evolving methods for multimodal generation** (continued).

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| RSIgen [260] arXiv'26 | A | curricula compiled from own probes | – | B | fixed external evaluator | B | agent and generator updates | 3 attempts | BAGEL, Qwen-Image, Wan-Image |
| OctoT2I [258] CVPR'26 | A | self-proposed prompts over concept dimensions | 5 | B | frozen MLLM probability of alignment | A | routing knowledge base (I) | 1 pass | Qwen2-0.5B router |
| GenEvolve [266] arXiv'26 | B | prompts with judge-extracted experience | 6 | B | Gemini KScore judge, program-sufficiency judge | A | GRPO, self-distillation from own experience-conditioned targets | – | Qwen3-VL-8B, Qwen-Image-Edit |

videos at $r=-0.17$ [256]. Instruments fixed before the loop score parts of the objective, such as detectors, OCR, and PickScore in Flow-GRPO [195] and HPS-v2.1, CLIP, and VideoAlign in DanceGRPO [267], and Flow-GRPO raises PickScore and OCR accuracy while human raters judge its images below the base model in quality [245]. Under optimization, IRIS attributes the realism drop of its HPSv2-trained baseline to reward hacking [129]. Only sub-families with a closed-form predicate escape the ceiling, RSI in UMMs gaining more on charts from execution-verified data than from the model's own best-rated images [191]. Localization stays near $L_\nu=1$, a single sampling pass forming one action and tool-orchestrated systems scoring renders with execution errors as their chief step-level signal [103, 256].

**Verifier Dependence on the Policy.** With localization nearly free, every deficit of feedback acquisition is a fidelity deficit, and the methods differ in how far the verifier depends on the policy it scores. A verifier that depends on the current policy keeps the agent edge into FA, letting $F_\nu^{(k)}$ move with each update and inviting compounding reward hacking, whereas weaker dependence slows or cuts that edge and leaves a bias fixed across cycles. Generator confidence depends on the policy entirely, and SOLACE and IRIS disagree even on its sign, rewarding higher denoising confidence and lower self-certainty, respectively [9, 129]. Understanding-branch checks hold only the prompt fixed and substitute for an external verifier only if the judging parameters are separated from the judged ones, the two directions are symmetric, and understanding exceeds generation. Shared weights violate the first condition, which KL penalties to a reference policy slow but do not remove [10, 13, 40, 128], the non-inverse relation between text-to-image and image-to-text weakens the second behind reverse-likelihood rewards [10, 257, 261], and the third, the internal gap that InternalGap converts into supervision [11], remains untested within a run [10]. Weaker dependence comes from co-evolving or lagged verifiers [245, 246, 249, 262, 263], from frozen self-verifiers [247, 251], including frozen backbones that also write the update [254, 256, 264], and from external or programmatic verifiers [191, 259, 260], and no report compares these designs at a matched number of rounds. Such designs thus trade a moving bias for a fixed one without raising $\bar{F}$.

## 5.3 UC Bound by Retention

Update consolidation in multimodal generation persists the scored experience in two forms, weight updates and training-free writes (Table 8). Weight-updating methods write $\Phi_\theta$ with an update rule fixed before the loop, through group-relative policy gradients, preference optimization, fine-tuning, or self-distillation, either continuously within a single training run or over two to five reported rounds [12, 191, 246, 247, 248, 249]. Training-free methods keep the generator and the agent frozen and let the agent write memory [165, 250, 253, 256, 265], skill, constraint-function, concept, or middleware libraries [103, 254, 255, 264], or routing records [258], over a task stream or up to five rounds. ImageRSI alone combines the two, admitting experience to a library while retraining its judge and not its generator [262].

**Retention and Generalization Constraints.** Update consolidation is bound by retention (Table 5), documented within runs and in RSI in UMMs over at most four rounds, whereas generalization is mostly untested and not shown to fail. Only RSI in UMMs measures retention on earlier-round goals, exact category text on an earlier vocabulary falling from 72% to 60% once a round replaces that vocabulary and recovering when later rounds keep half of the old specifications, while general understanding, which the chart goals do not exercise, erodes by 2.6 points on MMBench [191]. Within single runs, iteration robustness fails at the grain of optimizer steps, SOLACE degenerating into textureless images under prolonged optimization [9] and IRIS into meaningless images within hundreds of steps [129], and across rounds, per-round scores cannot separate saturation from erosion, as in the slight slip of DreamLLM under SILMM on DPG-Bench [12]. Generalization is tested only where a loop holds out evaluation goals [129, 253, 262], which MemoGen does not [265], and transfer is partial for both forms of writing, a memory that SIDiffAgent builds on GenAI-Bench raising DrawBench from 0.860 to 0.873 against 0.901 for in-domain memory [165] and the parametric writes of RSI in UMMs gaining only from 11.0% to 11.9% on a chart subset its authors judge too hard for open models [191]. Understanding-branch gains also remain entangled with the prior that judge and generator share in a unified model.

**Self-Judged Writes.** The exposure that consolidation faces comes from feedback acquisition, where the agent itself judges most writes, and the two forms of writing guard against it differently. Where the verifier depends on the policy, a parametric write is scored by the policy itself or by a judge that follows it, and the update rule admits every rewarded step, held back only by regularizers such as KL penalties [9, 10, 13, 40, 128, 129]. Training-free writes pass an admission test before they persist, yet the reviewer gating of ManimAgent [256] and the no-loss check of COMFYCLAW on three held-out prompts that the agent synthesizes [264] reuse the frozen model that judged the experience, protecting retention on that model's scale but not against its bias. Only ImageRSI, through paired validation on separate calibration

and holdout splits [262], and Drawing Middleware Evolution, through rollback against reference illustrations [255], test writes outside that model. Retention is thus protected only where a training-free write is tested outside the model that judged it, a condition that the key- or validation-gated writes of multimodal reasoning meet more often.

### 5.4 Limits and Preconditions

**Bottleneck Attribution.** In multimodal generation, the fidelity ceiling binds before self-reference, which changes the form of the deficit and leaves the ceiling in place. The fixed loops of the scenario, Flow-GRPO [195] and DanceGRPO [267] among them, already train against exploitable proxies [129], unlike the key-scored fixed loops of multimodal reasoning [152]. Weight-updating loops replace a proxy whose bias stays put with a policy-dependent verifier whose bias moves with the policy [9, 129], and training-free loops let the frozen model that judges an output also write the update, judge and writer sharing one bias. Goal selection is where self-reference earns back part of this cost, controlled returns being reported there, whereas The Image as Its Own Reward offers the only comparison of a moving verifier with a fixed one, by human preference after one run [245]. Both the ceiling and its cost rest on stage evidence and the provenance in Table 8, not on a controlled comparison.

**Cross-Cycle Degradation.** Across cycles, self-reference opens paths along which degradation follows the agent, with only one degradation measured across rounds. On the goal side, learned proposers extend the agent edge into $\sigma_\omega$, and the regeneration of failed prompts in InternalGap [11] and the pruning of prompts by pass rate in OctoT2I [258] open the goal edge, both inviting curriculum collapse. The goal edge also carries this one measured degradation, the forgetting in RSI in UMMs following a vocabulary change that a rule reading the agent's OCR score triggers [191]. On the feedback side, self-verifiers extend the agent edge into FA, inviting compounding reward hacking, observed only within single runs [9, 129], and training on self-generated images invites model collapse, which the narrowing of IRIS outputs under higher image self-certainty resembles without isolating [129]. Curriculum collapse, the extension into UC, and cross-round self-verifier fidelity remain untested.

**Presupposed Resources.** The mechanisms of multimodal generation presuppose resources that only part of its outputs supply. Understanding-branch checks presuppose an output that the agent's own perception can process and an understanding branch stronger than its generation [10, 11], built-in checks and execution-verified goals presuppose a sub-family with a closed-form predicate [128, 191], and admission gates presuppose held-out splits or reference sets outside the judging model [255, 262].

## 6 Self-Evolution in GUI and Web Navigation

GUI and Web navigation occupies the quadrant of Figure 3 in which only the quantity ceiling is low, each rollout being a multi-step episode that pays environment latency at every step, whereas the inspectable end state of an application or a web page keeps the attainable fidelity high in sandboxed environments, where per-task scripts read it. Much automated online learning here runs as fixed loops, whose tasks and verifiers come from external models or benchmark authors and never read the agent's record [101, 130, 200, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277]. Weight-updating loops allocate scarce rollouts by proposing goals or reweighting a fixed task set by its success record, and buy step-level credit with verifiers refit on the agent's trajectories, whose fidelity moves with the policy. Training-free loops keep the task set fixed and reuse each episode through memory or skills that a frozen backbone writes and judges, sharing one bias between judge and writer. Reported loops span one training run, a few rounds, or one pass over a task stream, mostly evaluated on the task sets they train on.

### 6.1 EG Bound by Rollout Cost

In GUI and Web navigation, experience generation draws goals that pair a natural-language instruction with the initial state of an application, desktop, or website, and each rollout is an episode of screenshots, often with an accessibility tree, and actions such as taps, typing, and scrolls, ending in success, failure, or a step limit. The task families span mobile apps in AitW, AndroidWorld, and AndroidLab [14, 17, 135], desktop software in OSWorld [16, 115, 278], and web navigation in WebArena and VisualWebArena sandboxes and on live WebVoyager and Online-Mind2Web sites [163, 201, 284]. These goals come from a benchmark task set, a task stream, or the agent's own proposals, and weight-updating loops run episode groups per goal in parallel instances, whereas training-free loops run one or a few episodes in sequence with a frozen backbone. Table 9 summarizes the self-evolving methods that satisfy the conditions of Sec. 2.3.

**Quantity and Quality Constraints.** Experience generation is bound by quantity at the episodes that parallel environment instances afford (Table 3), an episode cost that the single passes of multimodal reasoning and generation do not incur. A rollout spans up to 25 screen-action steps [130], each paying rendering and environment latency on top of policy inference, 2.5 to 4.0 seconds per step on the virtual machines of UI-TARS-2 [125], with DART-GUI reporting episodes that last tens of minutes [115]. Group sizes of eight to 64 rollouts per task [17, 101, 115, 130] are bought with parallel instances, up to 64 emulators in DigiRL [14], whereas training-free loops run each task once or a few times in sequence [162, 163, 280, 281]. Quality decides what each scarce episode returns, and on long-horizon tasks the loss falls mostly on goals beyond the agent's reach, where a binary outcome gives $p_g(1-p_g) \approx 0$, consistent with starting success rates of 11.3% on the professional software of SEAgent [16] and 18.1% on AndroidLab in UI-Genie [135].

**Reuse, Reweighting, and Goal Generation.** A larger group costs environment instances, and loops raise $\mathbb{E}[I_{\mathcal{A}_k}]$ instead along three routes, reuse, reweighting, and goal generation, which the inspectable end state keeps verifiable. Under stationary sampling, reuse raises the return of later episodes by conditioning them on memory and skills that the agent writes [162, 163, 278, 279, 280, 281, 282, 283] or by training the agent on steps that value functions or reward models fit on its rollouts select and label [14, 144], both lifting unreachable tasks toward informative success rates. Reweighting by the success record keeps the verifier of a fixed task set, lowering

TABLE 9: **Self-evolving methods for GUI and Web navigation.** Methods are grouped by whether and how the goal sampler reads the agent's record, and each loop component is tagged by its realizing party: A agent and B bootstrap. Notation: $^{c}$ co-trained and $^{s}$ same-model evaluator; M memory, T tool, and I instruction update; *1 run* continuous updating within one run; *1 pass* a single pass over the goals; – not reported.

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| ❖ *Stationary sampling of fixed goals* | | | | | | | | | |
| Mobile-Agent-E [162] arXiv'25 | B | five-task scenario sequences | – | A | Experience Reflectors over action and error histories$^{s}$ | A | Tips and Shortcuts (M, T) | 1 pass | GPT-4o, frozen |
| Agent S [278] ICLR'25 | B | exploration tasks, then OSWorld tasks | – | A | self-evaluator summaries of each task | A | narrative and episodic memory (M) | task stream | GPT-4o, frozen |
| MobileSteward [279] KDD'25 | B | cross-app instructions | – | A | StewardAgent evaluation$^{s}$ | A | expertise and guideline memory (M) | task stream | GPT-4o, frozen |
| AppAgentX [280] arXiv'25 | B | DroidTask, AndroidWorld, and A3 tasks, five repetitions | – | B | benchmark success evaluation | A | shortcut graph (M, T) | 5 | GPT-4o, frozen |
| WebCoach [163] ICLR'26 | B | WebVoyager stream of 643 tasks | – | A | actor self-evaluation, condensed by Qwen3-8B | B | cross-session episodic memory (M) | 1 pass | Qwen2.5-VL, Skywork-r1v3, frozen |
| EvoSkill-GUI [281] arXiv'26 | B | AndroidWorld, MobileWorld, and OSWorld tasks | – | A+B | isolated same-backbone critic, terminal task reward | A | skill packages (M, T) | $\leq$2 revisions per task | Claude, Qwen3.6, MAI-UI, frozen |
| MAGNET [282] arXiv'26 | B | AndroidWorld tasks, memory seeded from Amex | – | B | AndroidWorld success scripts | A | procedural and grounding memory (M) | 3 | Qwen2.5-VL-32B, frozen |
| DigiRL [14] NeurIPS'24 | B | AitW instructions | – | A+B | VLM evaluator with 2.8% error, value functions filtering steps$^{c}$ | B | offline-to-online RL with replay | 1 run | AutoUI |
| MagicGUI-RMS [144] arXiv'26 | B | real-world mobile tasks, 2,154 episodes per round | – | A+B | domain reward model on the agent backbone$^{c}$, GPT-4o arbitration | B | SFT of agent and reward model | 2+2 | Qwen3-VL-8B |
| Agent S4 [283] OpenReview'26 | B | synthetic OSWorld V2 training tasks | – | B | OSWorld V2 validation evaluators select parents | A | evolved interfaces and usage knowledge (T, I) | 15 | Qwen3.8-27B, frozen |
| ❖ *Adaptive sampling of fixed goals* | | | | | | | | | |
| DART-GUI [115] arXiv'25 | A | OSWorld tasks, rollout counts and length caps set by success rate | 8 | B | OSWorld evaluation scripts | B | decoupled multi-turn RL | 1 run | UI-TARS-1.5-7B |
| MobileRL [17] ICLR'26 | A | AndroidWorld tasks down-weighted after consecutive zero-reward epochs | 16 | B | rule-based AndroidWorld rewards, fine-tuned VLM for AndroidLab | B | AdaGRPO with positive replay | 1 run | Qwen2.5-VL-7B, GLM-4.1V-9B |
| UI-Genie [135] NeurIPS'25 | A+B | failed tasks of earlier rounds, manually crafted complex scenarios | 10, top 5 | A | reward model fit on continuation-rollout step labels$^{c}$ | B | SFT of agent and reward model | 3 | 3B, 7B, 72B MLLMs |
| ❖ *Adaptive generation of new goals* | | | | | | | | | |
| SEAgent [16] ICML'26 | A | tasks written from a guidebook of judged trajectories | 8 | B | World State Model trained on GPT-4o judgments, step-level | B | GRPO and adversarial imitation | 3 | UI-TARS-7B-DPO |
| ACuRL [15] arXiv'26 | A | tasks generated from agent experience and judge feedback | – | B | CUAJudge outcome judgment from key screenshots | B | RL | 3 | UI-TARS-1.5-7B, Qwen3-VL-8B |
| SCALE [284] CVPR'26 | A | website actions whose outcomes the agent is least sure of | – | A | predicted against observed next screen$^{s}$ | B | fine-tuning of Selector and Predictor | 20–60 hops | Qwen2.5-VL-7B, InternVL2.5-8B |
| SkillWeaver [201] arXiv'25 | A | skills proposed beyond the current library | – | A | success check from trajectory and final screenshot$^{s}$ | A | executable API library (T) | 160 per site | GPT-4o, frozen |

the rollout count of tasks with high running success in DART-GUI [115] and the sampling probability of tasks that earn no reward over consecutive epochs in MobileRL [17], or returning earlier failures [135]. Goal generation needs a verifier for each new goal, which reads the outcome off the screen through a model or judge fixed before the loop [15, 16] or a check that the agent itself runs [201, 284]. Each route carries a controlled return, and the most direct test of substitution favors reuse, EvoSkill-GUI with skills outperforming pass@3 without them on MobileWorld at a comparable token budget [281], while removing the failure filter of MobileRL lowers its AndroidWorld success [17] and random task generation lowers ACuRL's third-round LibreOffice Impress score from 40.7% to 36.4% [15].

## 6.2 FA Bound by Localization

Each cycle hands feedback acquisition an episode of screens and actions, which most loops score once at its end and loops with learned step judges score per step (Table 9). Scores from outside the agent (B) come from per-task scripts and rule-based rewards written with the benchmark [17, 115], the fixed evaluators of DigiRL and SEAgent [14, 16], and benchmark success [280]. Scores from the agent (A) come from value functions and reward models fit on its trajectories [14, 135, 144], the Judger role of the base model that SCALE fine-tunes in its other roles [284], and the frozen backbone that acts and judges its own episode in training-free loops [162, 163, 201, 278, 279, 281]. Weight-updating methods turn scores into group-normalized advantages or rejection-sampled training

targets [17, 115, 135, 144], whereas training-free methods turn them into success labels and diagnoses that gate memory and skill writes [163, 278, 281].

**Fidelity and Localization Constraints.** Feedback acquisition is bound by localization (Table 4), the attainable fidelity being high wherever the goal fixes an end state, though signal sources reach that ceiling unevenly. Task completion resides in the final screen, application database, or file system, which per-task scripts and rule-based rewards inspect directly [17, 115], whereas model judges fall short, EvoSkill-GUI's same-backbone critic erring on 28 of 143 audited failures and passing at least four as successes [281]. Free-form goals and live websites lack scripts and fall to screenshot judges [101, 200, 201], which see the screen but not the backend state. Under optimization, ACuRL audits its judge against on-policy trajectories over three rounds, with precision of 89.1% to 98.0% [15], whereas judge benchmarks score fixed trajectories [285] or the downstream success of policies trained on each judge [142]. Localization binds every source alike, even an exact checker spreading one verdict over episodes of up to 50 actions [281], with $L_\nu$ falling as episodes lengthen. Loops recover step-level signal at a credit-assignment cost through learned value functions and judges [14, 16, 135, 144] or a failure-diagnosing critic for training-free revision [281], step-level advantages speeding DigiRL's learning by about 12% on its Web Shopping subset [14].

**Localization Bought with Fidelity.** With end states checkable, feedback acquisition falls short in localization and lacks checkers for free-form goals, and loops open the agent edge into FA to buy either. A value function or reward model refit on the agent's trajectories follows the policy, letting $F_\nu^{(k)}$ move with each round and inviting compounding reward hacking, exploited errors re-entering the data on which it is refit [14, 135, 144]. MagicGUI-RMS slows this edge with a GPT-4o reward model that arbitrates its domain model [144], yet both it and UI-Genie track their reward models only on fixed labeled sets, excluding the on-policy rollouts where exploitation would surface [135]. A frozen self-judge keeps its bias fixed but shares it with the writer [162, 201, 278, 279, 281], its errors cutting revision short, as the over-optimistic EvoSkill-GUI critic shows. Verifiers fixed before the loop cut the edge, at the episode level in DART-GUI and MobileRL [17, 115] and at the step level in SEAgent's World State Model [16], which localizes credit independently of the agent but exposes the loop to distribution shift once rollouts leave its training distribution. Localization and free-form goal coverage thus cost either fidelity that follows the agent or, from a model fixed before the loop, distribution shift.

## 6.3 UC Bound by Generalization

Update consolidation in GUI and Web navigation persists the scored experience in two forms, weight updates and training-free writes, each method using one of them (Table 9). Weight-updating methods write $\Phi_\theta$ with an update rule fixed before the loop, either continuously within a single training run [14, 17, 115] or over two or three rounds through GRPO with adversarial imitation, reinforcement learning on generated tasks, or fine-tuning of the agent and its reward model [15, 16, 135, 144], while SCALE fine-tunes its Selector and Predictor over repeated exploration rounds [284]. Training-free methods keep the backbone frozen and let the acting model write narrative, episodic, procedural, or grounding memory [278, 279, 282], tips and executable shortcuts [162, 280], or skill packages and executable APIs [201, 281], over one pass of a task stream, a few attempts per task, or a fixed budget of practice iterations, whereas a separate condenser writes the memory of WebCoach [163].

**Retention and Generalization Constraints.** Update consolidation is bound by generalization (Table 5), documented within one platform and mostly on the task sets that produced the experience, whereas general capability, where measured, holds. No method measures retention on earlier-round goals, and the closest evidence concerns general capability, ACuRL keeping or raising its scores on untrained environments by 2 to 7 points [15]. Elsewhere, replay sustains iteration robustness [14, 17], and per-round scores rise without a reported reversal [135, 144, 284], though non-monotonically in Mobile-Agent-E [162], tracking $\mathcal{J}_k$ on the current evaluation set, not $S_\zeta$. Training-free libraries evolve on their evaluation tasks [162, 163, 281], as in several fixed loops [104, 164, 187], measuring adaptation to a known goal distribution. Held-out applications show transfer within the platform, MagicGUI-RMS gaining about as much on unseen apps as in domain [144], whereas ACuRL reaches a new software version or operating system only by training there [15]. Zero-shot transfer across platforms appears only in the fixed loop GUI-HARVEST, whose frozen harness, evolved on OSWorld, adds 6.47 points for Qwen3-VL-32B-Instruct and 13.87 for GPT-5 on WindowsAgentArena [286]. Cross-form comparisons are confounded by backbone, training-free loops mostly running frozen proprietary models and weight-updating loops open ones.

**Author Checks and Reader Dependence.** What a write carries into the agent is fixed in feedback acquisition, and the two forms of writing differ in what checks a write before it persists and, for non-parametric writes, in who reads it afterward. Parametric writes pass on the verifier's score alone, carrying its exposure unchanged, compounding hacking for agent-derived verifiers [14, 135, 144, 284] and distribution shift for SEAgent's frozen World State Model [16], while exact checkers leave little to inherit [17, 115]. Non-parametric writes pass an admission test that mostly reuses the author's judgment, EvoSkill-GUI and MobileSteward admitting skills and memories largely on their own critics' verdicts [279, 281], whereas SkillWeaver unit-tests each synthesized API against execution feedback, a check outside the writer [201]. Self-written skills can carry errors, such as the redundant or missing taps in the generated shortcuts of Mobile-Agent-E [162], unlike the key- or validation-gated writes of multimodal reasoning. Once admitted, a non-parametric write takes effect only when a later rollout reads it, its gain depending on the reader, as the memory that a separate condenser writes for WebCoach lowers Qwen2.5-VL-7B from 32.8% to 31.1% yet raises Skywork-r1v3-38B from 47.3% to 61.4% [163]. A non-parametric write is thus checked mostly by its author and valued by its reader.

## 6.4 Limits and Preconditions

**Bottleneck Attribution.** In GUI and Web navigation, the quantity ceiling binds fixed and self-evolving loops alike, and self-reference leaves it in place, its cost lying in what

each entry point puts at risk. Fixed loops pay the same episode cost, scaling to thousands of virtual machines in ComputerRL [269], and already train against signals fixed before the loop, the verification functions of ComputerRL at high fidelity and, below it, the screen judges of ZeroGUI [101] and MobileGUI-RL [130]. A success-conditioned rule over a task set with fixed checkers risks little, only the allocation following the agent [17], whereas generated goals move verification from fixed checkers to learned or prompted judges [15, 16, 201, 284], refit value functions and reward models buy localization with a fidelity that moves with the policy [14, 135, 144], and a frozen backbone that judges and writes carries one bias into both roles [278, 281]. The higher return of each scarce episode recovers part of these costs, controlled gains being reported for reuse, reweighting, and goal generation. No controlled comparison with fixed loops isolates these costs, which we read off the stage evidence and the provenance in Table 9.

**Cross-Cycle Degradation.** Across cycles, self-reference opens paths along which degradation follows the agent, and the environment path adds a shift specific to the scenario. On the goal side, success-conditioned rules and outcome-conditioned generators open the goal edge and agent-realized proposers extend the agent edge into $\sigma_\omega$ [15, 16, 17, 115, 135, 201, 284], inviting curriculum collapse, which MobileRL risks by design in removing persistently failing tasks [17], and forgetting once the goal distribution moves, which replay mitigates [14, 17]. On the feedback side, refit verifiers extend the agent edge into FA, inviting compounding reward hacking untested beyond the two or three reported rounds, and training on self-generated trajectories invites model collapse [135, 144, 284], whereas frozen self-judges share a fixed bias with the writer and open no such path. The extension into UC carries erroneous writes, which Mobile-Agent-E reports within a single pass [162], and the environment path carries the drift of live websites between cycles, independently of the agent [200, 205].

**Presupposed Resources.** The mechanisms of GUI and Web navigation presuppose resources that its environments supply. Repeated attempts at a goal, on which group sampling and reweighting rest, presuppose an environment that resets to a known state [17, 115], checkers and screen judges presuppose an end state that a program or a model can inspect [16, 115], and non-parametric reuse presupposes a reader able to exploit what the agent writes [163].

# 7 Self-Evolution in Embodied Manipulation

Embodied manipulation occupies the quadrant of Figure 3 in which both ceilings are low on hardware, each rollout costing robot time, a reset, and wear, and each outcome checkable only by a per-task classifier or a learned judge. Simulation relaxes both, where reinforcement learning against simulator success already forms fixed loops [62, 287, 288] and most self-evolving loops close before any hardware evaluation. The methods differ chiefly in feedback source: simulator predicates keep fidelity, agent-derived verifiers densify feedback at the cost of an agent-tied bias, and external judges buy fidelity with per-query latency. Self-reference enters mostly through these verifiers in weight-updating loops and through memory, skills, and code that a frozen model writes and judges in training-free loops. Reported loops run at most tens of rounds or a bounded practice budget, and hardware evidence rests on eight to thirty evaluation trials per task.

## 7.1 EG Bound by Physical Rollouts

In embodied manipulation, experience generation draws goals that pair a language instruction with a scene, and each rollout is an episode of camera observations and robot actions, emitted by a vision-language-action policy or by programs that a language model writes over perception and control primitives. The task families center on tabletop and household manipulation in LIBERO and its variants, ManiSkill, RoboCasa, RoboTwin, Robosuite, VIMA-Bench, and VLABench [69, 190, 293, 294, 295, 300], extending to memory-dependent tasks in RoboMemArena [292] and physical tasks on robot arms [19, 21, 301, 305]. These goals come from a benchmark suite, a library of scene-feasible tasks, or the agent's own proposals, and weight-updating loops run groups of episodes in parallel simulators, training-free loops sequential attempts, and hardware loops one episode at a time, with RoboEvolve imagining episodes inside a learned video simulator [298]. Table 10 summarizes the self-evolving methods that satisfy the conditions of Sec. 2.3.

**Quantity and Quality Constraints.** Experience generation is bound by both factors on hardware and chiefly by quality in simulation (Table 3). On hardware, quantity binds through physical time, human support, and scene recoverability, each real-world experiment of Self-Improving EFMs running about 20 hours with human station resets [19] and 6.58% of the 456 autonomous episodes of Find Something You Can't Do ending in a scene that only a human restores [21]. Simulation lifts this bound with parallel environments and group sampling [18, 62, 306], and where it also supplies a success predicate, reinforcement learning needs no agent-derived verifier and appears chiefly in fixed loops [95, 161, 287, 307, 308]. Quality binds on both sides, a long-horizon task beyond the policy's reach returning $p_g(1-p_g) \approx 0$, and on hardware a persistent workspace leaves only part of a task library feasible after each episode [21], each unreachable or infeasible task costing a full physical rollout.

**Task Choice and World Models.** A larger group is affordable only in simulation, which also supplies the success predicate, and loops pull two other levers: task choice raises $\mathbb{E}[I_{\mathcal{A}_k}]$ at a cost paid in verification, and a learned world model buys rollouts at a cost paid in fidelity. Task choice stays within a library or its edits, Find Something You Can't Do sampling scene-feasible tasks of low recent success [21] and Reflection-VLA simplifying failing tasks through edited simulation definitions [304]. Self-proposed practice extends this choice [20, 166, 302] and needs a check for each new goal, supplied by simulator predicates, by validation seeds that a sealed judge grades in SEAR [20], or by verifiers of the agent's own model in RATs [302]. Controlled returns are reported for both, removing the performance-aware curriculum of Find Something You Can't Do markedly slowing its real-world improvement [21], failure-ranked collection lowering the ManiSkill failure rate of Policy-Level RSI below random collection [296], and curious play lifting the RATs executor on LIBERO-PRO to 44.3% against 36.3% without play and 38.3% with random play [302]. A learned world model

TABLE 10: **Self-evolving methods for embodied manipulation.** Methods are grouped by whether the environment, the agent, or an external model supplies the feedback that decides each update, and each loop component is tagged by its realizing party: A agent, B bootstrap, and E environment. Notation: $^{f}$ frozen-snapshot, $^{c}$ co-trained, and $^{s}$ same-model evaluator; M memory, T tool, and I instruction update; Sim simulated, Real physical, and WM world-model loop, with → a later hardware evaluation; – not reported.

| Method | Experience Generation (EG) | | | Feedback Acquisition (FA) | | Update Consolidation (UC) | | | Backbone |
|---|---|---|---|---|---|---|---|---|---|
| | $\sigma$ | Goal content | $K_k$ | $V$ | Scoring reference & signal | $f$ | Update | Rounds | |
| ✣ *Environment feedback* | | | | | | | | | |
| EmbodiedRSI [166] arXiv'26 | A | experiments chosen by value of information (Sim→Real) | – | E | physical trial outcomes | A | code, skills, hierarchical memory (T, M), PPO memory policy | – | frozen VLA, Qwen3.5-122B |
| RoboBridge [289] arXiv'26 | B | LIBERO-PRO training seeds (Sim→Real) | – | E+B | completion predicate, held-out validation gate | A | versioned skill documents (M, I) | 2 | $\pi_{0.5}$, Codex, frozen |
| EmbodiedEvo [290] arXiv'26 | B | repeated attempts with scene resets (Sim) | – | E | completion, errors, termination status | A | executable skills, lessons (T, M) | – | coding agent, frozen |
| RoboHarn-Evo [291] arXiv'26 | B | interaction rollouts on RMBench (Sim, Real) | – | E | physical feedback on rollouts | A | hierarchical physical knowledge (M) | 80 roll-outs | GPT-5.5, GPT-6, frozen |
| BATON [292] arXiv'26 | B | subtasks of RoboMemArena tasks (Sim) | – | E | subtask outcomes, entry-state checks | A | verified subtask units (M) | – | GPT-5.6-sol, frozen VLA |
| Harness VLA [293] arXiv'26 | B | LIBERO-Pro, RoboCasa365, RoboTwin tasks (Sim) | – | E | primitive post-conditions, task success | A | global memory of verified sequences (M) | – | Codex or Claude Code, frozen VLA |
| ASPIRE [294] arXiv'26 | B | LIBERO-Pro, Robosuite, BEHAVIOR-1K tasks (Sim→Real) | – | E | execution and collision feedback | A | growing skill library (T) | – | Claude Opus 4.6, frozen |
| ViSkill [295] arXiv'26 | B | PPO rollouts on ManiSkill PrimitiveSkill (Sim) | – | A+E | binary success, skill-guided partial credit | A | PPO with skill cards (M) | – | Qwen2.5-VL-3B |
| Policy-Level RSI [296] arXiv'26 | A | force perturbations ranked by a failure model refit each round (Sim) | – | E | simulator failure labels | B | PPO with BC anchor, risk-gated specialist | 17 | MoE policy |
| MERL [25] arXiv'26 | B | perturbed LIBERO-PRO tasks, trust-scheduled imagination (Sim, WM) | – | E+A | terminal success, reward proxy refit on rollouts$^{c}$ | B | RL on real and imagined chunks | – | OpenVLA-OFT, world model |
| Scale and Selection [297] arXiv'26 | B | fixed robosuite and LIBERO cases, screenshots of a 3D interface (Sim) | – | E | built-in success detectors, champion-challenger acceptance | A | harness prompts, tools, control rules (I, T) | 30 | GPT-5.6 Sol, frozen |
| ✣ *Agent-generated feedback* | | | | | | | | | |
| SC-VLA [145] arXiv'26 | B | residual RL on ManiSkill3 (Sim, Real) | – | A | predicted progress and displacement$^{f}$ | B | SAC residual policy | – | flow-matching VLA, frozen base |
| Self-Improving EFMs [19] NeurIPS'25 | B | on-policy rollouts on robot stations (Sim, Real) | – | A | decrease in predicted steps-to-go$^{f}$ | B | REINFORCE | – | PaLI-3B |
| RoboEvolve [298] arXiv'26 | A | tasks parsed from seed images, rising difficulty (WM) | – | A | planner as zero-shot critic$^{c}$ | B | GRPO and DPO for planner and simulator | – | Qwen3-VL-4B, Wan2.2-5B |
| Motus2 [299] arXiv'26 | B | imagined branches from robot-data prefixes (WM→Real) | – | A | own value model on imagined futures$^{f}$ | B | DiffusionNFT on action parameters | – | Wan2.2-based world model |
| EEAgent [300] arXiv'26 | B | repeated trials on VIMA-Bench (Sim) | – | A+E | binary success, failure localization$^{s}$ | A | long- and short-term reflections (M, I) | – | GPT-4o, frozen |
| SHAPER [190] arXiv'26 | B | VLABench and ESI-Bench episodes (Sim) | – | A+E | binary success, round-level critique$^{s}$ | A | skill text, context code (I, T) | 4 | Qwen3.6-27B, frozen |
| SEAR [20] arXiv'26 | A | self-designed practice (Sim, Real) | – | A+B | own local checks, judge on validation seeds | A | code skills, written memories (T, M) | – | coding agents, $\pi_{0.5}$, frozen |
| PragmaBot [301] arXiv'25 | B | tabletop tasks (Real) | – | A | VLM success detector$^{s}$ | A | reflections in long-term memory (M) | task stream | GPT-4o, frozen |
| RATs [302] arXiv'26 | A | self-proposed play tasks (Sim→Real) | – | A+E | goal and per-step verifiers$^{s}$, deterministic checks | A | code skill library (T) | 50 | Gemini 3.1 Pro, frozen |
| RoboRSI [303] arXiv'26 | A | LIBERO and LIBERO-PRO tasks not yet solved (Sim) | – | A+E | reviewer postcondition checks$^{s}$, simulator success gate | A | skill hierarchy, guidance (T, M) | 5 | GPT-5.6-SOL, frozen |
| ✣ *External-model feedback* | | | | | | | | | |
| Reflection-VLA [304] arXiv'25 | A | LIBERO-Adapt, simplified at low success (Sim) | – | B+E | GPT-4o-composed dense reward, binary success | B | PPO and SFT on own successes | – | OpenVLA-7B, GPT-4o |
| EvoVLA [18] ECCV'26 | B | 8 parallel environments (Sim→Real) | – | B+A | Gemini stage dictionary scored by CLIP, pose curiosity | B | PPO with dual critics | – | OpenVLA-OFT |
| Find Something [21] arXiv'26 | A | feasible library tasks of low recent success (Real) | – | B | VLM success from pre- and post-images | B | residual off-policy RL | – | $\pi_{0.5}$, frozen base |
| Agentic-VLA [69] ICML'26 | B | LIBERO and RoboTwin tasks, LLM sub-goals (Sim) | 8 | B+A | VLAC progress, sub-goal weights by success | B | GRPO, memory of adapted weights (M) | – | OpenVLA-OFT |
| Robo-COP [305] arXiv'26 | B | deployment trials (Sim, Real) | – | B | text and video judges, windowed verification | A | LoRA adapters, skills, task memory (M, T) | – | $\pi_{0.5}$, Gemini 3.8 Flash |

makes rollouts cheap and recoverable but passes its errors to feedback, as in RoboEvolve, which co-trains its planner with

a video simulator from unlabeled seed images [298], and in World-VLA-Loop, which sets long-horizon suites aside once generated video degrades beyond about 300 frames [24].

## 7.2 FA Bound by Real-Robot Fidelity

Each cycle hands feedback acquisition an episode of camera frames and actions, together with the simulator state in simulation, and Table 10 groups the methods by who supplies the feedback. Scores from the environment (E) come from simulator predicates, execution errors, and termination status [290, 292, 293, 294, 296, 297]. Scores from the agent (A) come from progress heads and value models frozen before reinforcement learning [19, 145, 299], from a co-trained critic or a reward proxy refit on the policy's rollouts [25, 298], and from the frozen model that acts and then judges its own trials [20, 190, 300, 301, 302, 303]. Scores from outside the agent (B) come from VLM judges and critics [18, 21, 69, 304, 305]. Weight-updating methods turn scores into rewards or advantages for on- or off-policy reinforcement learning [18, 19, 69, 145, 299], whereas training-free methods turn them into success labels and diagnoses that gate memory, skill, and code writes [290, 294, 300].

**Fidelity and Localization Constraints.** Feedback acquisition is bound on real robots by fidelity and by a localization whose step-level signals inherit the fidelity of their estimators (Table 4), the attainable fidelity being high in simulation and low on hardware. A simulator predicate checks completion exactly, whereas on a real robot a judge must infer it from images with an accuracy reported between 69% and 100% across judges and test sets, from 100% for the best of five VLM judges on 100 image pairs in Find Something You Can't Do [21] to 78.9% for the judge of SOAR on 546 trajectories [173] and 69% to 75% for that of VideoAgent on 36 videos [172]. Cameras show an outcome's appearance and not contact or force, which judges reading images cannot observe. Under optimization, SEAR reports agents working around its sealed judge, for instance by reading object geometry from asset files [20], and policies trained against VLM judges learn to exploit them [143]. Localization comes only from learned estimators, such as progress heads, VLAC critics, and per-step verifiers, which return a signal at every step or sub-goal [19, 69, 145, 302], whereas a simulator predicate or a real-robot judge returns one verdict per episode.

**Dense Feedback from Learned Verifiers.** A simulator predicate stays fixed as the policy improves, whereas the learned verifiers that make real-robot feedback dense or cheap incur degradations set by whether they follow the policy. Progress heads and value models frozen before reinforcement learning cut the agent edge [19, 145, 299] but meet distribution shift on states that the improving policy reaches outside its imitation data, the key challenge Self-Improving EFMs names for its reward model [19]. Co-trained critics keep the edge and invite compounding reward hacking, which RoboEvolve counters by binarizing its physical reward terms, citing as stability monotone reward curves that hacking would equally produce [298], and MERL by down-weighting low-trust imagined chunks, outperforming a matched control [25]. A frozen self-judge opens no path into FA but writes its misjudged trials into the memory or skills it later retrieves [20, 190, 300, 301, 302], unless a sealed judge checks them on validation seeds, a judge that the agents of SEAR still circumvent [20]. External judges stay independent of the policy but cost 2.6 to 16.9 s per query in Find Something You Can't Do [21], bounding the rollouts they can score. Dense feedback on hardware thus rests on fidelity audited within one task distribution, for judges on at most a few hundred samples [21, 172, 173] and for the progress head on target-task demonstrations [19].

## 7.3 UC Bound by Generalization

Update consolidation in embodied manipulation persists the scored experience in weight updates, training-free writes, or, in four methods, both (Table 10). Weight-updating methods write $\Phi_\theta$ with an update rule fixed before the loop, through reinforcement learning or fine-tuning on their own successes, RoboEvolve updating its video simulator alongside its planner [18, 19, 21, 25, 145, 296, 298, 299, 304]. Training-free methods keep the backbone frozen and let the agent's language model write reflections [300, 301], skills, code, or harnesses [20, 190, 289, 290, 294, 297, 302, 303], or memory of verified behaviors and physical knowledge [291, 292, 293], over two to 50 rounds, a rollout budget, or one pass of a task stream. ViSkill [295], Agentic-VLA [69], and Robo-COP [305] train the policy alongside skill cards, a memory of adapted weights, or admitted skills, and EmbodiedRSI keeps its VLA frozen while training the policy that manages its memory of code and skills [166].

**Retention and Generalization Constraints.** Update consolidation is bound by generalization (Table 5), success falling with the distance of the test from practice even where gains persist within one benchmark, whereas retention is measured only in fixed loops, self-evolving ones guarding against regression by gates. The code policies of SEAR fall from 87.8% on new initial states of practiced tasks to 33.3% on new tasks and 28.7% under a new simulator and action interface, and its trained VLAs from 73.2% to 43.1% and 10.2% [20]. Gains of written artifacts persist within one benchmark, SHAPER adding 2.5 points on its in-domain VLABench split and 6.0 to 10.0 on splits with unseen targets or task forms [190], although its absolute success on the splits with unseen targets falls from 42.5% to 26.0% and 19.5%. Hardware transfer is tested on small sets, five tasks of 30 trials each in EmbodiedRSI [166], eight scenes per task in RoboHarn-Evo [291], and 20 tests per task in Robo-COP, which doubles success on only one of three held-out tasks [305]. Retention is measured only in the fixed loops CRL-VLA and Simple Recipe [95, 161] and approximated in self-evolving loops only by gates against regression, the adoption test of Robo-COP and the risk gate of Policy-Level RSI [296, 305]. As in navigation, backbones confound cross-form comparison: training-free loops mostly run frozen frontier models, weight-updating loops open ones.

**Tests outside the Writer.** Feedback acquisition sets the exposure of consolidation, the two forms of writing differing in whether a test outside the writer checks a write before it persists. Parametric writes pass on the verifier's score, inheriting the distribution shift of frozen progress heads [19, 145, 299] and the compounding hacking of co-trained critics [25, 298], except where the adoption test of Robo-COP or the risk gate of Policy-Level RSI adds a test

outside the update rule [296, 305]. Non-parametric writes pass tests at different distances from the writer, from held-out seeds and fixed validation sets [190, 289, 297] through execution outcomes [290, 292, 293, 294] to the writer's own judgment [20, 300, 301, 302]. A write that only its writer checks persists with its errors, one SEAR run saving a self-calibrated placement verifier and finishing at 4.6% against 15.5% and 23.6% for two sibling runs without it [20]. A test outside the writer matters even when it checks only task success, the harness of Scale and Selection falling from 40% to 0% over 20 rounds once every revision is accepted, whereas champion-challenger selection raises held-out success from 51% to 67% [297]. The farther the test sits from the writer, the more of the retained gain it protects.

### 7.4 Limits and Preconditions

**Bottleneck Attribution.** In embodied manipulation, both ceilings bind fixed and self-evolving loops alike on hardware, self-reference changing who supplies the goal, the score, or the write but not the cost or attainable fidelity of a physical rollout. Fixed loops meet the same ceilings, training against simulator success [62, 287] or letting a separate model write memory or revise a skill library [188, 309, 310, 311]. The cost of self-reference lies in what each entry point puts at risk: a success-conditioned choice over a fixed library risks little, only the allocation following the agent [21], whereas self-proposed practice moves verification to checks the agent builds [20], a co-trained critic lets its bias follow the policy [25, 298], and a frozen model that judges and writes carries one bias into both roles [300, 301]. Task choice and practice offset part of these costs, and only there are controlled gains reported [21, 302]. Beyond these ablations, the costs rest on stage evidence and the provenance in Table 10.

**Cross-Cycle Degradation.** Across cycles, self-reference opens paths along which degradation follows the agent, and the environment path adds the sim-to-real gap. On the goal side, success-conditioned task choice, simplification, and failure-ranked collection open the goal edge [21, 296, 304] and self-proposed practice extends the agent edge into $\sigma_\omega$ [20, 166, 298, 302], both inviting curriculum collapse. On the feedback side, co-trained critics extend the agent edge into FA, inviting compounding reward hacking [25, 298], and training a planner on videos of its co-trained simulator invites model collapse [298], whereas frozen progress heads stay off the agent edge yet admit distribution shift [19]. The extension into UC carries erroneous writes [20] and regression when every checkpoint is deployed, below the frozen policy on three of the four tasks whose first candidate its gate rejected [305], while the environment path carries dynamics that loops closed in simulation never score. Within reported budgets, regression and erroneous writes are documented, whereas curriculum collapse, compounding reward hacking, model collapse, and distribution shift remain inferred.

**Presupposed Resources.** The mechanisms of embodied manipulation presuppose resources that simulation supplies and hardware withholds. Reinforcement learning against success presupposes simulator success predicates [62], held-out gates simulated or deployment trials [289, 305], learned hardware feedback an audited judge [21, 173], and written artifacts a test outside the writer [20, 297].

## 8 Cross-Scenario Asymmetry

This section characterizes the cross-scenario asymmetry, in which the two directions of a scenario pair receive different kinds of portability verdict, a mechanism that applies in one scenario needing an adaptation, being blocked, or becoming redundant in the other while the reverse direction meets a different combination. We interpret the asymmetry through two properties of each scenario, which factors are scarce there, exogenously or through resources that self-reference gives up, and which task-structural resources and conditions it holds, chiefly output equivalence, checkable outcomes, and outcome observability. A mechanism is portable on a target that holds the resource it presupposes, conditional where that resource is held only on a sub-family or after an adaptation, blocked where the outcome it judges or must check is unobservable, and redundant where the factor it raises is not scarce. The account concerns the kind of verdict each direction receives and leaves aside cost differences across scenarios, which the generation-verification gap would set. Quadrant placements and cost comparisons are inferred from single-scenario evidence, and the verdicts remain untested.

### 8.1 Exogenous and Endogenous Scarcity

A factor is scarce in a scenario where its value limits what a loop gains, a condition that a binding ceiling implies but that a factor without a ceiling can also meet. Scarcity is exogenous where it limits fixed loops as well and endogenous where self-reference gives up a resource that a fixed loop keeps, two reasons that comparing the self-evolving and fixed loops of each scenario separates (Secs. 4.4, 5.4, 6.4 and 7.4).

**Exogenous Scarcity.** Exogenous scarcity has two sources, the saturation of $\mathbb{E}[I_{\mathcal{A}_k}]$ in every scenario and the low ceilings of three. We infer the saturation from $p_g(1-p_g)$ falling near zero where goals are mastered or out of reach. Generation saturates from above, the fixed loop Flow-GRPO raising the GenEval score of SD3.5-M from 0.63 to 0.95 [195], and navigation and manipulation from below, with initial navigation success below 20% [16, 135] and physical scenes that afford only part of a task library [21]. Unbounded by any ceiling, $\mathbb{E}[I_{\mathcal{A}_k}]$ counts as scarce in every scenario. Fixed loops meet the low ceilings as well, scoring open-ended images with learned preference models in generation [195, 267, 312, 313, 314, 315], paying per-step environment latency in navigation [269], and facing resets and success evaluation on hardware [21, 188]. Self-reference leaves these ceilings in place, changing who supplies the goal, the score, or the write and with it the form of their cost.

**Endogenous Scarcity.** Endogenous scarcity, the residual bottleneck we place in multimodal reasoning, where neither ceiling is low, arises once loops give up the answer keys that fixed loops keep. Fixed loops filter rollouts by exact match against pre-collected keys, as in M-STAR [152], whereas loops that pose or label their own questions without a program to compute answers label by consensus or self-judgment at a lower realized fidelity $F_\nu^{(k)}$. On 500 shared questions, VQS finds 24% of retained majority-vote labels and 18% of retained model-judge labels wrong against 5.6% of retained program-computed answers [64]. Self-posed goals also give up scripts in SCALE [284] and SEAR [20] without an endogenous scarcity of fidelity, SCALE's web end states remaining open

to inspection and SEAR's goals staying in simulation, short of the hardware on which both ceilings bind.

## 8.2 Task-Structural Resources

Whether a meta-operator that the agent realizes can raise a scarce factor varies across scenarios only with resources that the scenario holds independently of the agent. We call *task structure* the part of a scenario fixed before any choice of agent or meta-operator, namely the environment family and evaluation distribution of the macro-environment (Definition 4) without its goal sampler, together with the interfaces through which a loop resets, replicates, or inspects its environments, a property being task-structural when replacing the agent and its three meta-operators leaves it unchanged. *Task-structural resources* decide whether a mechanism that presupposes them is defined or affordable, whereas capabilities of the agent's model and artifacts authored before the loop, the two other kinds a mechanism can presuppose, hold on every target.

**Resources by Scenario.** Three task-structural resources recur across the scenarios and are held unequally, output equivalence, checkable outcomes, and resettable rollouts. Output equivalence, held by short discrete answers but not by images or action trajectories, defines majority voting without converting outputs into answers. Checkable outcomes give each newly posed or reweighted goal its own reference outside the agent, coming from keys, generation programs, state scripts, or screen judges [14, 64, 115, 152, 316, 317, 318, 319]. Resettable, parallelizable rollouts let group sampling scale, at the price of emulator and virtual-machine instances where episodes run in software environments [14, 125]. Reasoning holds all three, navigation checkable outcomes and bought rollouts, generation free rollouts with checks only on program-checkable sub-families [191, 195], and manipulation checks and bought rollouts only in simulation [62], hardware episodes costing physical time [19, 21]. The verdicts turn chiefly on output equivalence and checkable outcomes, rollouts entering them only through $\bar{K}$, which decides whether world models are redundant. The ceilings of Figure 3 reflect these resources only in part, navigation keeping a low $\bar{K}$ under per-step latency despite resettable rollouts and sharing a high $\bar{F}$ with reasoning, which alone holds output equivalence. Portability is therefore derived from the resources of each scenario, not read off its quadrant.

**Observability and Online Checks.** Two further conditions decide whether a held resource serves the loop, the observability of the outcome, which task structure fixes, and an online check built on the resource, which the loop's design supplies or omits. The outcome is observable where it appears in the agent's observations or in a check available to the loop, a condition that every mechanism judging or checking it presupposes. An *online check*, a check built on a resource that runs inside the loop, is the only route by which the resource lowers the realized cost of a trade-off, and loops run one wherever answer keys, execution results, or simulator outcomes are available, training-free methods checking their writes against them [146, 201, 290]. Holding a resource does not ensure such a check, task streams, open prompts, and self-chosen practice tasks leaving none even in navigation, as in WebCoach's task stream [163]. The frozen acting model then judges its own episodes and writes the update, as in ManimAgent [256], Mobile-Agent-E [162], and SEAR [20], and an error of judgment enters the update unchecked, one simulated SEAR run saving a self-built verifier wrong on 91% of later rollouts. A loop without an online check thus gives up a resource that its scenario holds.

## 8.3 Trade-offs of Self-Evolving Mechanisms

Table 11 recasts the mechanisms of the scenario chapters as trade-offs, in which an agent-realized meta-operator raises a factor by supplying what the agent records, proposes, judges, or writes in place of a component fixed before the loop. The goal sampler supplies the agent's success record or its own proposals in place of a fixed goal distribution, the feedback function the agent's judgment in place of an answer key or a checker, and the evolution operator the agent's authorship in place of an update rule written in advance. Each trade-off pays what the supplied output lacks relative to the replaced component and presupposes a resource that is task-structural, a capability of the agent's model, or an artifact authored before the loop. Of its three elements, the factor raised, the resource presupposed, and the price paid, only the first two enter the portability verdicts.

**Experience Generation.** The mechanisms of experience generation raise $\mathbb{E}[I_{\mathcal{A}_k}]$, scarce in every scenario, and differ in what they give up to raise it, with a learned world model as the one exception. Success-conditioned allocation reweights a fixed task set, giving up coverage of the tasks it down-weights and presupposing checkable outcomes on that set. Created goals, posed by the agent or generated from its record, extend the factor beyond a fixed pool but give up the answer key or checking script of each pooled goal, presupposing in its place an outcome that something other than their generator can check or answers that a vote can compare. Program-checkable goal families keep the check by creating goals only where a program answers or renders them, as in VQS [64] and RSI in UMMs [191], giving up coverage of $\mathcal{G}$ outside the family. A learned world model, the one mechanism that replaces a component of the environment and not a fixed component of a meta-operator, instead raises $K_k$, replacing the environment dynamics $P$ with imagined rollouts that $\bar{K}$ does not bound and presupposing dynamics learnable over short horizons, the trade-off that model-based reinforcement learning has analyzed from Dyna [320] to MBPO [321]. It enters the loop through $V_\nu$, its imagined outcomes scored by a reward or value head [25, 299] or by the agent's own judge [298], and model error thus reaches $F_\nu$.

**Feedback Acquisition.** Each feedback mechanism pays for the substituted judgment in fidelity, its gain depending on the evaluator it replaces, and the mechanisms fall into three groups by the kind of resource they presuppose, the distinction on which their verdicts turn. The self-consistency vote raises $F_\nu$ without keys at a price of several rollouts per goal and presupposes output equivalence together with a prior concentrated on the true answer. Self-judgment and generator-confidence rewards raise $F_\nu$ without checkers and presuppose only capabilities of the agent's model, a generation-verification gap or the generator's confidence, which hold on every target, the gap pricing these rewards without deciding where they port. Verifiers refit on the agent's trajectories and frozen self-derived estimators raise

TABLE 11: **Trade-offs and degradation of self-evolving mechanisms.** *Evidence* stops short of cross-round measurement for most degradation modes. Notation: (T) task-structural resource, (M) capability of the agent's model, and artifact authored before the loop if unmarked; *cut* path closed by a frozen evaluator.

| Mechanism | | Trade-off | | | Degradation | | | Instances |
|---|---|---|---|---|---|---|---|---|
| **Name** | **Entry** | **Raises** | **Requires** | **Pays** | **Path** | **Mode** | **Evidence** | |
| ❖ *Experience generation* | | | | | | | | |
| Success-conditioned allocation | $\sigma_\omega$ | $\mathbb{E}[I_{\mathcal{A}_k}]$ | fixed task set with checkable outcomes (T) | coverage of tasks dropped by design [17] | goal | curriculum collapse | untested | MR [208], MG [11], GW [17, 115], EM [21, 296] |
| Agent-posed goals | $\sigma_\omega$ | $\mathbb{E}[I_{\mathcal{A}_k}]$ | checkable outcomes or output equivalence (T) | pooled keys or scripts, label accuracy 72% to 61% [5] | agent | curriculum collapse | untested, countered by diversity terms [5, 127] | MR [5, 22], MG [40, 260], GW [201, 284], EM [20, 302] |
| Record-conditioned goal generation | $\sigma_\omega$ | $\mathbb{E}[I_{\mathcal{A}_k}]$ | checkable outcomes or output equivalence (T) | pooled keys or scripts | goal | curriculum collapse | untested | GW [15, 16] |
| Program-checkable goal families | $\sigma_\omega$ | $\mathbb{E}[I_{\mathcal{A}_k}]$ at a program-checked $F_\nu$ | program, simulator, or scripted scene checks (T) | coverage outside the family | agent or goal | forgetting | 72% to 60% after a vocabulary switch [191] | MR [64, 218, 235], MG [191], EM [304] |
| Imagined rollouts in a world model | $V_\nu$ | $K_k$ beyond $\bar{K}$ via imagined rollouts | learnable short-horizon dynamics (T) | $F_\nu$ via model error, video degrades past 300 frames [24] | agent or cut | distribution shift, model collapse | inferred, imagined chunks weighted by trust [25] | EM [25, 298, 299] |
| ❖ *Feedback acquisition* | | | | | | | | |
| Self-consistency vote | $V_\nu$ | $F_\nu$ without keys | output equivalence (T), prior on the true answer (M) | $K_k$ rollouts per goal, label errors [64] | agent | compounding hacking | text-only reasoning [244] | MR [7, 22, 171], MG [40] |
| Self-judgment by the agent's model | $V_\nu$ | $F_\nu$ without checkers | generation-verification gap (M) | fidelity moving with the policy | agent | compounding hacking | untraced across rounds [10] | MR [126, 214], MG [10, 11], GW [162, 278], EM [298, 301] |
| Generator-confidence reward | $V_\nu$ | $F_\nu$ without checkers | generator's confidence (M) | fidelity tied to updated weights | agent | compounding hacking | within single runs [9, 129] | MG [9, 129] |
| Verifier refit on agent trajectories | $V_\nu$ | $L_\nu$, or $F_\nu$ on free-form goals | labeled seed or reference set | fidelity moving with the policy | agent | compounding hacking | audits on fixed labeled sets [135, 144] | MG [245, 262], GW [14, 135, 144], EM [25] |
| Frozen self-derived estimator | $V_\nu$ | $L_\nu$, or $F_\nu$ on free-form goals | demonstrations or $\mathcal{A}_0$ calibration | fixed bias off the training support | cut | distribution shift | named as the key challenge [19] | MR [212, 230], MG [247, 251], EM [19, 145, 299] |
| ❖ *Update consolidation* | | | | | | | | |
| Episode reuse via memory or skills | $f_\zeta$ | $G_\zeta$, $\mathbb{E}[I_{\mathcal{A}_k}]$ on hard goals | writer's generation-verification gap (M) or an online check (T) | exposure to the writer's errors | agent | erroneous writes | wrong verifier saved to memory [20] | MR [146, 225], MG [256, 264], GW [162, 281], EM [20, 290] |

$L_\nu$ besides $F_\nu$ and presuppose an artifact authored before the loop, a labeled seed, a reference set, or demonstrations.

**Update Consolidation.** Update consolidation contributes one trade-off, episode reuse through memory, skills, or harnesses, which raises $\mathbb{E}[I_{\mathcal{A}_k}]$ on goals initially out of reach and, where a written library carries beyond $\mathcal{G}_k$, also $G_\zeta$, paying in exposure to the writer's errors. It presupposes either the writer's generation-verification gap, a capability held on every target, or an online check that gates the writes, as keys and validation sets do in training-free reasoning loops [146, 222, 225]. Weight updates under a rule fixed before the loop supply no agent authorship and enter the trade-offs only through the feedback they consume.

## 8.4 Costs and Alternative Sources

The evidence on the price paid, the trade-off element the verdicts leave out, serves the account in two ways, setting the cost of the conditional routes the verdicts open and bearing on whether a factor other than scarcity and resources could produce the asymmetry. The comparisons below each stay inside one paper, bearing on a mechanism's gain and, where measured, its cost. Degradation, the rest of the cost, is left aside, its evidence too uneven for cross-scenario comparison.

**Generation-Verification Gap.** The generation-verification gap prices a trade-off wherever the agent's own judgment serves as the reference, as a capability that holds on every target (Sec. 8.3), and remains inferred in multimodal loops. Introduced for language models, the gap is the improvement a model obtains by filtering or reweighting its own generations with its own verification [322]. We extend the term by separating a task-level component, the difference in difficulty between verifying and generating on a task family, from a model-level component, the difference between the verification and generation accuracy of the agent's own model on the same goals. The model-level component remains unestimated, multimodal audits reporting only the verification side [64, 256, 281], and backbone priors would confound it even once measured (Sec. 3.5). We infer the task-level component to be large in GUI and Web navigation, where a final screen can be inspected without replaying the steps that reach it, and small in open-prompt generation, where judging whether an image satisfies a prompt calls on the compositional understanding that producing it requires, a difference that makes the cost of self-judgment vary across scenarios even where its verdict does not.

**Evaluator Fidelity.** The feedback ablations order evaluators by fidelity and thereby price the conditional routes that substitute a judge for a missing check. Two ablations, both in generation, compare a verifier built on the agent's own judgment with the evaluator it replaces, a frozen self-rater against a program check in one and a co-trained reward model against a learned preference scorer in the other. In

RSI in UMMs, training on the better of two images as rated by the frozen starting model gains 0.5 points, against 8.6 points with execution-verified charts for the same requests and steps [191]. The reward model that The Image as Its Own Reward [245] co-trains against reference images matches the PickScore of Flow-GRPO [195] trained on PickScore and wins 70.0% of the authors' human image-quality comparisons against it, a model that the same raters place below its base. A verifier built on the agent's own judgment thus falls below a program check and can exceed a learned preference scorer, an ordering whose two ends come from different verifiers, one frozen and one co-trained, and which the other scenarios leave untested. The ordering would be contradicted if such a verifier outperformed a keyed or programmatic check at a matched budget on fixed goals.

**Goal-Mechanism Gains.** Goal mechanisms gain in all four scenarios with one reported exception, consistent with a scarcity of $\mathbb{E}[I_{\mathcal{A}_k}]$ that gives this factor no part in the asymmetry. Removing allocation hurts both scenarios that ablate it, MobileRL [17] losing 6.3 points of AndroidWorld success without its failure curriculum filter and Policy-Level RSI [296] raising its ManiSkill failure rate from 2.34% to 3.36% under random scene selection. Created goals lead untrained, random, or frozen-source generators in VisPlay [5], RATs [302], ACuRL [15], and RSIgen [260], and trail the NNet-Nav curriculum by 1.3 points in one SEAgent setting [16]. Within a program-checkable family the error-driven chart distribution of RSI in UMMs leads a fixed distribution with the same number of verified charts by 2.8 to 4.1 points in every round [191]. These comparisons come from the authors of each method and may overstate the gain, yet their agreement across the four scenarios leaves the verdicts on goal mechanisms to turn on the resources they presuppose.

**Writer Dependence.** The gain of reuse depends on the writer, a model-level dependence that travels with the agent to every target and adds no asymmetry of its own. Authored writes outperform skill-free or raw alternatives in two navigation ablations, EvoSkill-GUI [281] gaining 6.7 points on MobileWorld over pass@3 without skills and Agent S [278] 7.7 points on an OSWorld subset over full trajectories in place of its self-evaluator's summaries. SEAR [20], holding the scenario fixed and varying only the writer, finds the sign of this gain reversed, GPT-5.6-sol on RoboLab dropping from 46.9% without a library to 42.4% with its own, beyond the spread of three runs, and rising to 60.6% with one that Claude Opus 5 wrote, a loss the authors trace to object-location recipes with few grasp checks. Whether such content tracks the generation-verification gap of the writer's model is untested, and a library that gained the same whoever wrote it would contradict the dependence on the writer.

## 8.5 Directed Portability across Scenario Pairs

The two directions of each scenario pair receive verdicts of different kinds, a difference traced to which factors are scarce and which task-structural resources and conditions hold. The direction from $X$ to $Y$ collects the mechanisms portable or conditional in $X$ that on $Y$ are redundant or rank lower in the order portable, conditional, blocked, and reversing the pair exchanges both the mechanisms carried and the conditions they meet. A mechanism whose verdict does not fall belongs to neither direction by construction, and the asymmetry accordingly concerns the kind of obstacle a direction meets, not how many mechanisms transfer. Table 12 assigns per-target verdicts by the rules below.

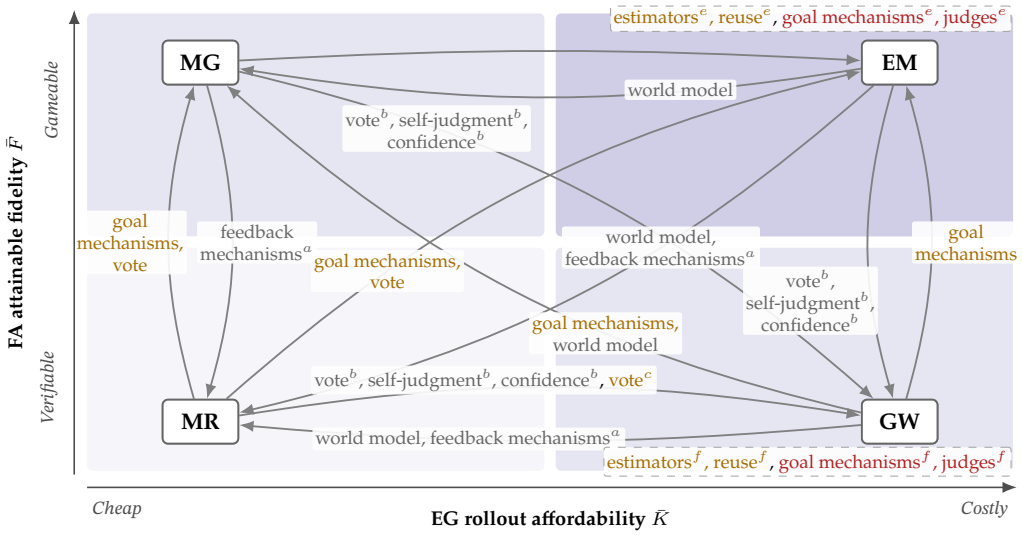
TABLE 12: **Portability of self-evolving mechanisms (list in Table 11) across target scenarios.** ● portable, ◐ conditional, ✗ blocked, and ○ redundant, with ◐ also marking a verdict capped by an unobservable outcome; * reported instance. A superscript limits an entry to the sub-family noted below and an unmarked entry covers the rest of its cell, EM entries referring to physical hardware unless marked $^{d}$. Verdicts are untested preconditions, not transfer gains.

| Mechanism | MR | MG | GW | EM |
|---|---|---|---|---|
| *Presupposes a task-structural resource* | | | | |
| Success-conditioned allocation | ●* | ◐* | ●* ✗$^{f}$ | ◐* ✗$^{e}$ ●$^{*d}$ |
| Agent-posed goals | ●* | ◐* | ●* ✗$^{f}$ | ◐✗$^{e}$ ●$^{*d}$ |
| Record-conditioned goal generation | ● | ◐ | ●* ✗$^{f}$ | ◐✗$^{e}$ |
| Program-checkable goal families | ●* | ◐* | ● | ◐●$^{*d}$ |
| Imagined rollouts in a world model | ○ | ○ | ◐ | ◐* |
| Self-consistency vote | ○$^{a}$ ●* | ◐* | ○$^{b}$ ◐$^{c}$ ✗$^{f}$ | ◐✗$^{e}$ |
| *Presupposes only the agent's model or a pre-loop artifact* | | | | |
| Self-judgment by the agent's model | ○$^{a}$ ●* | ●* | ○$^{b}$ ●$^{*c}$ ✗$^{f}$ | ●* ✗$^{e}$ |
| Generator-confidence reward | ○$^{a}$ ● | ●* | ○$^{b}$ ● | ● |
| Verifier refit on agent trajectories | ○$^{a}$ ● | ●* | ●* ◐$^{f}$ | ●◐$^{e}$ ●$^{*d}$ |
| Frozen self-derived estimator | ○$^{a}$ ●* | ●* | ●◐$^{f}$ | ●* ◐$^{e}$ |
| Episode reuse via memory or skills | ●* | ●* | ●* ◐$^{f}$ | ●* ◐$^{e}$ |

$^{a}$ key or program-computed answer, $^{b}$ outcome checked online, $^{c}$ screen-visible outcome without an online check, $^{d}$ simulation, $^{e}$ outcome neither in the agent's observations nor checked by instrumentation, with contact-rich assembly as a candidate, $^{f}$ background-state outcome that no reachable screen shows and no script checks.

**Verdict Rules.** The rules apply in a fixed order, observability first, a mechanism that judges or checks an unobservable outcome being blocked whatever its other conditions, then scarcity, which decides redundancy, and resources last, which decide between portable and conditional. A mechanism is *redundant* where the factor it raises is not scarce, judged at the ceiling or, where the loop gives up a resource, at the realized value, as for self-labeled questions without keys and task streams without online checks. $\mathbb{E}[I_{\mathcal{A}_k}]$ and $G_{\zeta}$, neither bounded by a ceiling, count as scarce everywhere (Sec. 8.1), and $L_{\nu}$ as not scarce where a single action fixes $L_{\nu}=1$. Otherwise a mechanism is *portable* where the target holds the resource it presupposes and *conditional* where an adaptation, such as converting outputs into answers or adding an external judge, or a sub-family supplies it, a missing resource admitting such a route in the surveyed targets wherever the outcome is observable. Capabilities of the agent's model and artifacts authored before the loop count as held everywhere (Sec. 8.2). Where the outcome is unobservable, observability caps the verdict and every mechanism that judges or checks it is *blocked*, even those presupposing only the agent's model, whereas low evaluator accuracy only raises the cost of a conditional route. A portable verdict states that a precondition holds, not that a transfer improves the target loop.

**Redundant Verdicts.** Redundant verdicts fall on world models where rollouts are cheap and on feedback mechanisms where an external reference already supplies every factor they raise. Reasoning and generation keep $\bar{K}$ high through a single decoding or sampling pass, leaving world models




*Factor not scarce:* redundant *Resource only on a sub-family or after adaptation:* conditional *Outcome unobservable:* blocked (conditional for estimators and reuse)

Fig. 4: **Directed portability on the positioning matrix.** Each arrow lists the mechanisms portable or conditional in its source but redundant or lower on its target, and dashed boxes list the verdicts on GW and EM goals with unobservable outcomes. Goal mechanisms: allocation, agent-posed and record-conditioned goals, and program-checkable families, the last inapplicable to unobservable outcomes. Judges: vote and self-judgment. Estimators: refit verifier and frozen estimator. Feedback mechanisms: judges, estimators, and generator-confidence reward. Superscripts follow Table 12.

redundant there. The vote, self-judgment, and generator-confidence rewards are redundant on reasoning questions with keys or program-computed answers and on navigation goals whose outcome a check inspects online. Self-judging navigation loops such as Agent S [278] and Mobile-Agent-E [162], by contrast, run on task streams or practice goals whose screen-visible outcomes no check inspects, leaving $F_\nu$ scarce at the realized value and self-judgment portable. Refit verifiers and frozen estimators are redundant only on these reasoning questions, where one action fixes $L_\nu = 1$, multi-step navigation and manipulation keeping $L_\nu$ scarce.

**Conditional Verdicts.** Conditional verdicts arise where the target holds a presupposed task-structural resource only on a sub-family or through an adaptation. Allocation and program-checkable families are conditional where checks cover only a sub-family, as in generation [11, 191] and on hardware, where an external judge grades a task library [21] or a scripted scene check covers a fixed family [323]. Created goals, checkable in reasoning and navigation, find a check in generation only through image questions, rendered charts, or external judges [40, 191, 260]. On hardware, agent-posed goals stay in simulation [20, 302], and the external judge of before-and-after images under which SEAR practices given tasks offers a route where success shows as object motion, a case FailBench finds nearly solved [324]. World models find learnable dynamics on hardware only over short horizons [24] and in navigation only over text-rendered web states, as in DynaWeb [325]. Outside the discrete answers of reasoning, the vote finds output equivalence only through questions that turn outputs into comparable answers, as ASG poses about generated images [40], a conversion untested on final screens and camera frames.

**Blocked Verdicts.** Blocked verdicts arise where the outcome of a goal appears neither in the agent's observations nor in any check available to the loop, as for background-state goals in navigation that no reachable screen shows and no script inspects. Contact-rich assembly is the candidate sub-family on hardware, where no judge in FailBench exceeds 0.60 balanced accuracy against 0.77 for the best judge overall, a result consistent with frames that omit the contact on which success depends, although the authors also name task-specific knowledge and precision [324]. On such goals the vote and self-judgment have nothing to judge, recorded actions showing what the agent did and not whether the state changed, created goals have nothing to check them, and success-conditioned allocation has no success record to read, a block reaching model-only mechanisms as well as resource-dependent ones. Refit verifiers and frozen estimators keep the progress that visible steps show, and episode reuse the writes that need no judgment of success, both becoming conditional there, whereas generator-confidence rewards, which never judge the outcome, stay portable at their cost in fidelity, a verdict certifying only that their precondition holds while their fidelity on outcomes that nothing observes stays unknown. No reported instance falls in a blocked cell, leaving the rules consistent yet unsupported.

**Asymmetry by Target.** The kind of verdict that a direction meets is fixed chiefly by its target, and the two directions of a pair differ wherever their targets differ in scarcity, resources, or observability. Checkable outcomes act on both sides of the pair of navigation and generation, the judges of generation becoming redundant on navigation goals that a check inspects online and the created goals of navigation becoming conditional on the open prompts of generation. Reasoning and manipulation show the pattern most sharply, the vote and the goal mechanisms carried from reasoning becoming conditional on hardware and, program-checkable families aside, blocked where contact decides success, while the world models and estimators of manipulation become redundant on reasoning questions with keys or program-

computed answers (Table 12). More broadly, directions into reasoning meet only redundancy, directions into generation and hardware chiefly conditional goal mechanisms and votes, and where the outcome is unobservable, navigation and hardware block goal mechanisms and judges from every source, a verdict that depends on the target alone (Figure 4).

**Structural Source.** Taken together, we interpret the structural source of the asymmetry as two scenario properties that the directions of a pair meet in different combinations, the scarcity that $\bar{K}$ and $\bar{F}$ fix, read at the realized value where a loop gives up a resource, and the unequal task structure, comprising its resources and the observability of its outcomes. Scarcity decides redundancy, task-structural resources decide between portable and conditional verdicts, and observability alone decides blocking. Observability is also the one task-structural condition that bounds mechanisms presupposing only the agent's model. Matrix positions anticipate most verdicts, failing where output equivalence or observability decides them, as for the vote and background-state goals in navigation. Still untested, the verdicts would fall to a gain over an equal-budget control for an unadapted mechanism lacking its resource, one raising a non-scarce factor, or one judging unobservable outcomes.

# 9 Self-Evolving Multimodal Agent Evaluation

This section reviews how self-evolving multimodal agents are evaluated on task-solving capability, on self-evolving capability, and through the judges and reward models that supply their feedback, and identifies the measurements whose absence leaves our account untestable.

## 9.1 Task-Solving Capability

Task-solving capability, the expected return $\mathcal{J}_k$ (Eq. (6)) that an agent attains on the goals of a scenario with its components fixed, is measured on task-specific benchmarks that fix a goal set, any environment that rollouts need, and usually a scorer (Table 13). A reported score estimates $\mathcal{J}_k$ only insofar as the goal set represents $\mu_{\mathcal{G}}$ and the scorer agrees with $R$, and audits find both conditions broken. The goal set fails through memorization, Sphinx-X-MoE reaching 43.6% on MMMU [327] without images, 17.9 points above its language-model backbone [359] and vision-language-action models above 90% on LIBERO [348] falling to 0.0% under the perturbations of LIBERO-PRO [349]. The scorer fails through divergence and exploits, GenEval [334] deviating from human judgment by up to 17.7 points [360] and BenchJack [361] reaching near-perfect scores on most of ten agent benchmarks without solving a task. A loop that trains with the scorer it is evaluated by, as Reflection-VLA [304] does on LIBERO, can write memorized goals and scorer errors into the agent, a self-evolving agent thus being more exposed to these failures than the static models and agents that the audits examine.

## 9.2 Self-Evolving Capability

Self-evolving capability, the ability to raise $\mathcal{J}_k$ over the cycles of a loop, is measured by self-evolution benchmarks that fix the goals, the environment, any feedback, and the schedule in which the agent attempts and is tested on goals (Table 14), whereas loops on task-specific benchmarks report $\mathcal{J}_k$ by round or only at the endpoints. Text-only benchmarks apply the same protocol to memory, lifelong-learning, and harness-evolution streams and to professional workflows [362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378], whereas benchmarks in which the agent remembers supplied histories [379, 380, 381], learns from supplied demonstrations or labeled streams [382, 383], or is scored without updating itself [384] fall outside this measurement, leaving no update that follows from the agent's own outcomes. Independent scores across rounds trace the increment $\Delta\mathcal{J} = \mathcal{J}_{k+1} - \mathcal{J}_k$, cumulative scores mix it with resampling, endpoint comparisons give only the total gain, and re-tests of earlier goals and held-out goals bear on retention $S_\zeta$ and generalization $G_\zeta$. A gain counts as evidence of the capability only to the extent that controls separate the agent's experience from added computation, repeated attempts, writing itself, and run variance, as in MemGUI-Bench [385], where retries alone raise M3A by 14.9 points while removing the long-term memory of Agent-S2 at the same three attempts costs it 20.0 points. Whether the check reaches the agent also shapes the gain, conditions that withhold it lowering gains or leaving them mixed [386, 387], with late-round successes in PhysRSI-eval falling from 15 to 5 of 36 without the grader [388], whereas the protocols marked † return the check, a gain under them possibly including its exploitation, as exposing a simulator's success criteria to embodied agents yields gains that vanish once the criteria change slightly [389]. These benchmarks measure mechanisms only on goals whose outcomes are checked, away from the goals where Table 12 sets verdicts by missing checks or unobservable outcomes, and cover navigation and manipulation, on hardware only in SEAR's five tasks [20], with reasoning and generation covered only by Flex-RSI [390], which omits navigation.

## 9.3 Judges and Reward Models

Where no answer key or program checks an output, a self-evolving agent realizes $V_\nu$ through a judge or a reward model, whose preferences $f_\zeta$ writes into $\mathcal{A}_{k+1}$ while the loop observes no $R$, moving the policy toward outputs that the scorer overrates. A scorer that shares the updated parameters or is refit moves with the policy as well. Evaluations compare these scorers with a reference outside the loop (Table 15) and bear on the realized fidelity $F_\nu^{(k)}$ only insofar as their outputs and scorer resemble those in the loop. On fixed outputs, agreement stays moderate and falls off the training distribution, the strongest judges reaching 62.9%–72% accuracy [285, 391, 392, 393], reward models dropping to near chance on pairs outside their training data [135, 394], and a log that reports success misleading judges on failed clips [395]. Optimization exposes further errors, reward hacking reaching 48.1% for VQA policies [396] and a judge passing repairs that humans mostly reject [395], although a fixed screen judge stays accurate on the policy trained against it [15] and fixed-output accuracy largely orders scorers by the downstream results of policies optimized against them [142, 397]. References that check outcomes reach open-ended outputs only through single-property interventions [398],

TABLE 13: **Task-specific benchmarks used with self-evolving multimodal agents.** Pool, Env, and Checker mark benchmarks supplying a method's goals, hosting its own goals, or driving its updates. ◇ discrete answers that admit voting without a key.

| Benchmark | Goals | Reset | Scorer | Pool | Env | Checker |
|---|---|---|---|---|---|---|
| ❖ *Multimodal reasoning* | | | | | | |
| MathVista [240] | 6,141 math questions | none needed | answer key◇ | ✓ | – | ✓ |
| MathVerse [241] | 2,612 math problems, 6 versions | none needed | answer key◇ | ✓ | – | – |
| MathVision [326] | 3,040 math problems, 16 disciplines | none needed | answer key◇ | ✓ | – | – |
| MMMU [327] | 11.5K college questions, 30 subjects | none needed | answer key◇ | ✓ | – | ✓ |
| ChartQA [242] | 32.7K chart questions | none needed | answer key◇ | ✓ | – | ✓ |
| Geometry3K [328] | 3,002 geometry problems | none needed | answer key◇ | ✓ | – | ✓ |
| PhysBench [329] | 10,002 physics image and video entries | none needed | answer key◇ | ✓ | – | ✓ |
| VSI-Bench [330] | 5,000+ spatial video questions | none needed | answer key◇ | – | – | – |
| HallusionBench [331] | 1,129 illusion questions | none needed | answer key | – | – | – |
| MMSI-Bench [332] | 1,000 multi-image spatial questions | none needed | answer key◇ | ✓ | – | ✓ |
| MMVU [333] | 3,000 expert video questions, 27 subjects | none needed | answer key | – | – | – |
| ❖ *Multimodal generation* | | | | | | |
| GenEval [334] | 553 object-composition prompts | none needed | object detector | ✓ | – | – |
| T2I-CompBench++ [335] | 8,000 compositional prompts | none needed | VQA, detector, MLLM | ✓ | – | – |
| DPG-Bench [336] | 1K dense prompts | none needed | VQA model | ✓ | – | – |
| WISE [337] | 1,000 knowledge prompts | none needed | VLM judge | ✓ | – | – |
| Text rendering [195] | 1,018 test prompts | none needed | OCR accuracy | ✓ | – | ✓ |
| VBench [338] | video prompts, 16 dimensions | none needed | per-dimension metrics | – | – | ✓ |
| Diagram and design sets | per-paper charts, diagrams, edits | none needed | per-paper judge or program | ✓ | – | – |
| ❖ *GUI and web navigation* | | | | | | |
| OSWorld [339] | 369 desktop tasks | virtual machine | per-task script | ✓ | ✓ | ✓ |
| ScienceBoard [340] | 169 scientific-software tasks | OS environment | per-task check | – | – | – |
| AndroidWorld [341] | 116 task templates, 20 apps | emulator | per-task script | ✓ | – | ✓ |
| AndroidLab [342] | 138 tasks, 9 apps | emulator | per-task check | ✓ | ✓ | – |
| AitW [343] | 30K instructions | none (offline) | none shipped | ✓ | – | – |
| AndroidControl [344] | 14,548 tasks, 833 apps | none (offline) | demonstrated actions | – | – | – |
| WebVoyager [345] | 643 tasks, 15 live sites | none (live) | VLM judge | ✓ | ✓ | – |
| WebArena [346] | 812 tasks, 5 sites | self-hosted sites | per-task check | – | ✓ | – |
| VisualWebArena [347] | visual tasks, 3 sites | self-hosted sites | per-task check | – | ✓ | – |
| ❖ *Embodied manipulation* | | | | | | |
| LIBERO(-PRO) [348, 349] | 130 tasks, perturbed variants | simulator | BDDL predicate | ✓ | ✓ | ✓ |
| ManiSkill3 [350] | GPU-parallel tasks | GPU simulator | success predicate | ✓ | – | ✓ |
| RoboCasa [351] | 100 kitchen tasks | simulator | success predicate | ✓ | – | ✓ |
| RoboTwin 2.0 [352] | 50 dual-arm tasks | simulator | success check | ✓ | – | ✓ |
| RoboLab [353] | 120 tasks | simulator | success predicate | ✓ | – | ✓ |
| RoboDojo [354] | 42 simulated, 18 real tasks | simulator, real reset | success check | ✓ | – | ✓ |
| VLABench [355] | 100 language-conditioned categories | simulator | binary success | ✓ | – | ✓ |
| BEHAVIOR-1K [356] | 1,000 activities, 50 scenes | simulator | BDDL predicate | ✓ | – | ✓ |
| DISCOVERSE [357] | long-horizon tasks | simulator | task reward | ✓ | – | ✓ |
| VIMA-Bench [358] | generated multimodal prompts | simulator | success predicate | ✓ | – | ✓ |
| Hardware task libraries | per-paper tasks | manual or self-reset | none shipped | ✓ | – | – |

leaving such outputs scored against human labels, larger models, or other reward models [396, 399, 400].

## 9.4 Measurement Gaps

We list a measurement as missing only where its absence leaves one of three claims untestable, that a method's gain comes from self-evolution, that a scenario sits where we place it on the positioning matrix, and that the portability verdicts hold, the last two specific to our account. These measurements share the need for a reference the loop never receives, a reference returned to the agent entering its feedback $V_\nu$ and becoming exploitable by its updates.

**Attribution of Gains.** Attributing a gain to self-evolution requires, in every scenario, the increment across rounds against a non-evolving baseline at a matched budget and, where the scorer that checks updates also produces the reported score, a reference withheld from the loop. Four self-evolution benchmarks run this baseline (Table 14), Flex-RSI among them with harness edits raising the test score in six of seven paired runs [390], as does RATs [302], whereas the other comparisons reported in reasoning and generation, such as V-Zero against supervised GRPO [7], state no matched budget. On hardware the budget also counts the human resets that $\mathcal{C}_1$ admits, which Find Something You Can't Do records as scene recoveries [21], lest an evolving arm draw on more of them than its baseline. The withheld reference, such as the test split that Flex-RSI scores once after each run [390], separates the gain from exploitation of the scorer, as for the protocols marked †, and, where training and reported goals coincide, from memorization. Agreement on fixed outputs ranks scorers without bounding that exploitation, and the audits in Table 15 cover parameter updates and single-output repair, leaving exploitation through memory and tool writes untested.

**Scenario Placement.** Placing a scenario depends only on whether each ceiling is low or high, and a misplacement would merely move verdicts into or out of the redundant class. Rollout cost in a common unit would test the split on $\bar{K}$, yet the reported units do not convert, ranging from tokens in MemGUI-Bench to hours of robot time in Self-Improving EFMs [19]. The split on $\bar{F}$ remains inferred, the evaluations of Table 15 defining each reference within one scenario and values taken from fixed outputs erring in either direction. A higher $\bar{F}$ than we place in generation or on hardware would make the feedback mechanisms there redundant, and for generation the references on optimized outputs reduce to human labels in a simulated repair loop [395] and, under retraining, other reward models [408].

**Verdict Testing.** Testing a verdict of Table 12 means running the matched-budget comparison in the target scenario with

TABLE 14: **Self-evolution benchmarks.** Notation: * external experience supplied; (req.) no non-pixel substitute; † reported scorer; P parameter, M memory, T tool, code, or skill, and I instruction or harness write, *any* open choice, parentheses the reference agents' writes; ✓ per-round scores, re-tests, or a matched baseline under $\Delta\mathcal{J}$, $S_\zeta$, and Budget, ◐ cumulative or endpoint scores or partial matching; $G_\zeta$ farthest held-out goals.

| Benchmark | Tasks | Observation | Feedback | Writes | Schedule | $\Delta\mathcal{J}$ | $S_\zeta$ | $G_\zeta$ | Cost | Budget |
|---|---|---|---|---|---|---|---|---|---|---|
| ❖ *GUI and web navigation* | | | | | | | | | | |
| MemGUI-Bench [385] | 128 tasks, 26 Android apps | screenshots or UI tree | LLM judge B† | any (M, T) | retry, 3 attempts | ◐ | – | – | tokens, time, API cost | – |
| MMG2Skill-Bench* [386] | 130 tasks, desktop and games | screenshots (req.), text | self-diagnosis A | T | retry, 5 attempts | ✓ | – | – | model calls, tokens | ✓ |
| ❖ *Embodied manipulation* | | | | | | | | | | |
| PhysRSI-eval [388] | 18 tasks, RoboDojo | images, state | RoboDojo score B† | M, T | retry, ≤17 rounds | ◐ | – | layouts | physical steps | ✓ |
| RobotAgentBench [401] | 116 tasks, Isaac Sim | images, state, map | execution feedback | T | iterations, 10 rounds | ✓ | – | – | – | – |
| SEAR [20] | 330 simulated, 5 hardware tasks | images or vision tools | sealed judge B† | P, M, T | task stream | ◐ | – | new simulator | tokens, env steps, GPU time | ✓ |
| ❖ *Across reasoning, generation, and manipulation* | | | | | | | | | | |
| Flex-RSI [390] | 24 tasks: robot simulation, video and spatial QA, image and video generation | images, video, simulator state | development labels B, withheld validation gate | I, T | rounds under a fixed budget | ◐ | – | hidden test split | tokens | ✓ |
| ❖ *Outside the four scenarios* | | | | | | | | | | |
| PokeGym [402] | 114 tasks, Pokémon Z-A | frames (req.) | composite score B+E† | any (T, I) | retry, 10 rounds | ✓ | – | – | tokens, API cost | – |
| GameVerse* [403] | 15 games | video, frames (req.) | failure video E | I | retry, 2 attempts | ◐ | – | – | – | – |
| OmniGameArena [404] | 2 of 12 UE5 games | frames (req.) | progress score B† | M, I | iterations, 10 rounds | ✓ | – | variants | – | – |
| SelfImprove-Bench [405] | 6 games | frames (req.) in 2, text in 4 | score, success E† | any (M, T, I) | task stream, replays | ◐ | ✓ | other games | – | – |
| EmbodiedCLBench [406] | 268 tasks, offline state graphs | images, actions as text | unstated | M, T | source to target tasks | – | – | compositions | – | – |
| GameBoyWorlds [387] | 500 tasks, 10 games | frames (req.) | intrinsic or self-judged A | any (P, M, I) | source to target games | – | – | new games | – | – |
| MC-EvoBench [407] | 80 tasks, Minecraft | frames, planner state | stage verifiers B† | M | source to reset target | – | – | related tasks | model calls, env steps | ◐ |

the mechanism moved unadapted, on goals outside the self-evolution benchmarks wherever the verdict turns on missing checks or unobservable outcomes. A conditional verdict also needs the adapted arm, and a blocked verdict an outcome check held outside the loop, one available to the loop making the outcome observable. Where a loop gives up a resource, the verdict rests on realized $F_\nu$, whose test needs the policy optimized against a scorer that changes with it, a joint condition that Table 15 checks against a hidden key only in text [409]. The reported transfers fall short of such a test, ASG [40] bringing voting to generation without an unadapted arm, failure-conditioned curricula moving from the text-only WebRL [410] to screen-based loops [15, 16] only within navigation, and SEAR starting its hardware tasks from an empty library instead of the one written in simulation. The verdicts thus remain untested under a shared protocol.

# 10 Open Challenges

This section discusses four open problems: reference construction, self-derived verification, and cross-substrate transfer, which inform our account of the cross-scenario asymmetry, along with safety under autonomy, which bears on the operational definition of self-evolution.

## 10.1 Reference Construction

Reference construction, the building of a substitute reference for an outcome absent from both the agent's observations and every check available to the loop, decides whether the blocked verdicts can be lifted, as for background-state goals in navigation and possibly contact-rich assembly on hardware (Sec. 8.5), and the four reported routes each succeed only in part. Converting outputs into answers [40] needs an equivalence relation that screens and action trajectories lack, and a program written together with the goal covers only goal families with a closed-form success condition, EvoCUA resolving by hand the synthesized tasks on which its two checks disagree [416]. A learned world model adds its own prediction error, its dynamics learned on hardware only over short horizons [24] and in navigation only over text-rendered states [325], while a vision-language judge reads camera frames that do not record contact forces [324]. Whether a loop can construct such a reference without a human inside the loop thus remains open, an affirmative answer turning the blocked verdicts into conditional ones that depend on how the reference is built and a negative one making the block a property of the outcome itself.

## 10.2 Self-Derived Verification

Self-derived verification, the scoring of outcomes or steps by a verifier derived from the agent's own model or trajectories, is adopted wherever no check scores an outcome or a check returns one verdict per long episode, and its reliability turns on how far the verifier depends on the policy. Verifiers that change with the policy drift or inherit its errors, confidence rewards collapsing in ablated configurations of SOLACE [9] and IRIS [129] and step reward models trained on the agent's trajectories inheriting its mistakes [135, 144]. Lagged, co-

TABLE 15: **Evaluations of multimodal judges and reward models.** Fixed, Optimized, and Retrained mark scoring of fixed outputs, of outputs trained or selected against the scorer, and across retraining of the scorer. Notation: ‡ outcome-checking reference; ◐ downstream results only.

| Evaluation | Scorer | Judged unit | Reference | Fixed | Optimized | Retrained |
|---|---|---|---|---|---|---|
| ❖ *Multimodal reasoning* | | | | | | |
| MLLM-as-a-Judge [411] | MLLM judges | vision-language responses | human scores and rankings | ✓ | – | – |
| VL-RewardBench [391] | VLM judges, reward models | vision-language responses | human-verified pairs | ✓ | ◐ | – |
| Multimodal RewardBench [392] | VLM judges | vision-language responses | expert-labeled triplets | ✓ | – | – |
| VisualProcessBench [412] | PRMs, MLLMs | reasoning steps | human step-level labels | ✓ | ◐ | – |
| VURB [393] | VLM judges, reward models | long-trace video answers | answer correctness‡ | ✓ | – | – |
| Multimodal reward hacking [396] | rule and judge rewards | VQA answers | 235B VLM oracle | – | ✓ | – |
| Self-improving VLM judge [413] | self-trained VLM judge | benchmark responses | benchmark preference labels | ✓ | – | ✓ |
| ❖ *Multimodal generation* | | | | | | |
| MJ-Bench [397] | CLIP-based scorers, VLMs | text-to-image outputs | human pairs and rankings | ✓ | ◐ | – |
| IsoVRBench [398] | video reward models | generated videos | single-property interventions‡ | ✓ | ◐ | – |
| TDPO [399] | reward models | text-to-image outputs | held-out reward models | – | ✓ | – |
| T2I reward hacking [400] | preference, detector, MLLM | text-to-image outputs | artifact labels, held-out metrics | ✓ | ✓ | – |
| Don't Read the Log [395] | MLLM judges | generated video clips | human labels on contact sheets | ✓ | ✓ | – |
| Self-Verifier Capture [408] | the policy's model | generated images | reward-model panel | ✓ | ✓ | ✓ |
| ❖ *GUI and web navigation* | | | | | | |
| AgentRewardBench [285] | LLM and VLM judges | web trajectories | expert success annotations | ✓ | – | – |
| Mobile judge benchmark [142] | VLM judges | mobile trajectories | human labels, task checker‡ | ✓ | ◐ | – |
| AndroidFlux [394] | reward models, likelihoods | recovery actions | agent-panel rollout success‡ | ✓ | – | – |
| ACuRL judge audit [15] | VLM judge | computer-use trajectories | rule checker‡, human labels | ✓ | ✓ | – |
| ❖ *Embodied manipulation* | | | | | | |
| FailBench [324] | VLM judges | robot attempts | source success labels | ✓ | – | – |
| SOLE-R1 [414] | progress reward models | per-step progress | task success, frame order‡ | ✓ | ✓ | – |
| ❖ *Text only, for comparison* | | | | | | |
| Self-play judge hacking [409] | the policy's model | math answers | hidden exact-match key‡ | – | ✓ | ✓ |
| Judge-hacking pace [415] | frozen LLM judges | math answers | answer key‡ | ✓ | ✓ | – |
| Self-Rewarding LMs [107] | the policy's model | chat responses | human rankings | ✓ | ◐ | ✓ |

trained, and frozen verifiers, such as the moving-average teacher of UniEvo-VL [263], the initial model in PRISM [247], and progress heads frozen before reinforcement learning [19, 145], depend on the policy less or not at all, trading a moving bias for one fixed by their source, a trade that no report compares across degrees of dependence. Whether any degree of dependence keeps a verifier consistent with $R$ once the policy leaves the support on which the verifier was formed remains open, an affirmative answer making reliability a choice of dependence and supplying step-level credit to long episodes, and a negative one turning the low $\bar{F}$ of generation and hardware and the scarcity of $L_\nu$ in navigation and manipulation into structural limits.

### 10.3 Cross-Substrate Transfer

Cross-substrate transfer, the carrying of what a loop writes, whether a skill library, a harness, or a policy, to another reader, platform, or embodiment once the missing resources are supplied, underlies the conditional verdicts for hardware targets, and the reported evidence leaves it unsettled. Across platforms, zero-shot transfer is reported only for the fixed loop GUI-HARVEST, whose harness evolved on OSWorld raises GPT-5 by 13.87 points on WindowsAgentArena [286]. Across simulators and action interfaces, the code policies of SEAR lose little while its VLA policies lose most of their success, the former falling from 33.3% to 28.7% and the latter from 43.1% to 10.2% [20]. Across readers, the same WebCoach memory lowers a 7B agent and raises larger ones [163], in line with the writer dependence of Sec. 8.4. No report carries a write across the simulation-to-hardware boundary, SEAR starting its hardware tasks from an empty library (Sec. 9.4). Whether a write takes effect across readers, platforms, and embodiments, the simulation-to-hardware boundary included, without new experience accordingly remains open, an affirmative answer loosening the conditional verdicts for hardware targets and a negative one making the substrate itself a missing resource there.

### 10.4 Safety under Autonomy

Autonomy ($\mathcal{C}_1$) excludes human veto from the loop, leaving the loop itself to preserve safety across modifications, as the first of the three laws proposed for self-evolving agents requires [28], and the reported evidence shows safety eroding along each path of self-modification examined. Degradation is reported along the model, memory, tool, and workflow paths, on text and code except for one screen-based case, SEAgent, whose unsafe completion rate on RiOSWorld rises from 39.5% to 55.3% [417]. Experience accumulated only on benign tasks in text-rendered web and embodied environments raises attack success on later high-risk tasks [418], and safety-oriented loops such as FailBank [419] draw their proposals from a teacher fixed before the loop, leaving untouched the loops in which the agent supplies its own feedback. Whether a loop can preserve a safety property across cycles while no human input reaches its goals, feedback, updates, or actions remains open, an affirmative answer showing autonomy and safety need not trade off under $\mathcal{C}_1$ and a negative one making the trade-off a consequence of the definition, common to all scenarios.

## 11 Conclusion

**Summary.** This survey investigates the structural source of cross-scenario asymmetry in mechanism portability across four scenarios. Through the three-stage self-evolving loop, we place the four scenarios in a positioning matrix spanned by ceilings on rollout count and feedback fidelity, and use fixed loops to separate scenario-imposed limits from those

incurred by self-evolving design. We then recast each mechanism as a trade-off that raises a scarce factor by presupposing a task-structural resource, and attribute the asymmetry to the different combinations of scarce factors and resources that the two directions of a scenario pair encounter. Inferred from single-scenario evidence, the resulting portability verdicts are testable preconditions, not empirical guarantees. Sec. 9 identifies the measurements that would test them, and Sec. 10 discusses the problems that existing mechanisms leave open.

**Limitations.** Although this survey characterizes the cross-scenario asymmetry through an explicit definition, framework, and set of verdict rules, it has the following limitations. First, inclusion requires the agent to realize at least one of the goal sampler, the feedback function, or the evolution operator, a criterion that confines the scenario patterns to the self-evolving subset and admits fixed loops only as controls. The provenance assignments and the grouping axes of the scenario taxonomies were made by a single coder, the axes after the binding constraints had been identified, and the consolidation profiles partly reflect which scenarios report retention and generalization at all. Second, the quadrant placement rests on single-scenario estimates of rollout cost and feedback fidelity that do not separate the pretraining coverage of the backbone from the task structure, and the verdicts are derived from rules, not observed in transfer experiments. Third, the three-stage decomposition is one of several possible stage boundaries, and credit assignment, treated here as a refinement of feedback acquisition, could instead form a separate stage. Finally, the surveyed methods center on image, video, screen, and robot-camera inputs, leaving loops with audio, tactile, or force feedback unexamined, and later work, particularly in embodied manipulation, may revise the placements and verdicts.

## References


[1] Z. Durante, Q. Huang, N. Wake, R. Gong, J. S. Park, B. Sarkar, R. Taori, Y. Noda, D. Terzopoulos, Y. Choi *et al.*, "Agent AI: Surveying the horizons of multimodal interaction," *arXiv:2401.03568*, 2024. 1, 2

[2] J. Xie, Z. Chen, R. Zhang, X. Wan, and G. Li, "Large multimodal agents: A survey," *arXiv:2402.15116*, 2024. 1, 2

[3] H. Yao, R. Zhang, J. Huang, J. Zhang, Y. Wang, B. Fang, R. Zhu, Y. Jing, S. Liu, G. Li *et al.*, "A survey on agentic multimodal large language models," *arXiv:2510.10991*, 2025. 1, 2

[4] J. Hu, J. Liu, J. Lai, J. Hu, Y. Sheng, S. Chen, J. Li, D. Du, and S. Guo, "GUI agents with reinforcement learning: Toward digital inhabitants," *arXiv:2604.27955*, 2026. 1

[5] Y. He, C. Huang, Z. Li, J. Huang, and Y. Yang, "VisPlay: Self-evolving vision-language models," in *CVPR*, 2026. 1, 2, 4, 6, 9, 10, 11, 13, 14, 15, 16, 27, 28

[6] Z. Li, X. Wu, C. Huang, H. Du, X. Wu, L. Yu, Y. He, Z. Liu, J. Zhang, F. Liu, and J. Xie, "Self-evolving vision language models for structured visual understanding," in *EMNLP*, 2026. 1, 9, 13, 15

[7] H. Wang, Y. Yang, J. Hu, M. Zhu, and W. Chen, "V-Zero: Self-improving multimodal reasoning with zero annotation," *arXiv:2601.10094*, 2026. 1, 4, 7, 9, 11, 14, 15, 16, 27, 32

[8] J. He, J. Fang, F. Xiong, Z. Yao, F. Shen, H. Guo, J. Wang, and T.-S. Chua, "Active Zero: Self-evolving vision-language models through active environment exploration," *arXiv:2602.11241*, 2026. 1, 9, 15, 16

[9] S. Kim and M. Cho, "Improving text-to-image generation with intrinsic self-confidence rewards," in *CVPR*, 2026. 1, 9, 10, 11, 13, 17, 18, 19, 20, 27, 34

[10] J. Hong, Y. Zhang, G. Wang, Y. Liu, J.-R. Wen, and R. Yan, "SUDER: Self-improving unified large multimodal models for understanding and generation with dual self-rewards," *arXiv:2506.07963*, 2025. 1, 4, 10, 17, 18, 19, 20, 27

[11] Y. Han, H. Chen, A. Han, Z. Wang, X. Liu, Y. Zhang, S. Zhang, and D. Zou, "Turning internal gap into self-improvement: Promoting the generation-understanding unification in MLLMs," in *ICLR*, 2026. 1, 17, 18, 19, 20, 27, 29

[12] L. Qu, H. Li, W. Wang, X. Liu, J. Li, L. Nie, and T.-S. Chua, "SILMM: Self-improving large multimodal models for compositional text-to-image generation," in *CVPR*, 2025. 1, 17, 18, 19

[13] J. Jiang, C. Si, J. Luo, H. Zhang, and C. Ma, "Co-reinforcement learning for unified multimodal understanding and generation," in *NeurIPS*, 2025. 1, 17, 18, 19

[14] H. Bai, Y. Zhou, M. Cemri, J. Pan, A. Suhr, S. Levine, and A. Kumar, "DigiRL: Training in-the-wild device-control agents with autonomous reinforcement learning," in *NeurIPS*, 2024. 1, 4, 7, 10, 11, 13, 20, 21, 22, 23, 26, 27

[15] T. Xue, Z. Liao, T. Shi, Z. Wang, K. Zhang, D. Song, Y. Su, and H. Sun, "Autonomous continual learning for environment adaptation of computer-use agents," *arXiv:2602.10356*, 2026. 1, 9, 11, 20, 21, 22, 27, 28, 32, 33

[16] Z. Sun, Z. Liu, Y. Zang, Y. Cao, X. Dong, T. Wu, D. Lin, and J. Wang, "SEAgent: Self-evolving computer use agent with autonomous learning from experience," in *ICML*, 2026. 1, 7, 10, 20, 21, 22, 23, 26, 27, 28, 32

[17] Y. Xu, X. Liu, X. Liu, J. Fu, H. Zhang, B. Jing, S. Zhang, Y. Wang, W. Zhao, and Y. Dong, "MobileRL: Online agentic reinforcement learning for mobile GUI agents," in *ICLR*, 2026. 1, 4, 9, 10, 11, 20, 21, 22, 23, 27, 28

[18] Z. Liu, Z. Yang, Z. Zhang, and H. Tang, "EvoVLA: Self-evolving vision-language-action model," *arXiv:2511.16166*, 2025. 1, 9, 10, 11, 23, 24, 25

[19] S. K. S. Ghasemipour, A. Wahid, J. Tompson, P. Sanketi, and I. Mordatch, "Self-improving embodied foundation models," in *NeurIPS*, 2025. 1, 9, 10, 23, 24, 25, 26, 27, 32, 34

[20] X. Wang, B. Li, H. Wang, L. Zhang, A. Y. Cui, Z. Tang, S. Buch, and D. Xu, "SEAR: Self-evolving agentic robotics," *arXiv:2610.18792*, 2026. 1, 13, 23, 24, 25, 26, 27, 29, 32, 33, 34

[21] Y. Fang, Z. Li, H. Tong, P. Liu, and G. Chalvatzaki, "Find something you can't do: Agentic real-world reinforcement learning for self-improving VLA models," *arXiv:2609.32069*, 2026. 1, 9, 23, 24, 25, 26, 27, 29, 32

[22] O. Thawakar, S. Venkatraman, R. Thawkar, A. M. Shaker, H. Cholakkal, R. M. Anwer, S. Khan, and F. S. Khan, "EvoLMM: Self-evolving large multimodal models with continuous rewards," *arXiv:2511.16672*, 2025. 1, 2, 4, 6, 7, 9, 11, 13, 14, 15, 16, 27

[23] L. Wei, Y. Li, C. Wang, Y. Wang, L. Kong, W. Huang, and L. Sun, "First SFT, second RL, third UPT: Continual improving multimodal LLM reasoning via unsupervised post-training," in *NeurIPS*, 2025. 1, 14, 15

[24] X. Liu, Z. Bai, H. Ci, K. Y. Ma, and M. Z. Shou, "World-VLA-Loop: Closed-loop learning of video world model and VLA policy," *arXiv:2602.06508*, 2026. 1, 23, 27, 29, 33

[25] W. Li, Y. Qin, H. Zhou, H. Wang, Y. Guo, and R. Zhang, "Trust-calibrated VLA policy post-training with evolving imagination," *arXiv:2610.16517*, 2026. 1, 23, 24, 25, 26, 27, 28

[26] X. Hu, T. Xiong, B. Yi *et al.*, "OS agents: A survey on MLLM-based agents for general computing devices use," in *ACL*, 2025. 2

[27] Y. Huang, S. Tu, X. Yan, J. Zhan, Z. Yang, Z. Xing, H. Zhang, T. Zhang, Y.-G. Jiang, and Z. Wu, "Agentic visual generation: From generative models to agentic control," *arXiv:2609.06758*, 2026. 2

[28] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li *et al.*, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv:2508.07407*, 2025. 1, 2, 4, 6, 7, 34

[29] P. Jiang, J. Lin, Z. Shi *et al.*, "Adaptation of agentic AI: A survey of post-training, memory, and skills," *arXiv:2512.16301*, 2025. 1, 2

[30] M. Chen, L. Wang, and B. Qu, "Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops," *arXiv:2607.07663*, 2026. 2

[31] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Q. Ren *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026. 1, 2, 4, 5, 6, 7, 11

[32] J. Pu, A. Gao, J. Bai, Q. Wang, T. Wen, J. Zhao, K. Qu, J. Zhang, C. Shen, Z. Sun, J. Xu, and X. Zhang, "Trustworthy recursive self-improvement in the era of large models: Foundation, risks, evaluation and open challenges," *arXiv:2610.35193*, 2026. 2

[33] Z. Ren, Y. Chen, D. Guo, G. Rong, T. Li, R. B. Xiong, Q. Lan, W. Wang, N. Li, Y. Yang, M. Zhuge, and J. Schmidhuber, "Self-improvements in modern agentic systems: A survey," *arXiv:2607.13104*, 2026. 1, 2

[34] Z. Xiang, C. Yang, Z. Chen, Z. Wei, Y. Tang, Z. Teng, Z. Peng, Z. Li, C. Huang, Y. He, C. Yang, X. Wang, X. Huang, Q. Zhang, and J. Su, "A systematic survey of self-evolving agents: From model-centric to environment-driven co-evolution," *TechRxiv*, 2026. 2, 5, 6, 7

[35] Y. Xu, W. Zhang, Y. Chen, X. Lin, and Y. Zhang, "Self-evolving agents as dynamic graph transformation: A survey and new perspective," *arXiv:2608.18104*, 2026. 2

[36] G. Zhang, H. Geng, X. Yu *et al.*, "The landscape of agentic reinforcement learning for LLMs: A survey," *Transactions on Machine Learning Research*, 2026. 2

[37] S. Deng, K. Wang, T. Yang, H. Singh, and Y. Tian, "Self-improvement in multimodal large language models: A survey," in *Findings of EMNLP*, 2025. 2

[38] T. Feng, X. Wang, and W. Zhu, "Self-evolving embodied AI," *arXiv:2602.04411*, 2026. 2

[39] M. Guo, Q. Jiao, X. Liu *et al.*, "Seeing, maintaining, and learning: The evolution of multimodal agent memory," *arXiv:2610.19366*, 2026. 2

[40] R. Thawkar, S. Venkatraman, O. Thawakar, A. Shaker, F. S. Khan, H. Cholakkal, S. Khan, and R. M. Anwer, "Ask, solve, generate: Self-evolving unified multimodal understanding and generation via self-consistency rewards," *arXiv:2606.27376*, 2026. 1, 17, 18, 19, 27, 29, 32

[41] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024. 1

[42] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, "The rise and potential of large language model based agents: A survey," *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025. 1

[43] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Prentice Hall, 2009. 3

[44] S. Bai, K. Chen, X. Liu, J. Wang, W. Ge, S. Song, K. Dang, P. Wang, S. Wang, J. Tang, H. Zhong, Y. Zhu, M. Yang, Z. Li, J. Wan, P. Wang, W. Ding, Z. Fu, Y. Xu, J. Ye, X. Zhang, T. Xie, Z. Cheng, H. Zhang, Z. Yang, H. Xu, and J. Lin, "Qwen2.5-VL technical report," *arXiv:2502.13923*, 2025. 3

[45] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-VL technical report," *arXiv:2511.21631*, 2025. 3

[46] Z. Chen, J. Wu, W. Wang, W. Su, G. Chen, S. Xing, M. Zhong, Q. Zhang, X. Zhu, L. Lu *et al.*, "InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks," in *CVPR*, 2024. 3

[47] Chameleon Team, "Chameleon: Mixed-modal early-fusion foundation models," *arXiv:2405.09818*, 2024. 3

[48] J. Xie, W. Mao, Z. Bai, D. J. Zhang, W. Wang, K. Q. Lin, Y. Gu, Z. Chen, Z. Yang, and M. Z. Shou, "Show-o: One single transformer to unify multimodal understanding and generation," *arXiv:2408.12528*, 2024. 3

[49] X. Chen, Z. Wu, X. Liu, Z. Pan, W. Liu, Z. Xie, X. Yu, and C. Ruan, "Janus-Pro: Unified multimodal understanding and generation with data and model scaling," *arXiv:2501.17811*, 2025. 3

[50] X. Wang, X. Zhang, Z. Luo, Q. Sun, Y. Cui, J. Wang, F. Zhang, Y. Wang, Z. Li, Q. Yu, Y. Zhao, Y. Ao, X. Min, T. Li, B. Wu, B. Zhao, B. Zhang, L. Wang, G. Liu, Z. He, X. Yang, J. Liu, Y. Lin, T. Huang, and Z. Wang, "Emu3: Next-token prediction is all you need," *arXiv:2409.18869*, 2024. 3

[51] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *CoRL*, 2023. 3

[52] B. T. Willard and R. Louf, "Efficient guided generation for large language models," *arXiv:2307.09702*, 2023. 3

[53] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured NLP tasks without finetuning," in *EMNLP*, 2023. 3

[54] Y. Dong, C. F. Ruan, Y. Cai, R. Lai, Z. Xu, Y. Zhao, and T. Chen, "XGrammar: Flexible and efficient structured generation engine for large language models," *arXiv:2411.15100*, 2024. 3

[55] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," *arXiv:2310.08560*, 2023. 3, 11

[56] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023. 3

[57] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *NeurIPS*, 2023. 3

[58] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *ICLR*, 2024. 3

[59] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *NeurIPS*, 2024. 3

[60] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998. 3

[61] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *ICML*, 2015. 3

[62] H. Li, Y. Zuo, J. Yu, Y. Zhang *et al.*, "SimpleVLA-RL: Scaling VLA training via reinforcement learning," in *ICLR*, 2026. 4, 6, 7, 9, 23, 25, 26, 27

[63] Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan, "RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation," in *ICML*, 2024. 4, 9

[64] A. Heakl, S. Choi, M. Lee, and S. Khan, "Program-verified self-evolution for vision-language models," *arXiv:2609.33855*, 2026. 4, 10, 11, 14, 15, 16, 26, 27, 28

[65] Z. Shao, P. Wang, R. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv:2402.03300*, 2024. 4, 9, 11

[66] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *AAAI*, 2024. 4, 7

[67] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. 4, 7, 11

[68] X. Wu, Z. Li, G. Shi, A. Duffy, T. Marques, M. L. Olson, T. Zhou, and D. Manocha, "Co-evolving LLM decision and skill bank agents for long-horizon tasks," *arXiv:2604.20987*, 2026. 4

[69] R. Jin and Z. Zhang, "Agentic-VLA: Efficient online adaptation for vision-language-action models," in *ICML*, 2026. 4, 23, 24, 25

[70] Z. Chen, Z. Zhang, C. Zhang, Y. Xu, L. Yang, J. Liu, K. Li, and Y.-Z. Song, "PerfGuard: A performance-aware agent for visual content generation," in *ICLR*, 2026. 4

[71] A. Sharma, K. Xu, N. Sardana, A. Gupta, K. Hausman, S. Levine, and C. Finn, "Autonomous reinforcement learning: Formalism and benchmarking," in *ICLR*, 2022. 5

[72] Q. Xie, M.-T. Luong, E. Hovy, and Q. V. Le, "Self-training with noisy student improves ImageNet classification," in *CVPR*, 2020. 7

[73] K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, "FixMatch: Simplifying semi-supervised learning with consistency and confidence," in *NeurIPS*, 2020. 7

[74] H. Pham, Z. Dai, Q. Xie, and Q. V. Le, "Meta pseudo labels," in *CVPR*, 2021. 7

[75] Y. Sun, X. Wang, Z. Liu, J. Miller, A. A. Efros, and M. Hardt, "Test-time training with self-supervision for generalization under distribution shifts," in *ICML*, 2020. 7

[76] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell, "Tent: Fully test-time adaptation by entropy minimization," in *ICLR*, 2021. 7

[77] M. Zhang, S. Levine, and C. Finn, "MEMO: Test time robustness via adaptation and augmentation," in *NeurIPS*, 2022. 7

[78] Q. Wang, O. Fink, L. Van Gool, and D. Dai, "Continual test-time domain adaptation," in *CVPR*, 2022. 7

[79] S. Niu, J. Wu, Y. Zhang, Y. Chen, S. Zheng, P. Zhao, and M. Tan, "Efficient test-time model adaptation without forgetting," in *ICML*, 2022. 7

[80] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," in *ICLR*, 2023. 7, 9, 15

[81] C. Snell, J. Lee, K. Xu, and A. Kumar, "Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning," in *ICLR*, 2025. 7

[82] J. Jiang, C. Ma, X. Song, H. Zhang, and J. Luo, "Corvid: Improving multimodal large language models towards chain-of-thought reasoning," in *ICCV*, 2025. 7

[83] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *NeurIPS*, 2023. 7

[84] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *ICLR*, 2024. 7

[85] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *ICLR*, 2024. 7

[86] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of ACL*, 2024. 7

[87] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics*, 2011. 7

[88] Physical Intelligence, "$\pi^*_{0.6}$: a VLA that learns from experience," *arXiv:2511.14759*, 2025. 7

[89] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi, and H. Hajishirzi, "Self-Instruct: Aligning language models with self-generated instructions," in *ACL*, 2023. 7

[90] C. Gulcehre, T. L. Paine, S. Srinivasan, K. Konyushkova, L. Weerts, A. Sharma, A. Siddhant, A. Ahern, M. Wang, C. Gu, W. Macherey, A. Doucet, O. Firat, and N. de Freitas, "Reinforced self-training (ReST) for language modeling," *arXiv:2308.08998*, 2023. 7

[91] A. Singh, J. D. Co-Reyes, R. Agarwal, A. Anand, P. Patil, X. Garcia, P. J. Liu, J. Harrison, J. Lee, K. Xu *et al.*, "Beyond human data: Scaling self-training for problem-solving with language models," *Transactions on Machine Learning Research*, 2024. 7

[92] E. Zelikman, Y. Wu, J. Mu, and N. D. Goodman, "STaR: Bootstrapping reasoning with reasoning," in *NeurIPS*, 2022. 7

[93] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Natl. Acad. Sci. USA*, vol. 114, no. 13, pp. 3521–3526, 2017. 7

[94] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in *NeurIPS*, 2017. 7, 11

[95] Y. Liu, H. Li, S. Tian, Y. Qin, Y. Chen, Y. Zheng, Y. Huang, and D. Zhao, "CRL-VLA: Continual vision-language-action learning," *arXiv:2602.03445*, 2026. 7, 11, 12, 23, 25

[96] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," in *NeurIPS*, 2022. 7

[97] Y. Bai *et al.*, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *arXiv:2204.05862*, 2022. 7

[98] H. Touvron *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv:2307.09288*, 2023. 7

[99] N. Lambert *et al.*, "Tülu 3: Pushing frontiers in open language model post-training," *arXiv:2411.15124*, 2024. 7

[100] Kimi Team, "Kimi k1.5: Scaling reinforcement learning with LLMs," *arXiv:2501.12599*, 2025. 7

[101] C. Yang, S. Su, S. Liu, X. Dong, Y. Yu, W. Su, X. Wang, Z. Liu, J. Zhu, H. Li, W. Wang, Y. Qiao, X. Zhu, and J. Dai, "ZeroGUI: Automating online GUI learning at zero human cost," *arXiv:2505.23762*, 2025. 7, 9, 12, 20, 21, 22

[102] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, Z. Huang, C. Guestrin, and J. Zou, "TextGrad: Automatic "differentiation" via text," *arXiv:2406.07496*, 2024. 7

[103] J. Liu, J. Li, T. Kuang, X. Liu, K. Mei, Y. Wen, and W. Liu, "SymbOmni: Evolving agentic omni models via symbolic concept learning," *arXiv:2607.12042*, 2026. 7, 17, 18, 19

[104] H. Xiao, G. Wang, H. Wang, S. Liu, Y. Chai, Y. Pan, Y. Zhou, X. Chen, Y. Wen, and H. Li, "UI-Mem: Self-evolving experience memory for online reinforcement learning in mobile GUI agents," *arXiv:2602.05832*, 2026. 7, 11, 22

[105] A. Zhao, Y. Wu, Y. Yue, T. Wu, Q. Xu, M. Lin, S. Wang, Q. Wu, Z. Zheng, and G. Huang, "Absolute Zero: Reinforced self-play reasoning with zero data," in *NeurIPS*, 2025. 7

[106] R. Portelas, C. Colas, L. Weng, K. Hofmann, and P.-Y. Oudeyer, "Automatic curriculum learning for deep RL: A short survey," in *IJCAI*, 2020. 7

[107] W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston, "Self-rewarding language models," in *ICML*, 2024. 7, 33

[108] Y. Zuo *et al.*, "TTRL: Test-time reinforcement learning," in *NeurIPS*, 2025. 7

[109] C. Fernando, D. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel, "Promptbreeder: Self-referential self-improvement via prompt evolution," in *ICML*, 2024. 6, 7

[110] E. Zelikman, E. Lorch, L. Mackey, and A. T. Kalai, "Self-taught optimizer (STOP): Recursively self-improving code generation," *arXiv:2310.02304*, 2023. 7

[111] X. Yin, X. Wang, L. Pan, L. Lin, X. Wan, and W. Y. Wang, "Gödel agent: A self-referential agent framework for recursively self-improvement," in *ACL*, 2025. 7

[112] A. Panickssery, S. R. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in *NeurIPS*, 2024. 6

[113] M. Cheng, D. Wang, S. Yu, Q. Li, J. Ouyang, Y. Luo, Y. Zhang, Q. Liu, and E. Chen, "A comprehensive survey of the LLM-based agent: The contextual cognition perspective," *Preprints*, 2026. 7

[114] T. Liu and M. van der Schaar, "Position: Truly self-improving agents require intrinsic metacognitive learning," in *ICML*, 2025. 7

[115] P. Li, Z. Hu, Z. Shang, J. Wu, Y. Liu, H. Liu, Z. Gao, C. Shi, B. Zhang, Z. Zhang, X. Shi, Z. Yu, Y. Wu, X. Wu, Y. Jia, L. Xiang, Z. He, and Q. Li, "Efficient multi-turn RL for GUI agents via decoupled training and adaptive data curation," *arXiv:2509.23866*, 2025. 7, 9, 10, 20, 21, 22, 23, 26, 27

[116] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *NeurIPS*, 2023. 7, 11

[117] R. Wang, J. Lehman, J. Clune, and K. O. Stanley, "Paired open-ended trailblazer (POET): Endlessly generating increasingly complex and diverse learning environments and their solutions," *arXiv:1901.01753*, 2019. 7

[118] M. Jiang, E. Grefenstette, and T. Rocktäschel, "Prioritized level replay," in *ICML*, 2021. 7

[119] M. Dennis, N. Jaques, E. Vinitsky, A. M. Bayen, S. J. Russell, A. Critch, and S. Levine, "Emergent complexity and zero-shot transfer via unsupervised environment design," in *NeurIPS*, 2020. 7

[120] J. Parker-Holder, M. Jiang, M. Dennis, M. Samvelyan, J. Foerster, E. Grefenstette, and T. Rocktäschel, "Evolving curricula with regret-based environment design," in *ICML*, 2022. 7

[121] S. Sukhbaatar, Z. Lin, I. Kostrikov, G. Synnaeve, A. Szlam, and R. Fergus, "Intrinsic motivation and automatic curricula via asymmetric self-play," in *ICLR*, 2018. 7

[122] A. Campero, R. Raileanu, H. Küttler, J. B. Tenenbaum, T. Rocktäschel, and E. Grefenstette, "Learning with AMIGo: Adversarially motivated intrinsic goals," in *ICLR*, 2021. 7

[123] T. Anthony, Z. Tian, and D. Barber, "Thinking fast and slow with deep learning and tree search," in *NeurIPS*, 2017. 7

[124] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018. 7

[125] H. Wang, H. Zou, H. Song, J. Feng, J. Fang *et al.*, "UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning," *arXiv:2509.02544*, 2025. 9, 20, 27

[126] C. Xu, Y. Miao, P. Zhang, H. Dou, L. Sun, and X. Chu, "RISE: Reliable improvement in self-evolving vision-language models," *arXiv:2605.20914*, 2026. 9, 14, 15, 16, 27

[127] S. Huang, Z. Wang, Z. Zuo, H. Qiu, Q. She, and B. Wen, "EvoVid: Temporal-centric self-evolution for video large language models," *arXiv:2605.21931*, 2026. 9, 13, 14, 15, 27

[128] W. Mao, Z. Yang, and M. Z. Shou, "UniRL: Self-improving unified multimodal models via supervised and reinforcement learning," *arXiv:2505.23380*, 2025. 9, 10, 17, 18, 19, 20

[129] Y. Chen, Y. Ban, Y. Hong, and C.-J. Hsieh, "IRIS: Intrinsic reward image synthesis," *arXiv:2509.25562*, 2025. 9, 10, 11, 17, 18, 19, 20, 27, 34

[130] Y. Shi, W. Yu, Z. Li, Y. Wang, H. Zhang, N. Liu, H. Mi, and D. Yu, "MobileGUI-RL: Advancing mobile GUI agent through

reinforcement learning in online environment," *arXiv:2507.05720*, 2025. 9, 20, 22

[131] W. Liu, S. Qi, Y. Du, and Y. He, "Self-play only evolves when self-synthetic pipeline ensures learnable information gain," *arXiv:2603.02218*, 2026. 8, 9, 13, 15

[132] X. Chen, R. Aksitov, U. Alon, J. Ren, K. Xiao, P. Yin, S. Prakash, C. Sutton, X. Wang, and D. Zhou, "Universal self-consistency for large language model generation," *arXiv:2311.17311*, 2023. 9, 15

[133] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi *et al.*, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, no. 8081, pp. 633–638, 2025. 9

[134] W. Huang, B. Jia, Z. Zhai, S. Cao, Z. Ye, F. Zhao, Z. Xu, X. Tang, Y. Hu, and S. Lin, "Vision-R1: Incentivizing reasoning capability in multimodal large language models," in *ICLR*, 2026. 9

[135] H. Xiao, G. Wang, Y. Chai, Z. Lu, W. Lin, H. He, L. Fan, L. Bian, R. Hu, L. Liu, S. Ren, Y. Wen, X. Chen, A. Zhou, and H. Li, "UI-Genie: A self-improving approach for iteratively boosting MLLM-based mobile GUI agents," in *NeurIPS*, 2025. 9, 10, 20, 21, 22, 23, 26, 27, 32, 34

[136] S. Diao, P. Wang, Y. Lin, R. Pan, X. Liu, and T. Zhang, "Active prompting with chain-of-thought for large language models," in *ACL*, 2024. 9

[137] Y. Du, O. Watkins, Z. Wang, C. Colas, T. Darrell, P. Abbeel, A. Gupta, and J. Andreas, "Guiding pretraining in reinforcement learning with large language models," in *ICML*, 2023. 9

[138] L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang, "GenSim: Generating robotic simulation tasks via large language models," in *ICLR*, 2024. 9

[139] Y. Cao, D. Ran, M. Wu, Y. Guo, X. Chen, A. Li, G. Cao, Z. Gong, H. Yu, L. Li, W. Yang, and T. Xie, "GUI-Genesis: Automated synthesis of efficient environments with verifiable rewards for GUI agent post-training," *arXiv:2602.14093*, 2026. 9

[140] H. Bai, A. Taymanov, T. Zhang, A. Kumar, and S. Whitehead, "WebGym: Scaling training environments for visual web agents with realistic tasks," *arXiv:2601.02439*, 2026. 9

[141] X. Liu, S. Cui, G. Yao, Z. Cao, J. Ma, M. Zhang, and C. Zhang, "Affordance-graphed task worlds: Self-evolving task generation for scalable embodied learning," *arXiv:2602.12065*, 2026. 9

[142] Z. Wang, L. Gu, Z. Chi, Z. Liu, S. M. Ayyoubzadeh, Y. Yu, and Y. Wang, "Benchmarking LLM judges for mobile agent evaluation," *arXiv:2608.11434*, 2026. 10, 21, 32, 33

[143] N. Dern, S. Lyu, and K. Ndousse, "Vision-language judges prevent reward hacking in robot manipulation RL," in *NeurIPS Workshops*, 2026. 10, 25

[144] Z. Li, Z. Cao, W. Huang, Y. Zhang, K. Qi, R. Wang, Z. Zheng, J. Zhao, H. Zhu, H. Wu, Y. Wang, G. Fan, G. Wu, Y. Liu, Z. Gao, H. Xu, H. Yang, M. Xiang, X. Liu, and Z. Wang, "MagicGUI-RMS: A multi-agent reward model system for self-evolving GUI agents via automated feedback reflux," *arXiv:2601.13060*, 2026. 10, 20, 21, 22, 23, 27, 34

[145] C. Liu, W. Tan, L. Zhu, F. Li, J. Li, G. Yang, and H. T. Shen, "Self-correcting VLA: Online action refinement via sparse world imagination," *arXiv:2602.21633*, 2026. 10, 23, 24, 25, 27, 34

[146] T. Yang, L. Wu, R. Sun, Y. Shi, Y. Wang, M. Darbari, N. Liu, J. Sun, and L. Hong, "Self-evolving code-with-image reasoning," *arXiv:2608.11292*, 2026. 10, 14, 15, 16, 27, 28

[147] H. Le, Y. Wang, A. D. Gotmare, S. Savarese, and S. C. H. Hoi, "CodeRL: Mastering code generation through pretrained models and deep reinforcement learning," in *NeurIPS*, 2022. 10

[148] J. Liu, Y. Zhu, K. Xiao, Q. Fu, X. Han, W. Yang, and D. Ye, "RLTF: Reinforcement learning from unit test feedback," *Transactions on Machine Learning Research*, 2023. 10

[149] X. Chen, C. Zhou, Y. Liu, and J. Yang, "RM-RL: Role-model reinforcement learning for precise robot manipulation," *arXiv:2510.15189*, 2025. 10

[150] S. El Ghazouali, A. Gucciardi, and U. Michelucci, "Class-conditional self-reward mechanism for improved text-to-image models," *arXiv:2405.13473*, 2024. 10

[151] W. Jin, Y. Niu, J. Liao, C. Duan, A. Li, S. Gao, and X. Liu, "SRUM: Fine-grained self-rewarding for unified multimodal models," *arXiv:2510.12784*, 2025. 10

[152] W. Liu, J. Chen, X. Ji, T. Tan, S. Liu, and J. Li, "Diving into self-evolving training for multimodal reasoning," in *ICML*, 2025. 10, 16, 19, 26

[153] P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui, "Math-Shepherd: Verify and reinforce LLMs step-by-step without human annotations," in *ACL*, 2024. 10

[154] G. Cui, L. Yuan, Z. Wang, H. Wang, W. Li, B. He, Y. Fan, T. Yu, Q. Xu, W. Chen, J. Yuan, H. Chen, K. Zhang, X. Lv, S. Wang, Y. Yao, X. Han, H. Peng, Y. Cheng, Z. Liu, M. Sun, B. Zhou, and N. Ding, "Process reinforcement through implicit rewards," *arXiv:2502.01456*, 2025. 10

[155] Y. Wu, W. Yuan, E. Martinez Licon, A. Qi, V. Guizilini, J. Mao, and Y. Wang, "Large reward models: Generalizable online robot reward generation with vision-language models," *arXiv:2603.16065*, 2026. 10

[156] Q. Chen, H. Zheng, J. Yu, S. Huang, J. Sun, K. Goldberg, C. Wen, P. Abbeel, Y. Shentu, P. Wu, and M. Schwager, "SARM2: Multi-task stage aware reward modeling for self improving robotic manipulation," *arXiv:2606.10305*, 2026. 10

[157] Z. Bai, C. Gao, and M. Z. Shou, "EVOLVE-VLA: Test-time training from environment feedback for vision-language-action models," *arXiv:2512.14666*, 2025. 10

[158] Youtu-Agent Team, "SmartSnap: Proactive evidence seeking for self-verifying agents," *arXiv:2512.22322*, 2025. 10

[159] A. Pan, E. J. Shern, S. R. Bowman, and J. Steinhardt, "Feedback loops with language models drive in-context reward hacking," in *ICML*, 2024. 10

[160] R. Zhang, D. Ji, L. Zhu, X. Liu, Y. Meng, R. Chu, and Y. Yang, "Video-Zero: Self-evolution video understanding," in *NeurIPS*, 2026. 10, 13, 14, 15, 16

[161] J. Hu, J. Shim, C. Tang, Y. Sung, B. Liu, P. Stone, and R. Martín-Martín, "Simple recipe works: Vision-language-action models are natural continual learners with reinforcement learning," *arXiv:2603.11653*, 2026. 11, 12, 23, 25

[162] Z. Wang, H. Xu, J. Wang, X. Zhang, M. Yan, J. Zhang, F. Huang, and H. Ji, "Mobile-Agent-E: Self-evolving mobile assistant for complex tasks," *arXiv:2501.11733*, 2025. 11, 13, 20, 21, 22, 23, 27, 29

[163] G. Liu, S. Geng, S. Li, H. Cui, S. Zhang, X. Liu, and T. Liu, "WebCoach: Self-evolving web agents with cross-session memory guidance," *arXiv:2511.12997*, 2025. 11, 20, 21, 22, 23, 27, 34

[164] L. Hu, X. Liang, and L. Zhu, "From interaction traces to persistent skills: Online evolution for computer-use agents," *arXiv:2609.04869*, 2026. 11, 22

[165] S. Garg, A. Singh, and G. K. Nayak, "SIDiffAgent: Self-improving diffusion agent," *arXiv:2602.02051*, 2026. 11, 18, 19

[166] P. Song, Z. Liang, K. Fu, M. Wang, J. Yang, and S. Liu, "EmbodiedRSI: Active continual robot learning through hypothesis-guided co-evolution," *arXiv:2610.10498*, 2026. 11, 23, 24, 25, 26

[167] X. Lai, Z. Tian, Y. Chen, S. Yang, X. Peng, and J. Jia, "Step-DPO: Step-wise preference optimization for long-chain reasoning of LLMs," *arXiv:2406.18629*, 2024. 10

[168] Z. Wei, W. Yao, Y. Liu, W. Zhang, Q. Lu, L. Qiu, C. Yu, P. Xu, C. Zhang, B. Yin, H. Yun, and L. Li, "WebAgent-R1: Training web agents via end-to-end multi-turn reinforcement learning," in *EMNLP*, 2025. 10

[169] H. Deng, K. Yan, C. Mao, X. Wang, Y. Liu, C. Gao, and N. Sang, "DenseGRPO: From sparse to dense reward for flow matching model alignment," *arXiv:2601.20218*, 2026. 10

[170] M. Sunil, M. Venmathimaran, and M. S. Kavitha, "iReasoner: Trajectory-aware intrinsic reasoning supervision for self-evolving large multimodal models," in *Findings of ACL*, 2026. 10, 14, 15

[171] A. Singh, S. Marjit, W. Lin, P. Gavrikov, S. Yeung-Levy, H. Kuehne, R. Feris, S. Doveh, J. Glass, and M. J. Mirza, "TTRV: Test-time reinforcement learning for vision language models," in *CVPR*, 2026. 10, 14, 15, 27

[172] A. Soni, S. Venkataraman, A. Chandra, S. Fischmeister, P. Liang, B. Dai, and S. Yang, "VideoAgent: Self-improving video generation for embodied planning," *arXiv:2410.10076*, 2024. 10, 25

[173] Z. Zhou, P. Atreya, A. Lee, H. R. Walke, O. Mees, and S. Levine, "Autonomous improvement of instruction following skills via foundation models," in *CoRL*, 2024. 10, 25, 26

[174] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017. 11

[175] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *NeurIPS*, 2023. 11

[176] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human

behavior," in *ACM Symposium on User Interface Software and Technology*, 2023. 11

[177] D. Yan, H. Zhang, G. Huzhang, Y. Li, Y. Wang, Q.-G. Chen, Z. Xu, W. Luo, Y. Li, W. Dong, and C. Shen, "$M^2$: Dual-memory augmentation for long-horizon web agents via trajectory summarization and insight retrieval," *arXiv:2603.00503*, 2026. 11

[178] LV-NUS Lab, "Mem-W: Latent memory-native GUI agents," *arXiv:2605.09317*, 2026. 11

[179] T. Cai, X. Wang, T. Ma, X. Chen, and D. Zhou, "Large language models as tool makers," in *ICLR*, 2024. 11

[180] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba, "Large language models are human-level prompt engineers," in *ICLR*, 2023. 11

[181] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *ICLR*, 2024. 11

[182] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into state-of-the-art pipelines," in *ICLR*, 2024. 11

[183] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *CVPR*, 2017. 11

[184] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with A-GEM," in *ICLR*, 2019. 11

[185] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara, "Dark experience for general continual learning: A strong, simple baseline," in *NeurIPS*, 2020. 11

[186] Z. Li and D. Hoiem, "Learning without forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2935–2947, 2018. 11

[187] Z. Lin, F. Liu, Y. Yang, J. Lyu, Y. Gao, Y. Liu, Z. Lu, Y. Yu, M. Yang, J. Li, D. Ye, and J. Jiang, "UI-Voyager: A self-evolving GUI agent learning via failed experience," *arXiv:2603.24533*, 2026. 11, 22

[188] H. Li, Y. You, H. Su, and L. Guibas, "PhysMem: Scaling test-time memory for embodied physical reasoning," *arXiv:2602.20323*, 2026. 11, 25, 26

[189] X. Ding, L. Mi, M. Huang, Z. Wang, C. Zhang, Z. Hao, F. Chen, X. Li, Y. Zheng, Y. Guo, W. Wang, K. Li, H. Wu, Y. Liu, and T. Cao, "Zetta: An efficient closed-loop embodied harness for self-evolving physical intelligence," *arXiv:2608.16590*, 2026. 11

[190] P. Wang, Z. Ma, Y. Chang, X. Luo, Y. Zhang, Z. Wang, X. Yang, S. Feng, Y. Yang, and D. Li, "Self-evolving embodied agents via skill-harness evolution," *arXiv:2608.11350*, 2026. 11, 23, 24, 25

[191] H. Wang, C. Shi, C. Yang, Y. Wu, T. Berg-Kirkpatrick, and X. Ma, "Recursive self-improvement in unified multimodal models," *arXiv:2610.03002*, 2026. 12, 17, 18, 19, 20, 27, 28, 29

[192] S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton, "Loss of plasticity in deep continual learning," *Nature*, vol. 632, pp. 768–774, 2024. 13

[193] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, "AI models collapse when trained on recursively generated data," *Nature*, vol. 631, pp. 755–759, 2024. 13

[194] M. Gerstgrasser, R. Schaeffer, A. Dey, R. Rafailov *et al.*, "Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data," *arXiv:2404.01413*, 2024. 13

[195] J. Liu, G. Liu, J. Liang, Y. Li, J. Liu, X. Wang, P. Wan, D. Zhang, and W. Ouyang, "Flow-GRPO: Training flow matching models via online RL," in *NeurIPS*, 2025. 12, 17, 19, 26, 27, 28, 31

[196] M. Liu, Y. Zhang, S. Liu, L. Wang, and W. Zhang, "Wan-R1: Verifiable-reinforcement learning for video reasoning," *arXiv:2603.27866*, 2026. 12

[197] K. Zhao, J. Shi, B. Zhu, J. Zhou, X. Shen, Y. Zhou, Q. Sun, and H. Zhang, "Real-time motion-controllable autoregressive video diffusion," *arXiv:2510.08131*, 2025. 12

[198] X. Meng, Z. Zhang, Z. Zhang, J. Liao, L. Qin, and W. Wang, "Identity-GRPO: Optimizing multi-human identity-preserving video generation via reinforcement learning," *arXiv:2510.14256*, 2025. 12

[199] T. Wu, Y. Jiang, Y. Lu, Z. Wang, Z. Huang, Z. Qin, and X. Li, "MultiCrafter: High-fidelity multi-subject generation via disentangled attention and identity-aware preference alignment," in *CVPR*, 2026. 12

[200] H. He, W. Yao, K. Ma, W. Yu, H. Zhang, T. Fang, Z. Lan, and D. Yu, "OpenWebVoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization," in *ACL*, 2025. 12, 13, 20, 21, 23

[201] B. Zheng, M. Y. Fatemi, X. Jin, Z. Z. Wang, A. Gandhi, Y. Song, Y. Gu, J. Srinivasa, G. Liu, G. Neubig, and Y. Su, "SkillWeaver: Web agents can self-improve by discovering and honing skills," *arXiv:2504.07079*, 2025. 12, 20, 21, 22, 27

[202] A. Huang, A. Block, D. J. Foster, D. Rohatgi, C. Zhang, M. Simchowitz, J. T. Ash, and A. Krishnamurthy, "Self-improvement in language models: The sharpening mechanism," in *ICLR*, 2025. 12

[203] Y. Yue, Z. Chen, R. Lu, A. Zhao, Z. Wang, Y. Yue, S. Song, and G. Huang, "Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?" in *NeurIPS*, 2025. 12

[204] R. Shao, S. S. Li, R. Xin, S. Geng, Y. Wang, S. Oh, S. S. Du, N. Lambert, S. Min, R. Krishna, Y. Tsvetkov, H. Hajishirzi, P. W. Koh, and L. Zettlemoyer, "Spurious rewards: Rethinking training signals in RLVR," in *ICML*, 2026. 12, 16

[205] Y. Guo, W. Yang, H. Yu *et al.*, "OpAgent: Operator agent for web navigation," *arXiv:2602.13559*, 2026. 12, 23

[206] D. Manheim and S. Garrabrant, "Categorizing variants of Goodhart's law," *arXiv:1803.04585*, 2018. 13

[207] L. Gao, J. Schulman, and J. Hilton, "Scaling laws for reward model overoptimization," in *ICML*, 2023. 13

[208] X. Chen, W. Hu, H. Li, Y. Wang, J. Zhou, Z. Chen, M. Cao, Y. Zeng, K. Zhang, Y.-J. Yuan, J. Han, H. Xu, and X. Liang, "SyncLoop: A multimodal dual-loop framework for self-improving mathematical reasoning," *arXiv:2507.16518*, 2025. 13, 14, 15, 16, 27

[209] J. Zhu, Y. Zou, X. Li, J. Bailey, D. Phung, T. Yu, Y. Hou, J. Liu, J. Zhou, S. Wang, N. Duan, J. Liu, and Q. Ke, "Tempo-Zero: Recursive self-play in executable worlds for video reasoning," *arXiv:2610.32433*, 2026. 13, 15, 16

[210] S. Wang, Z. Wu, X. Zhou, C. Cui, C. Jia, B. Liu, C. Li, X. Chen, Y. Yang, Y. Zhang, and W. Huang, "TTRSD: Test-time reinforcement learning with self-distillation for vision-language models," *arXiv:2609.33414*, 2026. 14

[211] W. Wen, T. Zhi, K. Fan, Y. Li, X. Peng, Y. Zhang, Y. Liao, J. Li, and L. Zhang, "Self-evolving vision-language models for image quality assessment via voting and ranking," in *ICLR*, 2026. 13, 14

[212] Z. Wu, K. Shi, C. Zhang, Z. Liao, J. Yang, N. Yang, Q. Peng, L. Zhang, H. Xu, T. Su, Z. Yang, H. Lu, and H. Wang, "When models judge themselves: Unsupervised self-evolution for multimodal reasoning," *arXiv:2603.21289*, 2026. 14, 15, 27

[213] Y. Ding and R. Zhang, "Sherlock: Self-correcting reasoning in vision-language models," in *NeurIPS*, 2025. 14, 15

[214] D. Ahn, Y. Choi, S. Kim, Y. Yu, D. Kang, and J. Choi, "ISR-DPO: Aligning large multimodal models for videos by iterative self-retrospective DPO," in *AAAI*, 2025. 14, 15, 16, 27

[215] J. Yang, T. Xiao, Y. Dang, C. Wang, Y. Yang, Q. Fan, T. Ding, W. Li, F. Miao, and Y. Gao, "BUS: Brain-inspired unsupervised self-reflection via backward prediction for multimodal reasoning," *arXiv:2607.07361*, 2026. 14, 15

[216] Agent0-VL Team, "Agent0-VL: Exploring self-evolving agent for tool-integrated vision-language reasoning," in *ICML*, 2026. 14, 15, 16

[217] S. Wu, X. Wang, Y. Zhang, H. Zhu, and S. Yeung-Levy, "Transductive visual programming: Evolving tool libraries from experience for spatial reasoning," in *ICLR*, 2026. 14, 16

[218] Q. Wang, B. Liu, T. Zhou, J. Shi, Y. Lin, Y. Chen, H. H. Li, K. Wan, and W. Zhao, "Vision-Zero: Scalable VLM self-improvement via strategic gamified self-play," in *ICLR*, 2026. 14, 15, 16, 27

[219] H. Zhang, Y. Ding, Y. Zhou, X. Du, S. Zhang, Z. Zhao, Y. Xi, and H. Chen, "Spatial memory agent: Experience-grounded procedure memory for spatial intelligence," *arXiv:2608.12743*, 2026. 14, 16

[220] B. Yu, S. Lian, Z. Shen, X. Lin, Y. Zhang, H. Liu, H. Li, Y. Wei, K. Hu, Y. Du, Z. Zhang, B. Wang, and K. Chen, "EvoSpatial: Experience-driven skill and tool evolution for spatial reasoning agents," *arXiv:2610.16679*, 2026. 14, 15, 16

[221] Z. Li, W. Yu, C. Huang, R. Liu, Z. Liang, F. Liu, J. Che, D. Yu, J. Boyd-Graber, H. Mi, and D. Yu, "Self-rewarding vision-language model via reasoning decomposition," *arXiv:2508.19652*, 2025. 14, 15

[222] H. Li, J. Tang, R. Chen, L. Sun, and X. Chu, "$M^2$Note: Continual evolution of vision language models via mistake notebook learning," *arXiv:2607.00685*, 2026. 14, 15, 16, 28

[223] J. Wang, Y. Li, X. Li, S. Wang, X. Gong, D. Zhang, Z. Lv, Y. Le, S. Yuan, A. Zhang, D. Cao, and T.-S. Chua, "VisualEvolve: A skill self-evolution framework for visual reasoning," *arXiv:2610.28386*, 2026. 14, 15, 16

[224] D. Cheng, R. Yuan, R. Wei, N. Song, Y. Li, and W. Li, "CoWorld Reasoner: Agentic multimodal reasoning with the evolving video world model-driven harness," *arXiv:2610.21910*, 2026. 14, 15, 16

[225] Y. Sun, Y. Miao, H.-X. Ma, M. Zhou, M. Chen, T. Zhao, D. Wang, L. Lv, L. Xu, X. Jiang, and G. Jiang, "Dynamo: Dynamic skill-tool evolution for vision-language agents," *arXiv:2606.30185*, 2026. 14, 15, 16, 27, 28

[226] S. Zhang, Y. Xie, Y. Cheng, H. Zhang, and T. Zhang, "PhysNote: Self-knowledge notes for evolvable physical reasoning in vision-language model," in *ICLR Workshops*, 2026. 13, 14, 16

[227] S.-Y. Tian, Z.-X. Wang, X.-Y. Zhu, Z. Zhou, X. Yang, K.-Y. Yu, M. Yang, Y. Chen, and Y.-F. Li, "Self-evolving neuro-symbolic skills for tool-augmented spatial reasoning," *arXiv:2608.07955*, 2026. 14, 16

[228] S. Wei, H. Min, X. Dong, X. Lin, S. Cui, B. Jiang, Z. Dai, K. Kuang, G. Xu, F. Wu, and M. Zhang, "MetaForge: A self-evolving multimodal agent that retrieves, adapts, and forges tools on demand," *arXiv:2606.01801*, 2026. 14, 15, 16

[229] S. Venkatraman, R. Thawkar, O. Thawakar, R. M. Anwer, H. Cholakkal, S. Khan, and F. Khan, "Paying more attention to visual tokens in self-evolving large multimodal models," *arXiv:2606.27373*, 2026. 14

[230] Y. Liang, H. Zhou, M. Li, L. Li, C.-J. Hsieh, and T. Zhou, "Self-evolving visual questioner," *arXiv:2606.13929*, 2026. 14, 15, 27

[231] Y. Lee, Y.-J. Lee, and S. J. Hwang, "Trinity: Self-evolving vision-language models with a self-verifier," *arXiv:2610.04469*, 2026. 14, 16

[232] W. Yang, Y. Zhao, B. Zhang, Y. Liu, and Q. Ye, "Self-evolving visual agent via margin reward," *arXiv:2610.15084*, 2026. 14, 15, 16

[233] W. Xu, Y. Liu, Y. Wang, M. Rong, Z. Yang, Z. Zhong, S. Qian, and C. Xu, "VideoEvoHarness: Self evolving agent harness from unlabeled videos for video understanding," *arXiv:2610.15389*, 2026. 14, 16

[234] Y. Heng, C. Jiang, H. Yang, S. Zhang, and W. Ye, "EVE: Verifiable self-evolution of MLLMs via executable visual transformations," *arXiv:2604.18320*, 2026. 14, 15, 16

[235] D. Li, Y. Zhao, X. Cheng, K. Lin, H. Peng, H. Li, Z. Wang, Y. Dai, H. Li, J. Wang, Y. Shi, L. Zhao, J. Sun, Z. Ge, X. Zhang, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen, "SpatialEvo: Self-evolving spatial intelligence via deterministic geometric environments," *arXiv:2604.14144*, 2026. 13, 14, 15, 16, 27

[236] L. Qiu, H. Zeng, Y. Liu, B. Sun, G. Liao, and J. Liu, "DUEL: Adversarial self-play for multimodal reasoning," *arXiv:2605.24794*, 2026. 14

[237] J. Liu, Y. Li, Y. Sun, M. Wang, Y. Cheng, R. Qian, P. Koniusz, Y. Chen, and D. Wang, "Self-evolving spatial reasoning in vision language models via geometric logic consistency," *arXiv:2605.18162*, 2026. 14, 15, 16

[238] M. Lu, L. Zhu, O. Xiao, Y. Zhuang, Z. Wang, K. Wu, B. Liu, Y. Wang, C. Fleming, W. Shi, and X. Wang, "VICO: Visual environments co-evolving for vision-language model reasoning," in *NeurIPS*, 2026. 15, 16

[239] J. Hu, Y. Jiang, C. Tang, M. Cai, X. Liu, and X. Yue, "PhysEvolve: Verifier-grounded self-evolution for physical reasoning with executable worlds," *arXiv:2610.34921*, 2026. 13, 15, 16

[240] P. Lu, H. Bansal, T. Xia, J. Liu, C. Li, H. Hajishirzi, H. Cheng, K.-W. Chang, M. Galley, and J. Gao, "MathVista: Evaluating mathematical reasoning of foundation models in visual contexts," in *ICLR*, 2024. 13, 31

[241] R. Zhang, D. Jiang, Y. Zhang, H. Lin, Z. Guo, P. Qiu, A. Zhou, P. Lu, K.-W. Chang, Y. Qiao *et al.*, "MathVerse: Does your multi-modal LLM truly see the diagrams in visual math problems?" in *ECCV*, 2024. 13, 31

[242] A. Masry, X. L. Do, J. Q. Tan, S. Joty, and E. Hoque, "ChartQA: A benchmark for question answering about charts with visual and logical reasoning," in *Findings of ACL*, 2022. 13, 31

[243] C. Huang, W. Yu, X. Wang, H. Zhang, Z. Li, R. Li, J. Huang, H. Mi, and D. Yu, "R-Zero: Self-evolving reasoning LLM from zero data," in *ICLR*, 2026. 15

[244] S. Shafayat, F. Tajwar, R. Salakhutdinov, J. Schneider, and A. Zanette, "Can large reasoning models self-train?" *arXiv:2505.21444*, 2025. 15, 27

[245] W. Mao, H. Chen, Z. Yang, and M. Z. Shou, "The image as its own reward: Reinforcement learning with adversarial reward for image generation," in *CVPR*, 2026. 17, 18, 19, 20, 27, 28

[246] H. Yang, Y. Zhou, W. Han, and J. Shen, "Self-rewarding large vision-language models for optimizing prompts in text-to-image generation," in *Findings of ACL*, 2025. 17, 18, 19

[247] G. Tang, H. Luo, T. Wang, Y. Zhang, and H. Wang, "PRISM: Prompt refinement via image-grounded self-rewarding mechanism for text-to-image generation," *arXiv:2607.24353*, 2026. 17, 18, 19, 27, 34

[248] Y. Xue, Z. Li, P. Torr, and J. Gu, "Safe autoregressive image generation with iterative self-improving codebooks," in *ICML*, 2026. 17, 18, 19

[249] X. Yang, M. Yang, J. Gong, L. Qin, Z. Tan, and H. Li, "Dual-IPO: Dual-iterative preference optimization for text-to-video generation," in *ICLR*, 2026. 17, 18, 19

[250] X. Guan, S. Yang, Y. Bao, C. Wang, Z. Gu, Z. Chen, K. Chen, J. Huo, and Y. Gao, "Agentic physics-grounded video generation via memory-augmented in-context reasoning," *arXiv:2610.23677*, 2026. 17, 18, 19

[251] L. Zhao, L. Zhuo, and M. Elhoseiny, "On-policy self-distillation for multi-turn image editing," *arXiv:2609.35611*, 2026. 17, 18, 19, 27

[252] Y. Lin, L. Wang, K. Lin, Z. Lin, K. Gong, W. Li, B. Lin, Z. Li, S. Zhang, Y. Peng, W. Dai, X. Ding, C. Wang, and Q. Lu, "JarvisEvo: Towards a self-evolving photo editing agent with synergistic editor-evaluator optimization," in *CVPR*, 2026. 17, 18

[253] T. Wang, L. Ding, Z. Tao, Y. Zhan, Z. Ma, W. Wu, Y. Lei, J. Wang, Y. Wu, Y. Xu, H. Zhu, Q. Liu, N. J. Yuan, Y. Zhang, and H. Xiong, "EvoDesign: Agentic editable diagram creation via design expertise evolution," *arXiv:2604.09568*, 2026. 17, 18, 19

[254] Z. Hu, A. Iscen, A. Jain, T. Kipf, Y. Yue, D. A. Ross, C. Schmid, and A. Fathi, "SceneCraft: An LLM agent for synthesizing 3D scenes as Blender code," in *ICML*, 2024. 17, 18, 19

[255] Z. Li, J. Zhang, P. Hu, J. Kuen, J. Gu, H. Rahmani, and J. Liu, "Automatic method illustration generation for AI scientific papers via drawing middleware creation, evolution, and orchestration," *arXiv:2603.29590*, 2026. 17, 18, 19, 20

[256] W. Jiang, Z. Cai, Y. Shao, C. Wang, B. Han, Z. Song, K. Chen, S. An, X. Yang, and Z. Yang, "ManimAgent: Self-evolving multimodal agents for visual education," *arXiv:2606.30296*, 2026. 17, 18, 19, 27, 28

[257] R. Huang, Q. Zhang, Z. Liu, Y. Gao, J. Wu, and H. Zhao, "Read it back: Pretrained MLLMs are zero-shot reward models for text-to-image generation," in *NeurIPS*, 2026. 17, 18, 19

[258] X. Jiang, B. Chen, G. Li, Y. Duan, R. Wang, and J. Zhang, "OctoT2I: A self-evolving agentic text-to-image router," in *CVPR*, 2026. 17, 19, 20

[259] W. Zhou, W. Gan, and J. Wang, "Mutually adversarial self-training with evolving data for unified multimodal models," *arXiv:2609.36224*, 2026. 17, 18, 19

[260] Y. Liu, H. Wang, J. Xiao, K. Zhu, X. Fu, and Z.-J. Zha, "RSI-gen: Reciprocal self-improvement for agentic visual generation," *arXiv:2610.18688*, 2026. 17, 19, 27, 28, 29

[261] J. Pan, L. Li, Y. Peng, Y.-M. Tang, S. Wang, Y. Sun, H. Wu, Q. Huang, and H. Wang, "Learning to generate via understanding: Understanding-driven intrinsic rewarding for unified multimodal models," *arXiv:2603.06043*, 2026. 18, 19

[262] S. Han, Z. Wang, P. Ling, W. Gong, Y. Liu, G. Jiang, Y. Cao, and X. Xie, "ImageRSI: Verification-driven recursive self-improvement for agentic image generation," *arXiv:2610.32120*, 2026. 18, 19, 20, 27

[263] F. Wu, D. Xing, Y. Huang, J. Wang, J. Wang, H. Geng, G. Wan, B. Zuo, X. Li, S. Tang *et al.*, "UniEvo-VL: An on-policy self-distillation training recipe for multimodal model self-improvement," *arXiv:2609.38721*, 2026. 18, 19, 34

[264] Z. Li, D. Liu, F. Liu, Y. Zhou, X. Wu, J. Chen, J. Xie, X. Wu, and L. Sun, "COMFYCLAW: Self-evolving skill harnesses for image generation workflows," *arXiv:2607.01709*, 2026. 17, 18, 19, 27

[265] W. Chen, K. Yu, B. Tian, J. Song, S. Liang, H. Jia, K. Cheng, H. Li, K. Yuan, L. Wang, J. Wu, S. Lai, and Y. Yue, "MemoGen: Can past experience improve future text-to-image generation?" *arXiv:2606.03243*, 2026. 18, 19

[266] S. Chen, Z. Xing, T. Ye, X. Geng, Y. Lin, J. Lai, X. He, F. Zhai, J. Gao, and L. Zhu, "GenEvolve: Self-evolving image generation agents via tool-orchestrated visual experience distillation," *arXiv:2605.21605*, 2026. 19

[267] Z. Xue, J. Wu, Y. Gao, F. Kong, L. Zhu, M. Chen, Z. Liu, W. Liu, Q. Guo, W. Huang, and P. Luo, "DanceGRPO: Unleashing GRPO on visual generation," *arXiv:2505.07818*, 2025. 17, 19, 26

[268] J. Ye, X. Zhang, H. Xu, H. Liu, J. Wang, Z. Zhu, Z. Zheng, F. Gao, J. Cao, Z. Lu, J. Liao, Q. Zheng, F. Huang, J. Zhou, and M. Yan, "Mobile-Agent-v3: Foundamental agents for GUI automation," *arXiv:2508.15144*, 2025. 20

[269] H. Lai, X. Liu, Y. Zhao, H. Xu, H. Zhang, B. Jing, Y. Ren, S. Yao, Y. Dong, and J. Tang, "ComputerRL: Scaling end-to-end online reinforcement learning for computer use agents," in *ICLR*, 2026. 20, 22, 26

[270] F. Lu, Z. Zhong, S. Liu, C.-W. Fu, and J. Jia, "ARPO: End-to-end policy optimization for GUI agents with experience replay," *arXiv:2505.16282*, 2025. 20

[271] S. Nong, X. Tang, J. Xu, S. Zhou, J. Chen, T. Jiang, and W. Xu, "CRAFT-GUI: Curriculum-reinforced agent for GUI tasks," in *ICLR Workshops*, 2026. 20

[272] Y. Han and T. Qian, "WM-R1: Training GUI agents to reason and leverage world models with reinforcement learning," *arXiv:2608.27508*, 2026. 20

[273] C. Liu, L. Chen, H. Zhou, X. Zhang, Q. Kong, P. Tong, W. Wang, X. Yu, S. Hoi, and Y. Wang, "What memory do GUI agents really need? from passive records to active task-driving states," *arXiv:2606.31612*, 2026. 20

[274] G. Liu, P. Zhao, G. Wu, Y. Yin, M. Li, L. Liu, C. Liu, Z. Qi, M. Wang, L. Guo, J. Zhang, and Y. Liu, "MobileForge: Annotation-free adaptation for mobile GUI agents with hierarchical feedback-guided policy optimization," *arXiv:2606.19930*, 2026. 20

[275] J. Wang, H. Xu, X. Zhang, Z. Zhu, M. Yan, J. Ye, and J. Sang, "STAMP: Training explicit memory for mobile GUI agents in controllable and scalable virtual environments," *arXiv:2605.29324*, 2026. 20

[276] W. Wu, K. Zhou, R. Yuan, V. Yu, S. Wang, Z. Hu, and B. Huang, "Auto-scaling continuous memory for GUI agent," *arXiv:2510.09038*, 2025. 20

[277] B. He, X. Zhang, M. Ding, and X. Liu, "SCAFFOLD: Self-improving web agents via recursive parametric skill abstraction," *arXiv:2609.05511*, 2026. 20

[278] S. Agashe, J. Han, S. Gan, J. Yang, A. Li, and X. E. Wang, "Agent S: An open agentic framework that uses computers like a human," in *ICLR*, 2025. 20, 21, 22, 27, 29

[279] Y. Liu, H. Sun, W. Liu, J. Luan, B. Du, and R. Yan, "MobileSteward: Integrating multiple app-oriented agents with self-evolution to automate cross-app instructions," in *KDD*, 2025. 20, 21, 22

[280] W. Jiang, Y. Zhuang, C. Song, X. Yang, J. T. Zhou, and C. Zhang, "AppAgentX: Evolving GUI agents as proficient smartphone users," *arXiv:2503.02268*, 2025. 20, 21, 22

[281] B. Chen, B. Zhang, F. Tang, Z. Lu, Y. Du, T. Chen, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen, "Reflect, revise, reuse: Training-free skill evolution for GUI agents," *arXiv:2609.17653*, 2026. 20, 21, 22, 27, 28, 29

[282] L. Sun, J. Zhang, S. Wang, and Z. Wei, "MAGNET: Towards adaptive GUI agents with memory-driven knowledge evolution," *arXiv:2601.19199*, 2026. 20, 21, 22

[283] Anonymous, "Agent S4: Evolutionary computer use," OpenReview, ICLR 2027 submission, 2026. 20, 21

[284] W. Chen, B. Miao, Q. Yu, W. Bu, G. Wang, W. Zhang, S. Zhang, J. Li, and S. Tang, "Learning to adapt: Self-improving web agent via cognitive-aware exploration," in *CVPR*, 2026. 20, 21, 22, 23, 26, 27

[285] X. H. Lù, A. Kazemnejad, N. Meade, A. Patel, D. Shin, A. Zambrano, K. Stańczak, P. Shaw, C. J. Pal, and S. Reddy, "AgentRewardBench: Evaluating automatic evaluations of web agent trajectories," in *Conference on Language Modeling*, 2025. 21, 32, 33

[286] G. Yang, Z. Qu, X. Li, Z. Wang, M. Zhang, S. Zeng, and Z. Dai, "GUI-HARVEST: Self-improving GUI agents through evidence-driven harness evolution," *arXiv:2610.00948*, 2026. 22, 34

[287] S. Tan, K. Dou, Y. Zhao, and P. Krähenbühl, "Interactive post-training for vision-language-action models," *arXiv:2505.17016*, 2025. 23, 25

[288] K. Chen, Z. Liu, T. Zhang, Z. Guo, S. Xu, H. Lin *et al.*, "$\pi_{r1}$: Online RL fine-tuning for flow-based vision-language-action models," *arXiv:2510.25889*, 2025. 23

[289] C. Li, Z. Zhao, R. Li, Y. Gao, K. Liu, J. Li, D. Wang, T. Si, M. Pan, W. Ouyang, and D. Zhou, "RoboBridge: A self-evolving embodied agent framework for sim-to-real transfer," *arXiv:2610.02717*, 2026. 24, 25, 26

[290] W. Li, Y. Chi, Y. Zheng, C. Liu, X. Wang, M. Gao, Z. Jiang, D. Hong, C. Xiao, K. Yang, and Q. Na, "EmbodiedEvo: Self-evolving coding agent for physical intelligence," *arXiv:2610.18867*, 2026. 23, 24, 25, 27

[291] S. Bao, F. Huang, Y. Lin, Y. Feng, G. Li, C. Zhao, Y. Li, J. He, C. Chi, and J. Zhang, "RoboHarn-Evo: Evolving hierarchical physical knowledge for self-improving robotic manipulation," *arXiv:2609.37583*, 2026. 24, 25

[292] B. Xu, Y. Shang, and E. Ferrara, "Don't drop the BATON: Long-horizon robot manipulation via agentic subtask exploration and transition-aware memory," *arXiv:2608.16889*, 2026. 23, 24, 25

[293] Y. Zhang, H. Zhang, F. Gao, X. Li, Z. Liu, Y. Nie, C. Zhu, J. Qiu, Y. Yan, J. Liu, W. Tang, J. Rao, Z. Fang, C. Wei, Y. Wang, W. Ding, and C. Yu, "Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents," *arXiv:2607.08448*, 2026. 23, 24, 25

[294] R. Lu, Y. Wu, E. Kou, H. Lin, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, M. Chowdhury, J. Kautz, Y. Zhu, L. Fan, and G. Wang, "ASPIRE: Agentic skills discovery for robotics," *arXiv:2607.00272*, 2026. 23, 24, 25

[295] H. Li, D. Li, Y. Li, Y. Du, W. Zhang, W. Lu, J. Xiao, Y. Zhuang, and Y. Shen, "ViSkill: Reinforcing VLM agents with evolving visual-native skills," *arXiv:2610.12403*, 2026. 23, 24, 25

[296] L. He, Y. Tian, L. Fan, J. Pi, Y. Lu, S. Su, M. Shi, and S. Feng, "Policy-level recursive self-improvement for embodied AI with a criticality world model," *arXiv:2607.28251*, 2026. 23, 24, 25, 26, 27, 28

[297] Z. Wei, F. Tan, J. Wang, and H. Zhou, "Scale and selection: What makes automatic harness evolution work for visual-interface robot agents," *arXiv:2609.39304*, 2026. 23, 24, 25, 26

[298] H. H. Chen, S. Chen, Y. Xu, W. Ge, and Y.-C. Chen, "RoboEvolve: Co-evolving planner-simulator for robotic manipulation with limited data," *arXiv:2605.13775*, 2026. 23, 24, 25, 26, 27, 28

[299] H. Bi, Z. Zhou, Y. Tang, J. Pang, S. Huang, H. Liu, R. Wang, S. Huang, Y. Wang, Y. Cheng, R. Zhao, Z. Li, H. Tan, X. Liu, J. Wan, J. Liu, M. Zhao, F. Bao, and J. Zhu, "Motus2: A self-evolving general world model for dexterous manipulation," *arXiv:2608.30237*, 2026. 23, 24, 25, 27, 28

[300] J. Wang, B. Zhao, Y. He, J. Peng, and X. Zhang, "Evolvable embodied agent for robotic manipulation via long short-term reflection and optimization," *arXiv:2604.13533*, 2026. 23, 24, 25, 26

[301] K. Qu, G. Lan, R. Zurbrügg, C. Chen, C. E. Mower, H. Bou-Ammar, and M. Hutter, "A pragmatist robot: Learning to plan tasks by experiencing the real world," *arXiv:2507.16713*, 2025. 23, 24, 25, 26, 27

[302] J. Zhang, J. Ge, H. Yoo, L. Fu, Z. Yang, Y. Liu, R. Saravanan, S. Yin, J. Yu, D. Niu, Z. Wang, R. Herzig, K. Goldberg, Y. Bai, D. M. Chan, I. Stoica, A. Kanazawa, J. Lei, H. Feng, and T. Darrell, "Playful agentic robot learning," *arXiv:2606.19419*, 2026. 23, 24, 25, 26, 27, 28, 29, 32

[303] Z. Wen, Y. Chen, Y. Cao, W. Chen, Y. Zou, W. Yu, F. Kuang, H. Xue, J. Lv, C. Wen, and C. Lu, "RoboRSI: Stable, efficient, and reusable robot self-evolution in complex real-world environments," *arXiv:2610.12424*, 2026. 23, 24, 25

[304] B. Li, D. Wu, Z. Yan, X. Liu, Z. Zeng, L. Li, and H. Zha, "Reflection-based task adaptation for self-improving VLA," *arXiv:2510.12710*, 2025. 23, 24, 25, 26, 27, 31

[305] X. Zhang, M. Wang, E. Bauer, H.-X. Yu, H. Huang, J. Wu, and M. Pavone, "Co-evolving robot orchestrators and policies through deployment," *arXiv:2610.09228*, 2026. 23, 24, 25, 26

[306] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac Gym: High performance GPU-based physics simulation for robot learning," *arXiv:2108.10470*, 2021. 23

[307] Z. Jiang, S. Zhou, Y. Jiang, Z. Huang, M. Wei, Y. Chen, T. Zhou, Z. Guo, H. Lin, Q. Zhang, Y. Wang, H. Li, C. Yu, and D. Zhao, "WoVR: World models as reliable simulators for post-training VLA policies with RL," *arXiv:2602.13977*, 2026. 23

[308] Y. Wang, R. Yu, F. Zhang, J. Lu, X. Qin *et al.*, "Reinforcing VLAs in task-agnostic world models," *arXiv:2605.12334*, 2026. 23

[309] Y.-J. Wang, H. Jiang, S. Deng, H. Xue, W. Ye, R. Duan, N. Haghtalab, S. S. Sastry, P. Abbeel, and H. Qi, "Reconstruct, practice, go real: Guided self-improvement for embodied agents," *arXiv:2610.02204*, 2026. 25

[310] Y. Zhao, Z. Ni, H. Chen, R. Zhang, and Q. Zhu, "Skills in weights, memory in code: Hybrid learning for memory-dependent robot manipulation," *arXiv:2608.09410*, 2026. 25

[311] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue *et al.*, "EMPIRE: Agentic robot policy self-improvement in the real world," *arXiv:2606.19980*, 2026. 25

[312] R. Jha, A. Bhattacharyya, Y. K. Singla, R. R. Shah, C. Chen, and B. Krishnamurthy, "SPRO: Improving image generation via self-play," in *NeurIPS*, 2025. 26

[313] J. Zhang, J. Wu, W. Chen, Y. Ji, X. Xiao, W. Huang, and K. Han, "OnlineVPO: Align video diffusion model with online video-centric preference optimization," *arXiv:2412.15159*, 2024. 26

[314] K. Chen, Z. Xu, Y. Shen, Z. Lin, Y. Yao, and L. Huang, "SuperFlow: Training flow matching models with RL on the fly," *arXiv:2512.17951*, 2025. 26

[315] H. Guo, J. Wu, J. Liu, Y. Gao, Z. Ye, L. Yuan, X. Wang, Y. Yu, and W. Huang, "Leveraging verifier-based reinforcement learning in image editing," *arXiv:2604.27505*, 2026. 26

[316] K. Cheng, Y. Li, F. Xu, J. Zhang, H. Zhou, and Y. Liu, "Vision-language models can self-improve reasoning via reflection," in *NAACL*, 2025. 26

[317] M. Wu, J. Yang, S. Qian, X. Wang, J. Jiang, Q. Wang *et al.*, "SVR-R1: Bootstrapping multi-modal reasoning with self-verification in reinforcement learning," *arXiv:2607.10966*, 2026. 26

[318] S. Amjith, "Self-questioning vision-language models: Reinforcement learning for compositional visual reasoning," *arXiv:2606.15651*, 2026. 26

[319] H. Li, K. Zheng, J. Wu, C. Xu, Q. Sun, H. Hu, and Y. Yang, "VeriEvol: Scaling multimodal mathematical reasoning via verifiable evol-instruct," *arXiv:2606.23543*, 2026. 26

[320] R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *ACM SIGART Bull.*, vol. 2, no. 4, pp. 160–163, 1991. 28

[321] M. Janner, J. Fu, M. Zhang, and S. Levine, "When to trust your model: Model-based policy optimization," in *NeurIPS*, 2019. 28

[322] Y. Song, H. Zhang, C. Eisenach, S. Kakade, D. Foster, and U. Ghai, "Mind the gap: Examining the self-improvement capabilities of large language models," in *ICLR*, 2025. 28

[323] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine, "Scalable deep reinforcement learning for vision-based robotic manipulation," in *CoRL*, 2018. 29

[324] Z. Navasardyan, T. Danielyan, and H. Davtyan, "FailBench: How reliable are VLMs at judging robot task success?" *arXiv:2609.03611*, 2026. 29, 30, 33

[325] H. Ding, P. Liu, J. Wang, Z. Ji, M. Cao, R. Zhang, L. Ai, E. Yang, T. Shi, and L. Yu, "DynaWeb: Model-based reinforcement learning of web agents," *arXiv:2601.22149*, 2026. 29, 33

[326] K. Wang, J. Pan, W. Shi *et al.*, "Measuring multimodal mathematical reasoning with MATH-Vision dataset," *arXiv:2402.14804*, 2024. 31

[327] X. Yue, Y. Ni, K. Zhang *et al.*, "MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert AGI," *arXiv:2311.16502*, 2023. 31

[328] P. Lu, R. Gong, S. Jiang *et al.*, "Inter-GPS: Interpretable geometry problem solving with formal language and symbolic reasoning," *arXiv:2105.04165*, 2021. 31

[329] W. Chow, J. Mao, B. Li *et al.*, "PhysBench: Benchmarking and enhancing vision-language models for physical world understanding," *arXiv:2501.16411*, 2025. 31

[330] J. Yang, S. Yang, A. W. Gupta *et al.*, "Thinking in space: How multimodal large language models see, remember, and recall spaces," *arXiv:2412.14171*, 2024. 31

[331] T. Guan, F. Liu, X. Wu *et al.*, "HallusionBench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models," *arXiv:2310.14566*, 2023. 31

[332] S. Yang, R. Xu, Y. Xie *et al.*, "MMSI-Bench: A benchmark for multi-image spatial intelligence," *arXiv:2505.23764*, 2025. 31

[333] Y. Zhao, L. Xie, H. Zhang *et al.*, "MMVU: Measuring expert-level multi-discipline video understanding," *arXiv:2501.12380*, 2025. 31

[334] D. Ghosh, H. Hajishirzi, and L. Schmidt, "GenEval: An object-focused framework for evaluating text-to-image alignment," *arXiv:2310.11513*, 2023. 31

[335] K. Huang, C. Duan, K. Sun *et al.*, "T2I-CompBench++: An enhanced and comprehensive benchmark for compositional text-to-image generation," *arXiv:2307.06350*, 2023. 31

[336] X. Hu, R. Wang, Y. Fang *et al.*, "ELLA: Equip diffusion models with LLM for enhanced semantic alignment," *arXiv:2403.05135*, 2024. 31

[337] Y. Niu, M. Ning, M. Zheng *et al.*, "WISE: A world knowledge-informed semantic evaluation for text-to-image generation," *arXiv:2503.07265*, 2025. 31

[338] Z. Huang, Y. He, J. Yu *et al.*, "VBench: Comprehensive benchmark suite for video generative models," *arXiv:2311.17982*, 2023. 31

[339] T. Xie, D. Zhang, J. Chen *et al.*, "OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments," *arXiv:2404.07972*, 2024. 31

[340] Q. Sun, Z. Liu, C. Ma *et al.*, "ScienceBoard: Evaluating multimodal autonomous agents in realistic scientific workflows," *arXiv:2505.19897*, 2025. 31

[341] C. Rawles, S. Clinckemaillie, Y. Chang *et al.*, "AndroidWorld: A dynamic benchmarking environment for autonomous agents," *arXiv:2405.14573*, 2024. 31

[342] Y. Xu, X. Liu, X. Sun *et al.*, "AndroidLab: Training and systematic benchmarking of Android autonomous agents," *arXiv:2410.24024*, 2024. 31

[343] C. Rawles, A. Li, D. Rodriguez *et al.*, "Android in the wild: A large-scale dataset for Android device control," *arXiv:2307.10088*, 2023. 31

[344] W. Li, W. Bishop, A. Li *et al.*, "On the effects of data scale on UI control agents," *arXiv:2406.03679*, 2024. 31

[345] H. He, W. Yao, K. Ma *et al.*, "WebVoyager: Building an end-to-end web agent with large multimodal models," *arXiv:2401.13919*, 2024. 31

[346] S. Zhou, F. F. Xu, H. Zhu *et al.*, "WebArena: A realistic web environment for building autonomous agents," *arXiv:2307.13854*, 2023. 31

[347] J. Y. Koh, R. Lo, L. Jang *et al.*, "VisualWebArena: Evaluating multimodal agents on realistic visual web tasks," *arXiv:2401.13649*, 2024. 31

[348] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "LIBERO: Benchmarking knowledge transfer for lifelong robot learning," in *NeurIPS Datasets and Benchmarks Track*, 2023. 31

[349] X. Zhou, Y. Xu, G. Tie *et al.*, "LIBERO-PRO: Towards robust and fair evaluation of vision-language-action models beyond memorization," *arXiv:2510.03827*, 2025. 31

[350] S. Tao, A. Shukla, T.-k. Chan, H. Hao, F. Yang, X. Tang, and H. Su, "ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI," *arXiv:2410.00425*, 2024. 31

[351] S. Nasiriany, A. Maddukuri, L. Zhang *et al.*, "RoboCasa: Large-scale simulation of everyday tasks for generalist robots," *arXiv:2406.02523*, 2024. 31

[352] T. Chen, Z. Chen, B. Chen *et al.*, "RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv:2506.18088*, 2025. 31

[353] J. X. Yang, R. Dagli, A. Zook *et al.*, "RoboLab: A high-fidelity simulation benchmark for analysis of task generalist policies," *arXiv:2604.09860*, 2026. 31

[354] T. Chen, Y. Chen, Z. Li *et al.*, "RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies," *arXiv:2607.04434*, 2026. 31

[355] S. Zhang, Z. Xu, P. Liu *et al.*, "VLABench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks," *arXiv:2412.18194*, 2024. 31

[356] C. Li, R. Zhang, J. Wong *et al.*, "BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation," *arXiv:2403.09227*, 2024. 31

[357] Y. Jia, G. Wang, Y. Dong *et al.*, "DISCOVERSE: Efficient robot simulation in complex high-fidelity environments," *arXiv:2507.21981*, 2025. 31

[358] Y. Jiang, A. Gupta, Z. Zhang *et al.*, "VIMA: General robot manipulation with multimodal prompts," *arXiv:2210.03094*, 2022. 31

[359] L. Chen, J. Li, X. Dong *et al.*, "Are we on the right way for evaluating large vision-language models?" *arXiv:2403.20330*, 2024. 31

[360] A. Kamath, K.-W. Chang, R. Krishna *et al.*, "GenEval 2: Addressing benchmark drift in text-to-image evaluation," *arXiv:2512.16853*, 2025. 31

[361] H. Wang, H. Li, Q. Mang *et al.*, "Do androids dream of breaking the game? systematically auditing AI agent benchmarks with BenchJack," *arXiv:2605.12673*, 2026. 31

[362] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi, C. Wang, S. Chen, F. Pereira, W.-C. Kang, and D. Z. Cheng, "Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory," *arXiv:2511.20857*, 2025. 31

[363] Y. Wang, Z. Zhang, M. Chi, K. Yu, Y. Li, M. Peng, B. Tong, C. Zhang, Y. Zhou, and J. Li, "EvoMemBench: Benchmarking agent memory from a self-evolving perspective," in *NeurIPS*, 2026. 31

[364] Z. Ma, J. Liu, H. Tang, Y. Ma, R. Wang, Y. Yuan, Y. Zheng, and J. Hao, "Benchmarking continual agent memory for online learning, transfer, and forgetting," in *ICLR Workshops*, 2026. 31

[365] J. Zheng, X. Cai, Q. Li, D. Zhang, Z.-Z. Li, Y. Zhang, L. Song, and Q. Ma, "LifelongAgentBench: Evaluating LLM agents as lifelong learners," *arXiv:2505.11942*, 2025. 31

[366] Y. Cai, Y. Hao, J. Zhou, H. Yan, Z. Lei, R. Zhen, Z. Han, Y. Yang, J. Li, and Q. Pan, "Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark," *arXiv:2508.19005*, 2025. 31

[367] Y. Shu, B. Jiménez Gutiérrez, P. Jonnalagedda, Y. Yao, H. Sun, and Y. Su, "AgentCL: Toward rigorous evaluation of continual learning in language agents," *arXiv:2606.02461*, 2026. 31

[368] X. Yang, X. Zhang, W. Li, W. Liu, J. Sheng, Y. Hua, W. Yin, T. Fang, C. Shen, and X. Wang, "PATH-Bench: Path-dependent evaluation of lifelong agents," *arXiv:2608.01149*, 2026. 31

[369] X. Chen, Y. Huang, Y. Xu, Z. Zhao, E. McQuinn, J. Liu, and H. Yao, "Are self-evolving agents continual learners?" *arXiv:2610.25858*, 2026. 31

[370] Z. Ke, V. Patil, H. Shi, Y. Li, Y. Liu, S. Shekkizhar, A. Koul, J. Wang, X. P. Nguyen, S. Yavuz, M. Bansal, and S. Joty, "EvoHarness-Bench: Can your agents keep pace with an evolving harness?" *arXiv:2609.04280*, 2026. 31

[371] L. Huang, C. Yang, H. Zhou, H. Song, Z. Chen, R. Le, Y. Song, W. X. Zhao, and T. Zhang, "Evo-Bench: Can language models improve agent harness?" *arXiv:2608.09096*, 2026. 31

[372] M. Lin, J. Wu, Z. Wang, Z. Shi, Y. Sang, B. He, Z. Liu, T. Wei, Z. Wu, Z. Zhang, D. Wang, X. Zhang, B. Dumoulin, C. Xie, Y. Zhou, S. Wang, and H. Lu, "Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents," *arXiv:2605.30621*, 2026. 31

[373] X. Gao, C. Hu, H. Chen, P. Yao, Z. Wang, Y. Bai, Z. Wu, Y. Han, X. Cong, J. Gui, Y. Deng, and T. Li, "EvoAgentBench: Benchmarking agent self-evolution via ability transfer," *arXiv:2607.05202*, 2026. 31

[374] S. Jiang, L. Ma, Z. Hong, K. Wang, Z. Lu, T. Wang, S. Chen, J. Zhang, T. Pan, W. Li, J. Liang, and Y. Xiao, "SEA-Eval: A benchmark for evaluating self-evolving agents beyond episodic assessment," *arXiv:2604.08988*, 2026. 31

[375] B. Deng, K. Zhou, L. Guo, C. Tao, X. Chen, C. Xie, R. Liang, F. Chen, and C. Zhang, "FinEvo-Bench: A longitudinal benchmark for self-evolving agents in professional financial workflows," *arXiv:2608.06144*, 2026. 31

[376] Y. Zhu, Z. Deng, L. Wang, J. Lu, J. Wang, Y. Li, J. Shen, and C. Ran, "FinEvolveBench: A benchmark for self-evolving agents on low-repetition tasks with implicit rewards," *arXiv:2606.06960*, 2026. 31

[377] L. Zhou, Z. Liu, X. Qu, C. Liu, Y. Liu, Y. Yu, J. Xu, X. Wu, B. Qian, X. Chen, Y. Zheng, and J. Hu, "GDPevo: Evaluating agent self-evolution on real business tasks," *arXiv:2608.03764*, 2026. 31

[378] H. Lin, C. Liu, X. Bai, X. Jin, Y. Li, N. Zheng, and X. Cao, "Beyond endpoint performance: Process-level evaluation of self-evolving agents," *arXiv:2609.24663*, 2026. 31

[379] Y. Hu, Y. Wang, and J. McAuley, "Evaluating memory in LLM agents via incremental multi-turn interactions," in *ICLR*, 2026. 31

[380] C. Liu, Y. Yang, S. X. Pu, Y. Liu, L. Long, Y. Guo, N. Chen, Z. Weng, Y. Zhong, A. Zong, E. Kochkina, S. Kaur, C. Smiley, X. Liu, J. Zou, S. Liu, Y. Bu, S. Peng, and X. Wang, "WorldMemArena: Evaluating multimodal agent memory through action–world interaction," in *NeurIPS*, 2026. 31

[381] D. Wu, Z. Ji, A. Kawatkar, B. Kwan, J.-C. Gu, N. Peng, and K.-W. Chang, "LongMemEval-V2: Evaluating long-term agent memory toward experienced colleagues," *arXiv:2605.12493*, 2026. 31

[382] Z. Yao, Z. Huang, Y. Zeng, J. Han, H. Xu, C.-M. Feng, J. Ma, and W. Zuo, "CGL: Advancing continual GUI learning via reinforcement fine-tuning," *arXiv:2603.02951*, 2026. 31

[383] Z. Liu, B. Kang, H. Yuan, Z. Zhao, W. Li, Y. Zhu, and T. Feng, "Continual GUI agents," *arXiv:2601.20732*, 2026. 31

[384] Q. Wei, Y. Yang, S. Wang, J. Chen, B. Wang, J. Wang, S. Chen, Z. Li, Y. Shi, Y. Tang, W. Wang, Y. Yu, C. Fu, Q. Li, and Y.-F. Zhang, "Agentic-MME: What agentic capability really brings to multimodal intelligence?" *arXiv:2604.03016*, 2026. 31

[385] G. Liu, P. Zhao, Y. Liang, Q. Luo, S. Tang, Y. Chai, W. Lin, H. Xiao, W. Wang, S. Chen *et al.*, "MemGUI-Bench: Benchmarking memory of mobile GUI agents in dynamic environments," *arXiv:2602.06075*, 2026. 31, 33

[386] X. Che, J. Xiong, Y. Ge, X. Lei, S. Li, H. Yan, H. Li, Y. Zhang, Z. Bai, J. Hao *et al.*, "MMG2Skill: Can agents distill in-the-wild guides into self-evolving skills?" *arXiv:2606.01993*, 2026. 31, 33

[387] D. Ashok, A. Shen, A. H. Feng, C. Khanna, J. R. Huang, R. Sarmukaddam, S. B. Natarajan, X. Cui, X. Zhang, T. Yen, H. Namkoong, J. May, and J. Thomason, "GameBoyWorlds: A testbed for self-improvement in embodied video games," *arXiv:2609.32093*, 2026. 31, 33

[388] Z. Li, J. Yu, J. Zhang, L. Niu, Z. Qi, J. Zheng, X. Chen, D. Liu, Y. Zhang, H. Wang, and L. Yi, "Where is physical RSI capability from? An observational scaling law of LLM-as-policy," *arXiv:2610.26848*, 2026. 31, 33

[389] W. Huang, J. Bi, S. Yang, R. Si, R. Ni, Q. Wang, S. Jiang, Y. Zhang, F. Ruan, K. Zhang, Y. Huang, S. Gong, and D. Zhou, "Learning the task or learning the verifier? Rethinking criteria exposure in embodied self-improvement," *arXiv:2611.08318*, 2026. 32

[390] J. Zhang, J. Wang, Z. Yu, Y. Cai, W. Chow, C. Choy, Z. Dou, H. Hua, J. Li, Z. Liu, C. Wang, S. Wei, S. Yang, Y. Yuan, X. Zhan, Y. Zhang, S. Zheng, X. Li, L. Liu, W. Matusik, Z. Tu, and W. Wang, "FlexRSI: Benchmarking recursive self-improvement of frontier agents," *arXiv:2611.08946*, 2026. 32, 33

[391] L. Li, Y. Wei, Z. Xie, X. Yang, Y. Song, P. Wang, C. An, T. Liu, S. Li, B. Y. Lin, L. Kong, and Q. Liu, "VL-RewardBench: A challenging benchmark for vision-language generative reward models," in *CVPR*, 2025. 32, 33

[392] M. Yasunaga, L. Zettlemoyer, and M. Ghazvininejad, "Multimodal RewardBench: Holistic evaluation of reward models for vision language models," *arXiv:2502.14191*, 2025. 32, 33

[393] Y. Wei, L. Yao, L. Li *et al.*, "Video understanding reward modeling: A robust benchmark and performant reward models," *arXiv:2605.07872*, 2026. 32, 33

[394] G. Lim, Y. Koo, G. Jang *et al.*, "Androidflux: Evaluating and reward modeling failure recovery capabilities in mobile use multimodal agents," *arXiv:2610.34384*, 2026. 32, 33

[395] J. Xu, "Don't read the log: Execution traces contaminate verifiers in video-generation agents," *arXiv:2609.28564*, 2026. 32, 33

[396] J. Yao, Y. Wang, A. Zhang, Z. Sun, S. Wang, L. Mei, Y. Ge, and S. Liu, "Multimodal reward hacking in reinforcement learning," *arXiv:2607.09492*, 2026. 32, 33

[397] Z. Chen, Z. Wen, Y. Du, Y. Zhou, C. Cui, S. Han, J. Weng, C. Wang, Z. Tong, L. Huang, C. Chen, H. Tu, Q. Ye, Z. Zhu, Y. Zhang, J. Zhou, Z. Zhao, R. Rafailov, C. Finn, and H. Yao, "MJ-Bench: Is your multimodal reward model really a good judge for text-to-image generation?" in *NeurIPS Datasets and Benchmarks Track*, 2025. 32, 33

[398] S. S. Li, J. He, J. Chen *et al.*, "IsoVRBench: Video reward modeling without human feedback," *arXiv:2610.17234*, 2026. 32, 33

[399] Z. Zhang, S. Zhang, Y. Zhan, Y. Luo, Y. Wen, and D. Tao, "Confronting reward overoptimization for diffusion models: A perspective of inductive and primacy biases," in *ICML*, 2024. 32, 33

[400] Y. Hong, K.-C. Kao, H. Zhou, and C.-J. Hsieh, "Understanding reward hacking in text-to-image reinforcement learning," *arXiv:2601.03468*, 2026. 32, 33

[401] L. Mao, J. Cao, L. He, W. Xu, Y. Yan, H. Jin, Z. Zhang, J. Tang, S. Jin, Z. Liu *et al.*, "RobotAgentBench: Towards open-world robot autonomy," *arXiv:2610.34649*, 2026. 33

[402] R. Zhang, Y. Huang, Y. Pan, C. Shen, Z. Liu, T. Xie, H. Lei, and L. Duan, "Mastering PokeGym: Graph-guided multimodal evolution at test time," *arXiv:2604.08340*, 2026. 33

[403] K. Zhang, D. Liu, Q. Zhao, J. Hou, X. Zhang, Q. Xie, M. Liu, and Y. Li, "GameVerse: Can vision-language models learn from video-based reflection?" *arXiv:2603.06656*, 2026. 33

[404] M. Lin, S. Qian, Y. Liu, Y.-H. Huang, Y. Wang, W. Huang, Y. Li, F. Zhang, Z. Hu, L. Zhu, X. Wang, and X. Qi, "OmniGameArena: A unified UE5 benchmark for VLM game agents with improvement dynamics," *arXiv:2606.09826*, 2026. 33

[405] B. Xu, J. Yang, S. Chen, X. Zhou, J. Qiu, Q. Yang, Z. Wang, F. Wu, and Z. Wen, "SelfImprove-Bench: Evaluating generalization in self-improving agents," *arXiv:2611.02797*, 2026. 33

[406] J. Huang, Y. Chen, R. Liu, K. He, X. He, R. Huang, Z. Zheng, Z. Zhang, and Y. Zhong, "EmbodiedCLBench: Evaluating continual learning for self-evolving embodied agents," *arXiv:2610.13021*, 2026. 33

[407] J. Song, Z. Chen, B. Zeng, J. Tang, Y. Hou, J. Jing, Z. Xie, and L. Zhang, "MC-EvoBench: Auditing experience-driven self-improvement in Minecraft agents," *arXiv:2610.13829*, 2026. 33
[408] B. Chen and T. Pan, "Self-verifier capture in unified visual generators: Diagnosing and repairing closed-loop self-reward," *arXiv:2610.16500*, 2026. 32, 33
[409] C. Zhou, "More convincing, not more correct: Self-play reward hacking of reference-free LLM judges," *arXiv:2607.05904*, 2026. 32, 33
[410] Z. Qi, X. Liu, I. L. Lv, Z. Wang, J. Tang, and Y. Dong, "WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning," in *ICLR*, 2025. 32
[411] D. Chen, R. Chen, S. Zhang, Y. Liu, Y. Wang, H. Zhou, Q. Zhang, Y. Wan, P. Zhou, and L. Sun, "MLLM-as-a-Judge: Assessing multimodal LLM-as-a-Judge with vision-language benchmark," in *ICML*, 2024. 33
[412] W. Wang, Z. Gao, L. Chen, Z. Chen, J. Zhu, X. Zhao, Y. Liu, Y. Cao, S. Ye, X. Zhu, L. Lu, H. Duan, Y. Qiao, J. Dai, and W. Wang, "VisualPRM: An effective process reward model for multimodal reasoning," *arXiv:2503.10291*, 2025. 33
[413] I. W. Lin, Y. Hu, S. Li, S. Geng, P. W. Koh, L. Zettlemoyer, T. Althoff, and M. Ghazvininejad, "Self-improving VLM judges without human annotations," *arXiv:2512.05145*, 2025. 33
[414] P. Schroeder, T. Weng, K. Schmeckpeper *et al.*, "SOLE-R1: Video-language reasoning as the sole reward for on-robot reinforcement learning," *arXiv:2603.28730*, 2026. 33
[415] H. Wang, "Who sets the pace of judge hacking? judge capability versus the policy–judge gap," *arXiv:2610.15778*, 2026. 33
[416] T. Xue, C. Peng, M. Huang, L. Guo, T. Han, H. Wang, J. Wang, X. Zhang, X. Yang, D. Zhao, J. Ding, X. Ma, Y. Xie, P. Pei, X. Cai, and X. Qiu, "EvoCUA: Evolving computer use agents via learning from scalable synthetic experience," *arXiv:2601.15876*, 2026. 32
[417] S. Shao, Q. Ren, C. Qian, B. Wei, D. Guo, J. Yang, X. Song, L. Zhang, W. Zhang, D. Liu, and J. Shao, "Your agent may misevolve: Emergent risks in self-evolving LLM agents," in *ICLR*, 2026. 34
[418] W. Zhao, Y. Zhang, Y. Wang, Y. Deng, Y. Zhao, X. Zhi, Y. Huang, H. He, W. Che, B. Qin, and T. Liu, "On safety risks in experience-driven self-evolving agents," in *Findings of ACL*, 2026. 34
[419] M. Cui, Z. Liu, Y. Zhu, Z. Zhang, and M. Jiang, "Learning from runtime feedback through failure-bank self-evolution for vision-language-action models," *arXiv:2609.39820*, 2026. 34